

**TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC THÀNH PHỐ HỒ CHÍ
MINH**

KHOA CÔNG NGHỆ THÔNG TIN

୨୨୪୪

KHÓA LUẬN TỐT NGHIỆP

**Nghiên cứu và thiết kế hệ thống
ngoại vi tốc độ cao dựa trên kỹ thuật
Double Data Rate (DDR) tích hợp
trên SoC RISC-V.**

GVHD: ThS. Hồ Lê Minh Toàn

SV THỰC HIỆN: Ngô Nguyễn Gia Huy 23DH114340

TP. Hồ Chí Minh, tháng ... năm ...

TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

**Nghiên cứu và thiết kế hệ thống
ngoại vi tốc độ cao dựa trên kỹ thuật
Double Data Rate (DDR) tích hợp
trên SoC RISC-V.**

GIẢNG VIÊN HƯỚNG DẪN: ThS. Hồ Lê Minh Toàn

SINH VIÊN THỰC HIỆN: Ngô Nguyễn Gia Huy

TP. HỒ CHÍ MINH – .../...

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện luận văn tốt nghiệp, em đã nhận được nhiều sự quan tâm, hướng dẫn và hỗ trợ quý báu từ thầy cô, gia đình và bạn bè. Đây là nguồn động viên to lớn giúp em hoàn thành luận văn này.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến Ths Hồ Lê Minh Toàn, người đã tận tình định hướng, góp ý chuyên môn và đồng hành cùng em trong suốt quá trình nghiên cứu. Những nhận xét và chỉ dẫn quý báu của thầy/cô đã giúp em từng bước hoàn thiện nội dung cũng như phương pháp thực hiện luận văn.

Em xin chân thành cảm ơn quý thầy cô trong khoa và nhà trường đã truyền đạt những kiến thức, kỹ năng cần thiết trong suốt thời gian học tập. Những kiến thức đó không chỉ là nền tảng để em thực hiện đề tài mà còn là hành trang quan trọng cho công việc và quá trình học tập sau này.

Em cũng xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn quan tâm, động viên, chia sẻ và tạo điều kiện thuận lợi để em có thể tập trung hoàn thành luận văn.

Mặc dù đã cố gắng nghiêm túc trong quá trình thực hiện, luận văn khó tránh khỏi những thiếu sót. Em kính mong nhận được sự góp ý của quý thầy cô và các bạn để nội dung luận văn được hoàn thiện hơn.

Em xin chân thành cảm ơn!

LỜI CAM ĐOAN

Tôi xin cam đoan khóa luận tốt nghiệp này là kết quả nghiên cứu do cá nhân tôi thực hiện dưới sự hướng dẫn của giảng viên. Toàn bộ nội dung, kết quả và số liệu được trình bày trong khóa luận đều trung thực, có nguồn gốc rõ ràng và chưa từng được sử dụng trong bất kỳ công trình nào nhằm mục đích bảo vệ một học vị khác.

Mọi kiến thức, nhận định, số liệu và kết quả nghiên cứu của các tác giả khác được sử dụng trong khóa luận đều được trích dẫn và ghi nguồn đầy đủ theo quy định. Tôi không sao chép, giả mạo hoặc sử dụng kết quả của người khác như kết quả nghiên cứu của bản thân.

Tôi xin hoàn toàn chịu trách nhiệm trước Nhà trường và pháp luật về tính trung thực, tính chính xác và toàn bộ nội dung của lời cam đoan này.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

TÓM TẮT KHÓA LUẬN

Các nền tảng hệ thống trên chip, hay System-on-Chip (SoC), phụ thuộc nhiều vào các giao tiếp truyền thông nối tiếp như UART, SPI và I2C để tích hợp các thiết bị ngoại vi. Tuy nhiên, phương thức truyền dữ liệu đơn tốc độ truyền thông làm hạn chế thông lượng dữ liệu, trong khi việc tăng tần số xung clock thường làm gia tăng công suất tiêu thụ và gây khó khăn cho quá trình đáp ứng các ràng buộc thời gian.

Khóa luận này trình bày quá trình thiết kế và hiện thực một kiến trúc truyền thông SoC dạng mô-đun dựa trên RISC-V, có tích hợp cơ chế truyền dữ liệu tốc độ kép (Double Data Rate) DDR. Bằng cách khai thác đồng thời cạnh lên và cạnh xuống của tín hiệu clock, hệ thống được đề xuất nâng cao băng thông truyền thông mà không yêu cầu tăng tần số clock của hệ thống. Kiến trúc tích hợp các bộ điều khiển UART, SPI và I2C tùy chỉnh, cùng hoạt động trên một bus nội bộ dùng chung.

Kết quả đánh giá xác nhận hiệu quả của việc tích hợp cơ chế truyền thông theo định hướng DDR vào nền tảng RISC-V dạng mô-đun. Giải pháp cung cấp một nền tảng có khả năng mở rộng và đạt hiệu quả cao cho các ứng dụng Internet vạn vật (IoT) và điện toán biên hiện đại.

MỤC LỤC

LỜI CẢM ƠN	I
LỜI CAM ĐOAN	II
TÓM TẮT KHÓA LUẬN	III
MỤC LỤC	IV
DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT	VII
DANH MỤC CÁC BẢNG	XIX
DANH MỤC CÁC HÌNH VẼ, BIỂU ĐỒ	XX
1 GIỚI THIỆU	2
1.1 Lý do chọn đề tài	2
1.2 Mục tiêu nghiên cứu	4
1.2.1 Mục tiêu tổng quát	4
1.2.2 Mục tiêu cụ thể	4
1.3 Đối tượng nghiên cứu	5
1.4 Phạm vi và giới hạn nghiên cứu	5
1.4.1 Phạm vi nghiên cứu	5
1.4.2 Giới hạn đề tài	6
2 CƠ SỞ LÝ THUYẾT	7
2.1 Hệ thống SoC trên nền tảng RISC-V	7
2.2 Lỗi xử lý RISC-V PicoRV32	8
2.3 Sự khác biệt giữa truyền dữ liệu SDR và DDR	10
2.4 Giao tiếp UART	12
2.5 Giao tiếp I2C	15
2.6 Giao tiếp SPI	17
2.7 Giao tiếp AXI4-Lite và kết nối giữa các mô-đun	19
2.8 Timing và giới hạn tần số hoạt động	21

2.9	Công cụ Vivado	23
2.10	Nền tảng FPGA Xilinx 7 Series và thiết bị mục tiêu	24
3	CÁC CÔNG TRÌNH NGHIÊN CỨU LIÊN QUAN	27
3.1	Tổng quan các hướng nghiên cứu về SoC trên nền RISC-V	27
3.2	Các giải pháp chuẩn hóa giao tiếp ngoại vi với bus AXI4-Lite	27
3.3	Nghiên cứu về kỹ thuật DDR và quản lý Timing ở tần số cao	28
3.4	So sánh tổng hợp và định hướng của đề tài	29
4	MÔ HÌNH VÀ PHƯƠNG PHÁP ĐỀ XUẤT	32
4.1	Tổng quan kiến trúc hệ thống đề xuất	32
4.2	Thiết kế chi tiết các khối chức năng	33
4.2.1	Lỗi xử lý PicoRV32 và bộ giải mã địa chỉ	33
4.2.2	Bộ chuyển đổi bus picorv32_axi_adapter	35
4.2.3	Khối xử lý song song và truyền dữ liệu DDR	38
4.2.4	Các khối ngoại vi chuẩn giao tiếp nối tiếp (UART, SPI, I2C)	42
4.3	Phương pháp xử lý Timing và ràng buộc XDC trên Vivado	49
4.4	Quy trình đóng gói và mô phỏng kiểm chứng mức RTL	52
4.4.1	Đóng gói các khối ngoại vi và tích hợp hệ thống (SoC Packaging)	53
4.4.2	Xây dựng môi trường mô phỏng Testbench	55
4.4.3	Kịch bản mô phỏng và Kết quả kiểm chứng (Simulation Scenarios)	55
5	KẾT QUẢ THỰC NGHIỆM	59
5.1	Kết quả thực hiện	59
5.2	Kết quả thử nghiệm	59
5.2.1	Môi trường và phương pháp thử nghiệm	59
5.2.2	Kiểm chứng bộ điều khiển UART	60
5.2.3	Kiểm chứng bộ điều khiển SPI	62
5.2.4	Kiểm chứng bộ điều khiển I ² C	64
5.2.5	Kiểm chứng khối gia tốc xử lý song song DDR	68
5.2.6	Kiểm chứng mức tích hợp hệ thống (SoC Integration)	72
5.2.7	Kết quả tổng hợp và thực thi phần cứng (RTL Implementation)	76
5.3	Đánh giá và thảo luận	77
5.4	Hạn chế của đề tài	78

6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	79
6.1	Kết luận	79
6.2	Những kết quả đạt được	79
6.3	Những hạn chế	80
6.4	Hướng phát triển trong tương lai	81
7	CÁC CÔNG TRÌNH NGHIÊN CỨU ĐÃ THAM GIA	82
	TÀI LIỆU THAM KHẢO	83

DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
1	ASIC	Application-Specific Integrated Circuit – vi mạch tích hợp chuyên dụng
2	BRAM	Block Random Access Memory – bộ nhớ RAM khối trên FPGA
3	CAD	Computer-Aided Design – thiết kế có sự hỗ trợ của máy tính
4	CDC	Clock Domain Crossing – chuyển miền xung clock
5	DDR	Double Data Rate – kỹ thuật truyền dữ liệu trên cả hai cạnh xung clock
6	DRC	Design Rule Check – kiểm tra quy tắc thiết kế
7	EDA	Electronic Design Automation – tự động hóa thiết kế điện tử
8	EMI	Electromagnetic Interference – nhiễu điện từ
9	FPGA	Field-Programmable Gate Array – mảng cổng logic có thể lập trình
10	HDL	Hardware Description Language – ngôn ngữ mô tả phần cứng

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
11	IDDR	Input Double Data Rate – khối nhận dữ liệu DDR đầu vào
12	I/O	Input/Output – đầu vào/đầu ra
13	IP Core	Intellectual Property Core – lõi sở hữu trí tuệ phần cứng
14	ODDR	Output Double Data Rate – khối phát dữ liệu DDR đầu ra
15	RISC-V	Reduced Instruction Set Computer Five – kiến trúc tập lệnh RISC-V
16	RTL	Register-Transfer Level – mức truyền thanh ghi
17	SDC	Synopsys Design Constraints – định dạng ràng buộc thiết kế Synopsys
18	SDR	Single Data Rate – kỹ thuật truyền dữ liệu trên một cạnh xung clock
19	SoC	System-on-Chip – hệ thống trên chip
20	STA	Static Timing Analysis – phân tích timing tĩnh

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
21	XDC	Xilinx Design Constraints – định dạng ràng buộc thiết kế Xilinx
22	f	Tần số hoạt động của xung clock
23	$P_{dynamic}$	Công suất tiêu thụ động của hệ thống
24	t_H	Thời gian giữ dữ liệu (Hold time)
25	t_{SU}	Thời gian thiết lập dữ liệu (Setup time)
26	CPU	Central Processing Unit – bộ xử lý trung tâm
27	MCU	Microcontroller Unit – bộ vi điều khiển
28	W	Độ rộng của bus dữ liệu
29	f_{clk}	Tần số của xung clock
30	R_{SDR}	Tốc độ truyền dữ liệu lý thuyết của phương thức SDR

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
31	R_{DDR}	Tốc độ truyền dữ liệu lý thuyết của phương thức DDR
32	T_{clk}	Chu kỳ của xung clock
33	t_{cq}	Độ trễ từ cạnh clock đến đầu ra của thanh ghi
34	t_{logic}	Độ trễ của khối logic tổ hợp
35	t_{setup}	Thời gian thiết lập dữ liệu của thanh ghi đích
36	t_{skew}	Độ lệch thời gian giữa các tín hiệu clock
37	ACK	Acknowledgement – tín hiệu xác nhận giao dịch thành công
38	AHB	Advanced High-performance Bus – bus hiệu năng cao của AMBA
39	ARM	Advanced RISC Machines – kiến trúc bộ xử lý ARM
40	AXI	Advanced eXtensible Interface – giao diện bus mở rộng của kiến trúc AMBA

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
41	AXI4-Lite	Phiên bản AXI4 gọn nhẹ dành cho giao tiếp ngoại vi ánh xạ bộ nhớ
42	ACLK	Xung nhịp hoạt động của giao diện AXI4-Lite
43	ARESETn	Tín hiệu reset tích cực mức thấp của giao diện AXI4-Lite
44	AWADDR	Write Address – địa chỉ của giao dịch ghi AXI4-Lite
45	AWVALID	Tín hiệu Master báo địa chỉ ghi hợp lệ
46	AWREADY	Tín hiệu Slave sẵn sàng nhận địa chỉ ghi
47	WDATA	Write Data – dữ liệu ghi trên kênh W của AXI4-Lite
48	WSTRB	Write Strobe – mặt nạ byte hợp lệ của dữ liệu ghi
49	WVALID	Tín hiệu Master báo dữ liệu ghi hợp lệ
50	WREADY	Tín hiệu Slave sẵn sàng nhận dữ liệu ghi

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
51	BRESP	Write Response – trạng thái phản hồi của giao dịch ghi
52	BVALID	Tín hiệu Slave báo phản hồi ghi hợp lệ
53	BREADY	Tín hiệu Master sẵn sàng nhận phản hồi ghi
54	ARADDR	Read Address – địa chỉ của giao dịch đọc AXI4-Lite
55	ARVALID	Tín hiệu Master báo địa chỉ đọc hợp lệ
56	ARREADY	Tín hiệu Slave sẵn sàng nhận địa chỉ đọc
57	RDATA	Read Data – dữ liệu đọc trả về trên kênh R
58	RRESP	Read Response – trạng thái phản hồi của giao dịch đọc
59	RVALID	Tín hiệu Slave báo dữ liệu đọc hợp lệ
60	RREADY	Tín hiệu Master sẵn sàng nhận dữ liệu đọc

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
61	CAN	Controller Area Network – mạng truyền thông điều khiển
62	CS	Chip Select – tín hiệu lựa chọn thiết bị SPI
63	DMA	Direct Memory Access – cơ chế truy cập bộ nhớ trực tiếp
64	DSP	Digital Signal Processing – khối xử lý tín hiệu số
65	FF	Flip-Flop – phần tử lưu trữ một bit dữ liệu
66	GPIO	General-Purpose Input/Output – cổng vào/ra đa dụng
67	I ² C	Inter-Integrated Circuit – bus truyền thông nối tiếp hai dây
68	IOLOGIC	Input/Output Logic – tài nguyên logic tại khối vào/ra FPGA
69	LUT	Look-Up Table – bảng tra cứu dùng để hiện thực logic tổ hợp
70	MISO	Master In Slave Out – đường dữ liệu từ thiết bị tổ đến thiết bị chủ SPI

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
71	MOSI	Master Out Slave In – đường dữ liệu từ thiết bị chủ đến thiết bị tớ SPI
72	NACK	Not Acknowledgement – tín hiệu không xác nhận giao dịch
73	PL	Programmable Logic – vùng logic lập trình được của Zynq
74	PS	Processing System – hệ thống xử lý cứng của Zynq
75	SCL	Serial Clock Line – đường xung clock của I ² C
76	SCLK	Serial Clock – đường xung clock của SPI
77	SDA	Serial Data Line – đường dữ liệu hai chiều của I ² C
78	SPI	Serial Peripheral Interface – giao tiếp ngoại vi nối tiếp đồng bộ
79	TX/RX	Transmit/Receive – đường truyền và nhận dữ liệu
80	UART	Universal Asynchronous Receiver/Transmitter – bộ truyền nhận nối tiếp bất đồng bộ

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
81	USB	Universal Serial Bus – bus nối tiếp đa dụng
82	mem_ready	Tín hiệu báo CPU rằng giao dịch bộ nhớ đã hoàn tất
83	ADC	Analog-to-Digital Converter – bộ chuyển đổi tín hiệu tương tự sang số
84	AMBA	Advanced Microcontroller Bus Architecture – kiến trúc bus cho vi điều khiển
85	CPHA	Clock Phase – pha lấy mẫu của xung clock trong giao tiếp SPI
86	CPOL	Clock Polarity – cực tính của xung clock trong giao tiếp SPI
87	DAC	Digital-to-Analog Converter – bộ chuyển đổi tín hiệu số sang tương tự
88	DRAM	Dynamic Random Access Memory – bộ nhớ truy cập ngẫu nhiên động
89	EEPROM	Electrically Erasable Programmable Read-Only Memory – bộ nhớ chỉ đọc có thể xóa và lập trình bằng điện
90	FIFO	First In, First Out – cấu trúc bộ đệm vào trước, ra trước

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
91	PHY	Physical Layer – lớp hoặc khối giao tiếp vật lý
92	RAM	Random Access Memory – bộ nhớ truy cập ngẫu nhiên
93	RTOS	Real-Time Operating System – hệ điều hành thời gian thực
94	SDRAM	Synchronous Dynamic Random Access Memory – bộ nhớ DRAM đồng bộ
95	VHDL	VHSIC Hardware Description Language – ngôn ngữ mô tả phần cứng VHDL
96	T_{ck}	Clock Period – chu kỳ xung clock
97	T_{suav}	Data valid to clock transition – thời gian thiết lập khả dụng tại đầu ra của thiết bị phát
98	T_{hav}	Clock transition to data invalid – thời gian giữ khả dụng tại đầu ra của thiết bị phát
99	T_{tcu}	Total clock uncertainty – tổng độ bất định của xung clock tại đầu ra của thiết bị phát
100	$T_{suav} + T_{hav}$	Data valid window – cửa sổ thời gian dữ liệu hợp lệ tại đầu ra

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
101	T_{su}	Input setup time – thời gian thiết lập yêu cầu tại đầu vào của thiết bị nhận
102	T_h	Input hold time – thời gian giữ yêu cầu tại đầu vào của thiết bị nhận
103	$T_{su} + T_h$	Required data window at input – cửa sổ dữ liệu yêu cầu tại đầu vào
104	WNS	Worst Negative Slack – độ dư thời gian thiết lập xấu nhất
105	WHS	Worst Hold Slack – độ dư thời gian giữ xấu nhất
106	WPWS	Worst Pulse Width Slack – độ dư độ rộng xung xấu nhất
107	TNS	Total Negative Slack – tổng độ dư thời gian âm
108	PPA	Power, Performance, Area – công suất, hiệu năng và diện tích
109	IOB	Input/Output Block – khối tài nguyên vào/ra của FPGA
110	BUFG	Global Clock Buffer – bộ đệm xung clock toàn cục

STT	KÝ HIỆU, CHỮ VIẾT TẮT	Ý NGHĨA
111	FSM	Finite-State Machine – máy trạng thái hữu hạn
112	LSB	Least Significant Bit – bit có trọng số thấp nhất
113	MSB	Most Significant Bit – bit có trọng số cao nhất
114	MMCM	Mixed-Mode Clock Manager – khối quản lý xung clock hỗn hợp
115	PLL	Phase-Locked Loop – vòng khóa pha
116	RAMB36E1	Khối Block RAM 36 Kb chuyên dụng trên FPGA 7 Series
117	OLOGIC	Output Logic – tài nguyên logic ngõ ra tại IOB
118	RV32I	Tập lệnh RISC-V cơ bản 32 bit dành cho số nguyên
119	P_{static}	Công suất tĩnh do dòng rò của hệ thống
120	P_{total}	Tổng công suất tiêu thụ trên chip
121	AI	Artificial Intelligence – trí tuệ nhân tạo
122	TinyML	Tiny Machine Learning – học máy nhẹ trên thiết bị nhúng

DANH MỤC CÁC BẢNG

STT	BẢNG	TRANG
1	Bảng 2.1: So sánh khái quát giữa SDR và DDR	11
2	Bảng 2.2: Thông số cấu hình và tài nguyên của FPGA Artix-7 mục tiêu	26
3	Bảng 3.1: Tổng hợp so sánh phạm vi các nghiên cứu liên quan và định hướng đề tài	30
4	Bảng 4.1: Bản đồ phân bố không gian địa chỉ AXI4-Lite của hệ thống SoC	35
5	Bảng 5.1: Tổng hợp sử dụng tài nguyên phần cứng thực tế trên FPGA Artix-7 (xc7a200tfbg676-2L)	76
6	Bảng 5.2: Tổng hợp kết quả phân tích định thời của hệ thống SoC tại tần số 50 MHz	77
7	Bảng 5.3: Tổng hợp công suất tiêu thụ hệ thống SoC tại tần số 50 MHz	77

DANH MỤC CÁC HÌNH VẼ, BIỂU ĐỒ

STT	HÌNH VẼ, BIỂU ĐỒ	TRANG
1	Hình 2.1: Kiến trúc tổng quan của hệ thống SoC RISC-V	7
2	Hình 2.2: Sự khác biệt giữa phương thức truyền dữ liệu SDR và DDR	11
3	Hình 2.3: Kiến trúc và luồng dữ liệu của giao tiếp UART	13
4	Hình 2.4: Dạng sóng truyền nhận dữ liệu của giao tiếp UART	14
5	Hình 2.5: Kiến trúc và trình tự truyền dữ liệu của giao tiếp I ² C	15
6	Hình 2.6: Dạng sóng truyền dữ liệu của giao tiếp I ² C	16
7	Hình 2.7: Kiến trúc và luồng dữ liệu song công của giao tiếp SPI	17
8	Hình 2.8: Dạng sóng truyền dữ liệu của giao tiếp SPI	18
9	Hình 2.9: Kiến trúc năm kênh truyền dữ liệu giữa AXI Master và AXI Slave	20
10	Hình 2.10: Giao diện phần mềm Vivado Design Suite	23
11	Hình 2.11: Bo mạch Zynq XC7Z030 tham khảo thuộc nền tảng Xilinx 7 Series	25
12	Hình 3.1: Quan hệ định thời Edge-aligned và Center-aligned tại phía nhận	28
13	Hình 4.1: Sơ đồ khối tổng thể kiến trúc SoC RISC-V đề xuất	33

STT	HÌNH VẼ, BIỂU ĐỒ	TRANG
14	Hình 4.2: Sơ đồ khối tham khảo của hệ thống SoC tích hợp lõi RISC-V	34
15	Hình 4.3: Sơ đồ RTL giao diện Native Memory của bộ điều khiển PicoRV32–AXI4-Lite	35
16	Hình 4.4: Sơ đồ RTL các kênh tín hiệu của bộ chuyển đổi AXI4-Lite	36
17	Hình 4.6: Sơ đồ RTL của khối xử lý song song và giao tiếp DDR	39

STT	HÌNH VẼ, BIỂU ĐỒ	TRANG
18	Hình 4.5: Máy trạng thái trong picorv32_axi_adapter	36
19	Hình 4.7: Giao tiếp giữa kết quả các lần tính toán và bộ phát DDR	40
20	Hình 4.8: Khởi tạo phần tử cứng ODDR trong AMD/Xilinx FPGA	41
21	Hình 4.9: Sơ đồ RTL của khối giao tiếp AXI4-Lite UART	43
22	Hình 4.10: Các đoạn mã RTL cốt lõi của mô-đun axi_uart_slave	43
23	Hình 4.11: Sơ đồ RTL của khối giao tiếp AXI4-Lite SPI	45
24	Hình 4.12: Các đoạn mã RTL cốt lõi của mô-đun axi_spi_slave	46
25	Hình 4.13: Sơ đồ RTL của khối giao tiếp AXI4-Lite I ² C	47
26	Hình 4.14: Các đoạn mã RTL cốt lõi của mô-đun axi_i2c_slave	48
27	Hình 4.15: Quan hệ định thời ngõ ra Edge-aligned và Center-aligned	50
28	Hình 4.16: Khai báo ràng buộc XDC hoàn chỉnh cho giao tiếp ngõ ra DDR	51
29	Hình 4.17: Sơ đồ RTL tổng thể SoC – lõi xử lý và AXI Crossbar	53
30	Hình 4.18: Sơ đồ RTL tổng thể SoC – ngoại vi và khối DDR	54

STT	HÌNH VẼ, BIỂU ĐỒ	TRANG
31	Hình 5.1: Dạng sóng kiểm tra tốc độ baud và cấu trúc khung truyền UART	60
32	Hình 5.2: Kết quả kiểm tra tốc độ baud và cấu trúc khung truyền UART	60
33	Hình 5.3: Dạng sóng truyền mẫu dữ liệu UART TX 0xA5	61
34	Hình 5.4: Kết quả truyền mẫu dữ liệu UART TX 0xA5	61
35	Hình 5.5: Dạng sóng kiểm tra trạng thái Ready/Busy của UART	61
36	Hình 5.6: Kết quả kiểm tra trạng thái Ready/Busy của UART	61
37	Hình 5.7: Dạng sóng kiểm tra chức năng nhận dữ liệu UART RX	62
38	Hình 5.8: Kết quả kiểm tra chức năng nhận dữ liệu UART RX	62
39	Hình 5.9: Dạng sóng kiểm tra CPOL và truyền nhận song công SPI	63
40	Hình 5.10: Kết quả kiểm tra CPOL và truyền nhận song công SPI	63
41	Hình 5.11: Dạng sóng kiểm tra thanh ghi Chip Select của SPI	64
42	Hình 5.12: Kết quả kiểm tra thanh ghi Chip Select của SPI	64
43	Hình 5.13: Dạng sóng đọc/ghi thanh ghi I ² C qua AXI	65

STT	HÌNH VẼ, BIỂU ĐỒ	TRANG
44	Hình 5.14: Kết quả đọc/ghi thanh ghi I ² C qua AXI	65
45	Hình 5.15: Dạng sóng kiểm tra điều kiện START của I ² C	66
46	Hình 5.16: Kết quả kiểm tra điều kiện START của I ² C	66
47	Hình 5.17: Dạng sóng truyền byte và nhận ACK của I ² C	66
48	Hình 5.18: Kết quả truyền byte và nhận ACK của I ² C	67
49	Hình 5.19: Dạng sóng truyền byte và nhận NACK của I ² C	67
50	Hình 5.20: Kết quả truyền byte và nhận NACK của I ² C	67
51	Hình 5.21: Dạng sóng kiểm tra điều kiện STOP của I ² C	68
52	Hình 5.22: Kết quả kiểm tra điều kiện STOP của I ² C	68
53	Hình 5.23: Dạng sóng kích hoạt đồng bộ các lần xử lý DDR	69
54	Hình 5.24: Kết quả kích hoạt đồng bộ các lần xử lý DDR	69
55	Hình 5.25: Dạng sóng kích hoạt độc lập từng lần xử lý DDR	70
56	Hình 5.26: Kết quả kích hoạt độc lập từng lần xử lý DDR	70

STT	HÌNH VẼ, BIỂU ĐỒ	TRANG
57	Hình 5.27: Dạng sóng cách ly dữ liệu giữa hai sườn xung DDR	71
58	Hình 5.28: Kết quả cách ly dữ liệu giữa hai sườn xung DDR	71
59	Hình 5.29: Dạng sóng so sánh độ trễ giữa kiến trúc 1 Lane và DDR 2 Lane	72
60	Hình 5.30: Kết quả đánh giá thông lượng và hệ số tăng tốc DDR	72
61	Hình 5.31: Dạng sóng truy cập và giải mã địa chỉ AXI của SoC	73
62	Hình 5.32: Kết quả truy cập và giải mã địa chỉ AXI của SoC	73
63	Hình 5.33: Dạng sóng UART và SPI hoạt động đồng thời	74
64	Hình 5.34: Kết quả UART và SPI hoạt động đồng thời	74
65	Hình 5.35: Dạng sóng UART, SPI và I ² C hoạt động đồng thời	74
66	Hình 5.36: Kết quả UART, SPI và I ² C hoạt động đồng thời	75
67	Hình 5.37: Dạng sóng các giao dịch AXI đan xen giữa nhiều ngoại vi	75
68	Hình 5.38: Kết quả các giao dịch AXI đan xen giữa nhiều ngoại vi	75

Chương 1

GIỚI THIỆU

1.1 Lý do chọn đề tài

Kiến trúc hệ thống trên chip, hay System-on-Chip (SoC), được sử dụng rộng rãi trong các hệ thống nhúng, thiết bị Internet vạn vật (IoT) và nền tảng điện toán biên nhờ khả năng tích hợp bộ xử lý, bộ nhớ, hệ thống liên kết và các bộ điều khiển ngoại vi trên cùng một nền tảng phần cứng. Cấu trúc này giúp giảm kích thước hệ thống và tạo điều kiện thuận lợi cho việc mở rộng thiết kế. Trong số các kiến trúc bộ xử lý hiện nay, RISC-V là một lựa chọn phù hợp cho nghiên cứu SoC nhờ tính mở, cấu trúc mô-đun và khả năng hỗ trợ xây dựng các nền tảng phần cứng tùy chỉnh [1].

Một SoC thực tế thường cần nhiều giao tiếp truyền thông để kết nối với cảm biến, bộ nhớ, màn hình và các mô-đun bên ngoài. UART, SPI và I²C là những lựa chọn phổ biến vì có cấu trúc đơn giản và được nhiều thiết bị nhúng hỗ trợ. UART thường được sử dụng cho mục đích gỡ lỗi và trao đổi dữ liệu bất đồng bộ; SPI hỗ trợ truyền song công với tốc độ cao; trong khi I²C phù hợp khi nhiều thiết bị tốc độ thấp dùng chung một bus. Các nghiên cứu về thiết kế SoC bằng Verilog và triển khai trên FPGA cho thấy các khối truyền thông nối tiếp vẫn giữ vai trò quan trọng trong hệ thống phần cứng nhúng [2–4].

Mặc dù các giao tiếp này đã được tích hợp trong nhiều vi điều khiển thương mại, cấu trúc bên trong thường cố định nên khó điều chỉnh cho mục đích nghiên cứu, giảng dạy hoặc phát triển SoC tùy chỉnh. Vì vậy, một nền tảng có khả năng tái sử dụng và được mô tả ở mức RTL là cần thiết, đặc biệt khi thiết kế cần được kiểm chứng, tổng hợp và thử nghiệm trên FPGA trước khi hướng đến hiện thực ASIC. Các công trình về thiết kế ngoại vi dựa trên bus AMBA đã cho thấy tầm quan trọng của một cấu trúc bus rõ ràng khi kết nối bộ xử lý với các khối phần cứng tùy chỉnh [6]. Bên cạnh đó, kiến trúc liên kết AXI cũng đã được nghiên cứu để kết nối các ngoại vi tùy chỉnh với bộ xử lý RISC-V [5].

Một vấn đề quan trọng khác trong truyền thông SoC là thông lượng dữ liệu. Với phương thức truyền đơn tốc độ, dữ liệu chỉ được phát hoặc lấy mẫu tại một cạnh xung clock. Việc tăng tần số clock có thể cải thiện băng thông nhưng đồng thời làm cho quá trình đáp ứng ràng buộc thời gian và kiểm soát công suất trở nên khó khăn hơn. Kỹ thuật truyền dữ liệu tốc độ kép (Double Data Rate – DDR) là một giải pháp phù hợp hơn nhờ khai thác đồng thời cạnh lên và cạnh xuống của xung clock. Cơ chế này cho phép nâng cao tốc độ truyền dữ liệu mà không cần trực tiếp tăng gấp đôi tần số clock chính. Các kỹ thuật dựa trên DDR đã được ứng dụng trong bộ điều khiển bộ nhớ và các hệ thống truyền thông tốc độ cao trên chip [8]. Những nghiên cứu về liên kết mạng trên chip sử dụng DDR cũng cho thấy việc truyền dữ liệu trên hai cạnh clock có thể cải thiện thông lượng truyền thông nội bộ [9, 10].

Từ những vấn đề trên, khóa luận đề xuất một kiến trúc SoC dạng mô-đun dựa trên RISC-V, tích hợp các kênh truyền thông và cơ chế truyền dữ liệu DDR. Hệ thống gồm các bộ điều khiển UART, SPI và I²C dưới dạng các mô-đun RTL độc lập, được kết nối thông qua bus APB dùng chung. Cơ chế DDR hỗ trợ phát dữ liệu tại cả cạnh lên và cạnh xuống của xung clock nhằm nâng cao thông lượng. Thiết kế được mô tả bằng Verilog/SystemVerilog có khả năng tổng hợp, sau đó được kiểm chứng bằng mô phỏng và các báo cáo sau hiện thực trên FPGA. Quy trình này cho phép đánh giá chức năng, đặc tính định thời và chi phí tài nguyên trước khi xem xét mở rộng sang thiết kế ASIC [11].

Các đóng góp chính của khóa luận được tóm tắt như sau:

- Thiết kế kiến trúc SoC dạng mô-đun dựa trên RISC-V, tích hợp các bộ điều khiển truyền thông UART, SPI và I²C cho các ứng dụng truyền thông nhúng.
- Tích hợp cơ chế truyền dữ liệu theo kỹ thuật DDR nhằm cải thiện thông lượng bằng cách khai thác cả cạnh lên và cạnh xuống của xung clock.
- Xây dựng thiết kế RTL bằng Verilog/SystemVerilog có khả năng tổng hợp, phục vụ triển khai trên FPGA và tái sử dụng trong các thiết kế hướng ASIC sau này.
- Thực hiện mô phỏng chức năng và kiểm chứng sau hiện thực để xác nhận hoạt động của hệ thống đề xuất.
- Đánh giá thiết kế theo các tiêu chí về mức sử dụng tài nguyên, tần số hoạt động, đặc tính định thời và chức năng truyền thông.

Phần còn lại của khóa luận được tổ chức như sau: Chương 2 trình bày kiến thức tổng quan về các giao tiếp UART, SPI, I²C, hệ thống bus và kiến trúc SoC. Chương 3 tổng hợp các nghiên cứu liên quan. Chương 4 trình bày mô hình và phương pháp đề nghị. Chương 5 trình bày các kết quả thực nghiệm và đánh giá. Cuối cùng, Chương 6 đưa ra kết luận và đề xuất các hướng phát triển trong tương lai.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là nghiên cứu và thiết kế hệ thống ngoại vi tốc độ cao dựa trên kỹ thuật Double Data Rate (DDR), tích hợp trên SoC RISC-V nhằm nâng cao thông lượng xử lý và truyền dữ liệu. Hệ thống được mô tả bằng Verilog ở mức RTL, bao gồm các mô-đun chức năng chính, mạch kết nối nội bộ và môi trường kiểm chứng hoạt động.

1.2.2 Mục tiêu cụ thể

Các mục tiêu cụ thể của đề tài gồm:

- Khảo sát và phân tích thiết kế hiện hữu: Nghiên cứu nguyên lý truyền nhận dữ liệu Double Data Rate (DDR); đánh giá kiến trúc các mô-đun ngoại vi Single Data Rate (SDR) hiện có để xác định phương án cải tiến và chuyển đổi sang cấu trúc DDR.
- Thiết kế mô-đun ngoại vi DDR và cơ chế kiểm soát lỗi: Xây dựng mô tả RTL cho khối ngoại vi DDR, đồng thời tích hợp cơ chế kiểm soát lỗi nhằm bảo đảm tính toàn vẹn dữ liệu khi truyền nhận ở tốc độ cao.
- Xử lý chuyển miền xung clock (CDC): Thiết kế mạch đồng bộ giữa miền clock của ngoại vi DDR và miền clock hệ thống nhằm hạn chế rủi ro bất ổn định trạng thái (Metastability).
- Tối ưu hóa pipeline và thiết kế cầu nối (Bridge): Áp dụng kỹ thuật pipelining để rút ngắn đường truyền tới hạn (Critical Path); thiết kế mô-đun cầu nối để tích hợp ngoại vi DDR vào bus nội bộ của SoC RISC-V.

- Đặt ràng buộc và phân tích định thời: Thiết lập các tệp ràng buộc thời gian SDC/XDC; phân tích và tối ưu các thông số setup, hold và slack nhằm nâng cao tần số hoạt động tối đa của hệ thống.
- Tổng hợp và đánh giá kết quả: Tổng hợp kết quả mô phỏng, hiện thực phần cứng, mức sử dụng tài nguyên và báo cáo định thời để đánh giá thiết kế theo các tiêu chí đã đề ra.

1.3 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là khối ngoại vi tốc độ cao áp dụng kỹ thuật Double Data Rate (DDR), được mô tả ở mức RTL bằng ngôn ngữ Verilog. Nghiên cứu tập trung vào nguyên lý thiết kế và mô phỏng chức năng của khối ngoại vi; phương thức tích hợp vào bus APB của SoC; kỹ thuật xử lý chuyển miền xung clock (Clock Domain Crossing – CDC); cùng quy trình thiết lập và đáp ứng các ràng buộc thời gian. Đồng thời, đề tài đánh giá ảnh hưởng của độ lệch clock và độ lệch dữ liệu đến tính toàn vẹn dữ liệu khi hệ thống hoạt động ở tần số cao.

Các thành phần chính được nghiên cứu bao gồm lõi xử lý RISC-V, các mô-đun truyền nhận dữ liệu, mạch điều khiển clock, cơ chế phát dữ liệu ở hai cạnh xung, mạch đồng bộ tín hiệu, kết nối giữa các khối chức năng và các báo cáo định thời sau tổng hợp, hiện thực. Ngoài ra, đề tài cũng xem xét ảnh hưởng của việc tăng tần số clock đến độ ổn định, độ trễ đường truyền, khả năng đáp ứng định thời và tính đúng đắn của dữ liệu.

1.4 Phạm vi và giới hạn nghiên cứu

1.4.1 Phạm vi nghiên cứu

Đề tài tập trung vào quy trình thiết kế, tích hợp và kiểm chứng phần cứng ở mức Register-Transfer Level (RTL) cho hệ thống ngoại vi tốc độ cao sử dụng kỹ thuật Double Data Rate (DDR). Phạm vi nghiên cứu bao gồm:

- Thiết kế phần cứng RTL: Sử dụng ngôn ngữ Verilog để mô tả mô-đun ngoại vi DDR (chuyển đổi từ mô-đun SDR hiện có), tích hợp cơ chế kiểm soát lỗi, mạch xử lý chuyển miền xung clock (Clock Domain Crossing – CDC) bất đồng bộ,

cấu trúc pipeline và mô-đun cầu nối (Bridge) để ghép nối vào bus APB của SoC RISC-V.

- Mô phỏng chức năng (Functional Simulation): Xây dựng môi trường testbench để xác minh tính đúng đắn về logic, trình tự thời gian và khả năng kiểm soát lỗi trong các kịch bản truyền nhận dữ liệu DDR.
- Tổng hợp và hiện thực phần cứng (Synthesis & Place-and-Route): Tổng hợp thiết kế trên FPGA AMD/Xilinx Artix-7 xc7a200tfbg676-2L để đánh giá mức sử dụng tài nguyên phần cứng, bao gồm LUT, FF, BRAM và I/O.
- Tối ưu định thời: Thiết lập các ràng buộc thời gian SDC/XDC; phân tích setup slack, hold slack và clock/data skew ở các mức tần số khác nhau để xác định tần số hoạt động tối đa khả thi của hệ thống.

1.4.2 Giới hạn đề tài

Để bảo đảm tính khả thi và tập trung vào các bài toán kỹ thuật trọng tâm trong thời gian thực hiện khóa luận, đề tài xác định các giới hạn sau:

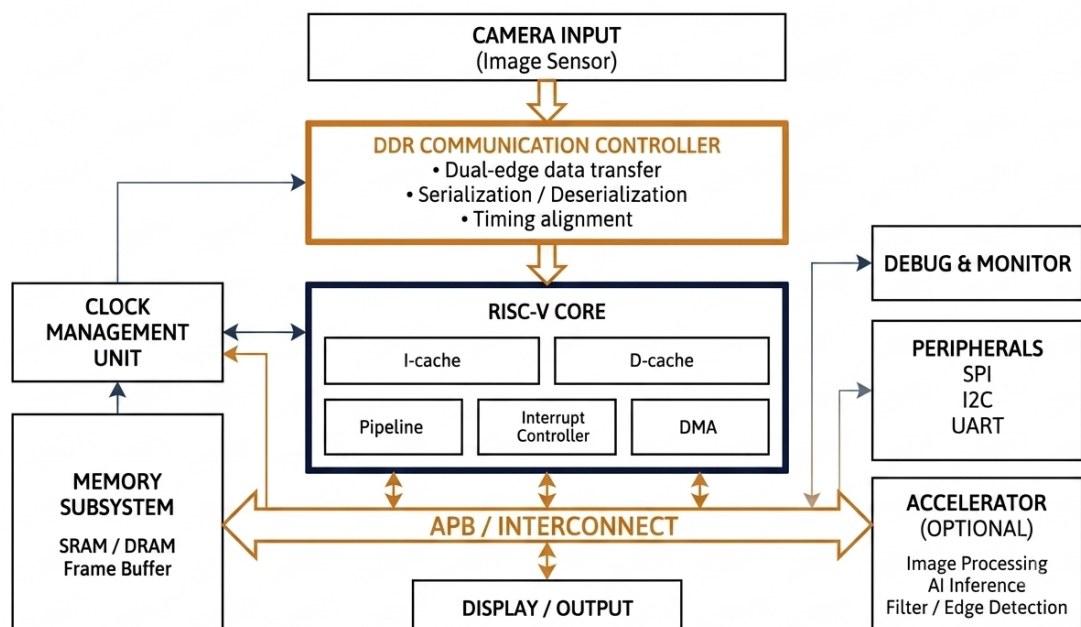
- Mô hình giao tiếp DDR: Khỏi ngoại vi DDR được thiết kế phục vụ truyền nhận dữ liệu hai cạnh xung cho giao diện ngoại vi/bus nội bộ, không tập trung xây dựng toàn bộ bộ điều khiển bộ nhớ DRAM thương mại phức tạp (như DDR3/DDR4/DDR5 SDRAM Controller với các quy trình Calibration/PHY riêng biệt).
- Môi trường kiểm chứng: Đề tài dừng lại ở bước kiểm chứng chức năng qua mô phỏng RTL và phân tích định thời, tài nguyên sau bước Place & Route trên AMD Vivado Design Suite 2025.2; chưa thực hiện đo đặc tính hiệu năng bằng máy hiện sóng hoặc bộ phân tích logic trên phần cứng thực tế.
- Hệ sinh thái phần mềm: Đề tài không xây dựng trình điều khiển, RTOS/Linux hoặc trình biên dịch cho SoC RISC-V.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Hệ thống SoC trên nền tảng RISC-V

SoC trong đề tài được xây dựng theo hướng một hệ thống xử lý nhúng gọn nhẹ, trong đó lõi RISC-V đóng vai trò điều khiển trung tâm và các mô-đun ngoại vi được kết nối thông qua bus nội bộ. Thành phần xử lý chính là PicoRV32, một lõi RISC-V 32 bit có kích thước nhỏ, phù hợp để tích hợp trên FPGA và thuận tiện cho việc quan sát luồng tín hiệu ở mức RTL. Lõi này phát sinh các yêu cầu truy cập bộ nhớ thông qua giao diện *native memory*, gồm địa chỉ, dữ liệu ghi, dữ liệu đọc, tín hiệu hợp lệ, tín hiệu sẵn sàng và tín hiệu cho phép ghi theo byte [12, 13].



Hình 2.1: Kiến trúc tổng quan của hệ thống SoC RISC-V tích hợp bộ điều khiển DDR và các khối ngoại vi.

Hình 2.1 minh họa kiến trúc tổng quan và mối liên kết giữa các thành phần chính của hệ thống. Ở mức cao nhất, hệ thống kết nối PicoRV32 với bộ nhớ lệnh/dữ liệu nội bộ và các ngoại vi thông qua bus AXI4-Lite. Vùng địa chỉ thấp được dùng cho BRAM 4 KB, trong đó chương trình được nạp từ tập tin firmware.hex. Khi CPU cần lấy lệnh, yêu cầu được đưa trực tiếp đến BRAM để giảm độ phức tạp của đường truy cập lệnh. Khi CPU thực hiện đọc hoặc ghi dữ liệu đến vùng ngoại vi, yêu cầu từ giao diện native memory được chuyển sang giao thức AXI4-Lite thông qua mô-đun `picorv32_axi_adapter` [6, 12].

Khối chuyển đổi (Adapter) có nhiệm vụ biến một truy cập nguyên thủy của CPU thành một giao dịch AXI4-Lite tiêu chuẩn với 5 kênh tín hiệu độc lập. Nhờ cơ chế bắt tay (Handshake) thông qua cặp tín hiệu `VALID` và `READY`, hệ thống đảm bảo dữ liệu chỉ được truyền đi khi cả Master và Slave đều sẵn sàng. Cách tổ chức này giúp lõi RISC-V không cần trực tiếp xử lý chi tiết của từng ngoại vi, đồng thời tạo ra một lớp kết nối thống nhất và có khả năng mở rộng bằng thông cao cho toàn bộ hệ thống [6].

Phần ngoại vi của SoC được chia theo vùng địa chỉ. Mô-đun `soc_axi_top` đóng vai trò như một bộ định tuyến (Crossbar), giải mã các vùng `0x4000_0000`, `0x4000_1000` và `0x4000_2000` lần lượt cho UART, SPI và I2C. Ngoài ra, khối `parallel` được ánh xạ riêng tại vùng `0x4000_3000`. Đây là khối xử lý song song gồm bốn lane, mỗi lane nhận hai toán hạng 32 bit và tạo kết quả cộng tương ứng. Các thanh ghi A, B, thanh ghi điều khiển/trạng thái và thanh ghi kết quả đều được truy cập thông qua bus AXI, nhờ đó phần mềm chạy trên PicoRV32 có thể ghi dữ liệu đầu vào, kích hoạt xử lý và đọc kết quả trả về [6, 12].

Như vậy, trọng tâm của SoC này không nằm ở việc xây dựng một MCU thương mại đầy đủ, mà là tạo ra một nền tảng RISC-V có đủ các thành phần cần thiết để kiểm chứng luồng truyền dữ liệu ở mức phần cứng. PicoRV32 đảm nhiệm vai trò phát lệnh và điều khiển, bộ chuyển đổi AXI bảo đảm việc luân chuyển giao dịch giữa CPU và ngoại vi, còn các khối UART, SPI, I2C và `parallel` cung cấp môi trường để đánh giá khả năng truyền nhận, xử lý song song và mở rộng bằng thông của hệ thống. Kiến trúc này là nền tảng để các phần sau phân tích cơ chế truyền dữ liệu hai cạnh clock, timing và khả năng nâng tần số hoạt động [12, 13].

2.2 Lõi xử lý RISC-V PicoRV32

RISC-V là kiến trúc tập lệnh mở, được tổ chức theo một tập lệnh cơ sở kết hợp với các phần mở rộng tùy chọn. Trong cấu hình RV32I, độ rộng thanh ghi nguyên *XLEN*

bằng 32 bit; kiến trúc cung cấp 32 thanh ghi nguyên từ x0 đến x31 và một thanh ghi bộ đếm chương trình. Thanh ghi x0 luôn có giá trị bằng không, trong khi các thanh ghi còn lại được sử dụng để lưu toán hạng, địa chỉ và kết quả trung gian của chương trình [13].

PicoRV32 là một lõi RISC-V mã nguồn mở được mô tả bằng Verilog, hướng đến việc tối ưu diện tích logic và tần số hoạt động trên FPGA hoặc ASIC. Dự án PicoRV32 cung cấp ba biến thể chính: `picorv32` sử dụng giao diện bộ nhớ nguyên bản, `picorv32_axi` sử dụng giao diện AXI4-Lite và `picorv32_wb` sử dụng giao diện Wishbone. Trong khóa luận này, biến thể `picorv32` được lựa chọn vì giao diện bắt tay đơn giản, thuận lợi cho việc kết nối BRAM siêu tốc (1 chu kỳ) và xây dựng cầu nối sang bus AXI4-Lite thông qua một bộ Adapter chuyên dụng [12].

Giao diện bộ nhớ nguyên bản của PicoRV32 hoạt động theo cơ chế `valid/ready` và xử lý một giao dịch tại mỗi thời điểm. Các tín hiệu chính bao gồm:

- `mem_valid`: báo hiệu CPU đang phát sinh một yêu cầu truy cập bộ nhớ;
- `mem_instr`: phân biệt chu kỳ lấy lệnh với chu kỳ truy cập dữ liệu;
- `mem_ready`: xác nhận bộ nhớ hoặc thiết bị đích đã hoàn tất giao dịch;
- `mem_addr`: mang địa chỉ truy cập 32 bit;
- `mem_wdata` và `mem_rdata`: lần lượt mang dữ liệu ghi và dữ liệu đọc 32 bit;
- `mem_wstrb`: gồm bốn bit cho phép ghi, tương ứng với bốn byte của một từ dữ liệu 32 bit.

Khi bắt đầu một giao dịch, PicoRV32 đưa `mem_valid` lên mức cao và giữ ổn định các tín hiệu địa chỉ, dữ liệu và điều khiển cho đến khi nhận được `mem_ready`. Đối với giao dịch đọc, `mem_wstrb` bằng không và dữ liệu được nhận qua `mem_rdata`. Đối với giao dịch ghi, một hoặc nhiều bit của `mem_wstrb` được kích hoạt để xác định các byte cần cập nhật trong từ dữ liệu. Cơ chế này cho phép ghép nối trực tiếp với bộ nhớ đơn giản hoặc chuyển đổi sang một giao thức bus khác thông qua mạch cầu nối [12].

PicoRV32 còn hỗ trợ nhiều tham số cấu hình nhằm cân bằng giữa diện tích và hiệu năng, chẳng hạn lựa chọn tập thanh ghi một cổng hoặc hai cổng đọc, bộ dịch barrel, hỗ trợ ngắt và giao diện đồng xử lý PCPI. Các phép nhân, chia có thể được bổ

sung thông qua các khối PCPI tương ứng. Nhờ khả năng cấu hình này, lõi có thể được điều chỉnh cho các hệ thống nhúng nhỏ hoặc mở rộng khi ứng dụng cần thêm chức năng xử lý [12].

Trong SoC của khóa luận, PicoRV32 thực hiện lấy lệnh trực tiếp từ BRAM nội bộ với độ trễ tối thiểu, và truy cập các ngoại vi thông qua mô-đun `picorv32_axi_adapter`. Cầu nối tiếp nhận yêu cầu từ giao diện bộ nhớ nguyên bản, biên dịch thành các kênh giao dịch độc lập của AXI4-Lite và chỉ đưa `mem_ready` lên mức cao sau khi ngoại vi hoàn tất quá trình phản hồi `BVALID` hoặc `RVALID`. Vì vậy, PicoRV32 đóng vai trò bộ điều khiển trung tâm, trong khi bus AXI cung cấp không gian địa chỉ thống nhất cho UART, SPI, I²C và khối xử lý song song DDR.

2.3 Sự khác biệt giữa truyền dữ liệu SDR và DDR

Ở phương thức Single Data Rate (SDR), dữ liệu thường được phát hoặc lấy mẫu tại một cạnh cố định của xung clock, chẳng hạn cạnh lên hoặc cạnh xuống. Với tần số clock f_{clk} và bus dữ liệu có độ rộng W , tốc độ truyền lý thuyết có thể biểu diễn gần đúng bởi [8, 14]:

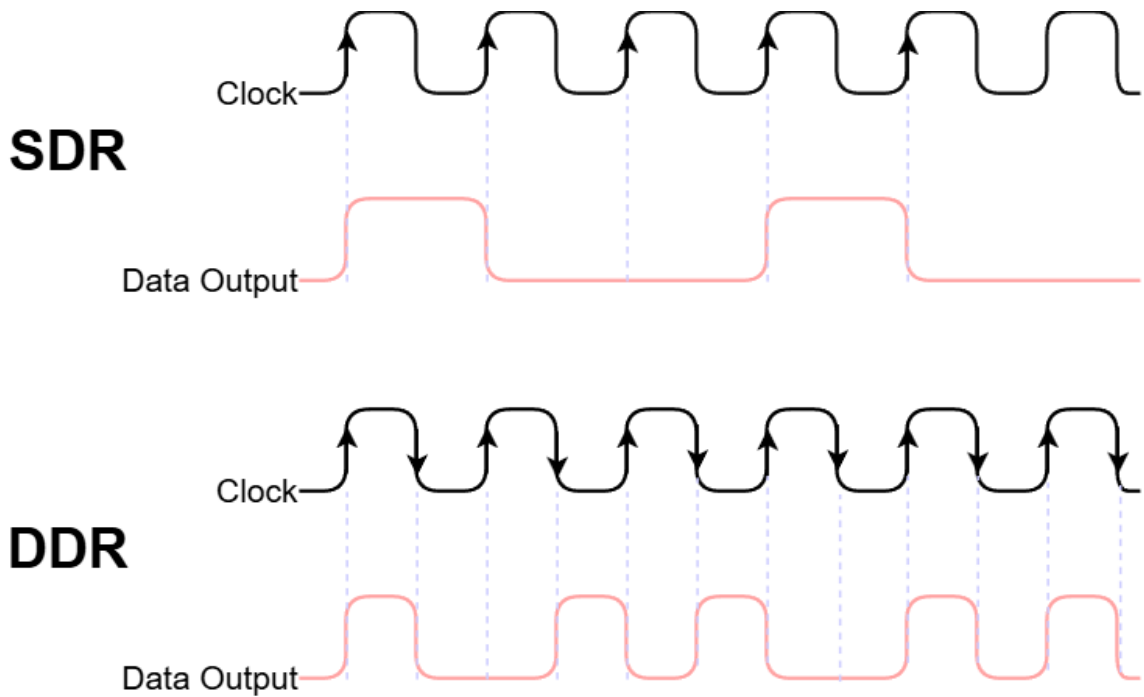
$$R_{SDR} = W \times f_{clk}. \quad (2.1)$$

Trong khi đó, Double Data Rate (DDR) sử dụng cả cạnh lên và cạnh xuống của xung clock để truyền dữ liệu. Như vậy, trong một chu kỳ clock có thể truyền hai lần dữ liệu. Tốc độ truyền lý thuyết khi đó được xác định bởi [10, 14]:

$$R_{DDR} = 2 \times W \times f_{clk}. \quad (2.2)$$

Việc khai thác hai cạnh clock cũng được áp dụng trong các giao diện nối tiếp tốc độ cao, chẳng hạn chế độ truyền DDR trên các bộ điều khiển nhớ ngoài của vi điều khiển NXP [15].

Theo đó, DDR có thể tăng gấp đôi số lần truyền trong cùng một khoảng thời gian mà không nhất thiết phải tăng độ rộng bus. Đây là điểm trọng tâm của đề tài. Tuy nhiên, việc sử dụng hai cạnh clock cũng làm yêu cầu về thời điểm phát dữ liệu, thời điểm lấy mẫu, độ lệch clock và thời gian ổn định của tín hiệu trở nên nghiêm ngặt hơn [9, 10, 16].



Hình 2.2: Minh họa sự khác biệt giữa phương thức truyền dữ liệu SDR và DDR [14].

Hình 2.2 cho thấy SDR chỉ thay đổi dữ liệu theo một cạnh clock, trong khi DDR khai thác cả cạnh lên và cạnh xuống để tăng gấp đôi số lần truyền trong mỗi chu kỳ so với SDR [14].

Bảng 2.1: So sánh khái quát giữa SDR và DDR

Tiêu chí	SDR	DDR
Số lần truyền trong một chu kỳ	Một lần	Hai lần
Điểm phát hoặc lấy mẫu	Một cạnh clock	Cả cạnh lên và cạnh xuống
Băng thông lý thuyết	Thấp hơn ở cùng tần số	Cao hơn ở cùng tần số
Yêu cầu timing	Tương đối đơn giản	Chặt chẽ hơn
Độ phức tạp thiết kế	Thấp hơn	Cao hơn do phải quản lý hai cạnh clock

Sự khác biệt trực quan về mật độ dữ liệu giữa hai phương thức được thể hiện trên Hình 2.2. Việc chuyển sang sử dụng cả hai cạnh xung nhịp mang lại cho kỹ thuật DDR những thế mạnh cốt lõi so với phương thức SDR [9, 10]:

Thứ nhất, DDR giúp tăng gấp đôi băng thông truyền tải mà không cần nâng cao tần số xung nhịp hệ thống. Để đạt cùng một hiệu năng xử lý, phương thức SDR

phải tăng tần số f_{clk} , kéo theo nguy cơ gia tăng nhiễu điện từ (EMI), làm phức tạp mạng phân phối clock (*Clock Tree*) và dễ vi phạm ràng buộc timing. DDR giải quyết vấn đề này bằng cách nhân đôi lưu lượng dữ liệu trên một chu kỳ clock [10, 16].

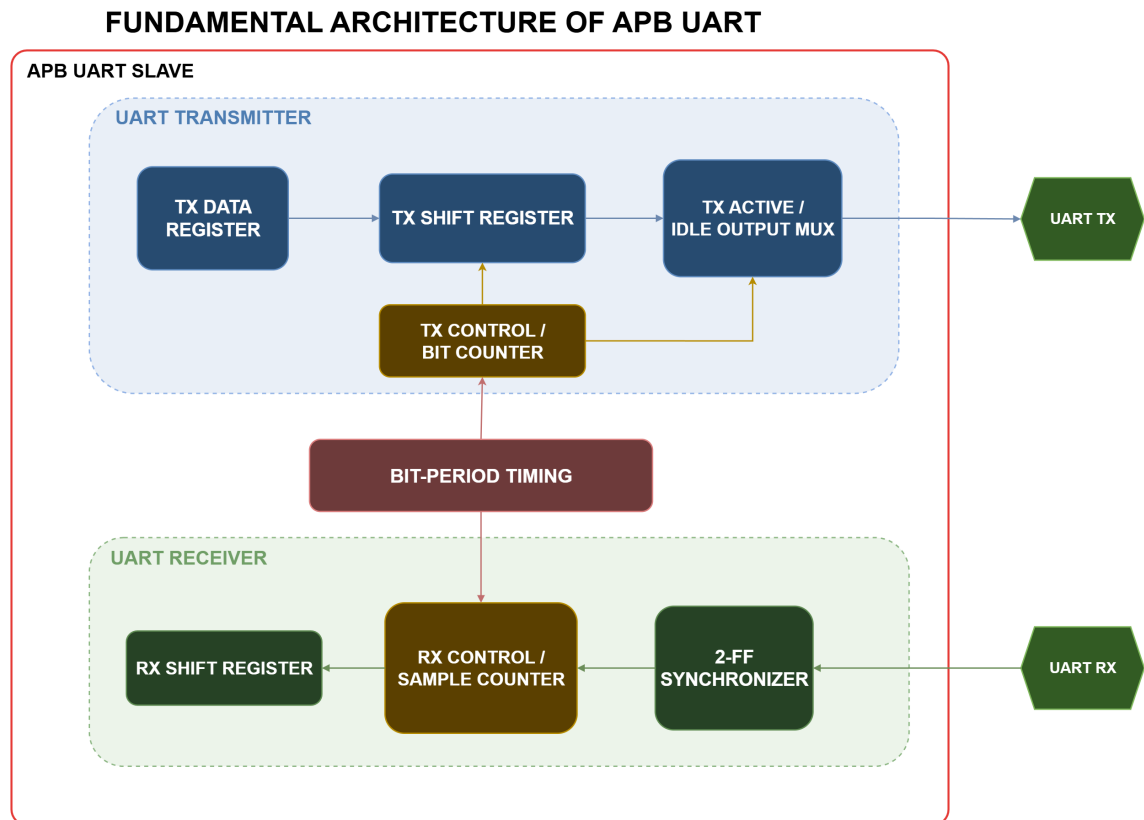
Thứ hai, DDR hỗ trợ tiết kiệm tài nguyên chân I/O khi cần tăng băng thông. Với SDR, một phương án tăng băng thông mà không tăng tần số là mở rộng độ rộng bus dữ liệu, làm tăng số lượng đường tín hiệu. Kỹ thuật DDR cho phép giữ nguyên số đường dữ liệu vật lý nhưng tăng số lần truyền trong mỗi chu kỳ [8, 14].

Thứ ba, DDR có thể cải thiện hiệu suất năng lượng khi đạt cùng mức băng thông ở tần số clock thấp hơn. Trong mạch số, công suất động phụ thuộc vào tần số chuyển mạch; vì vậy việc hạn chế tăng clock có thể giảm áp lực công suất trên mạng xung nhịp [16].

Mặc dù sở hữu nhiều ưu điểm về mặt hiệu năng và tài nguyên, phương thức DDR cũng đặt ra thách thức lớn hơn về quản lý timing. Do thời gian dành cho một lần truyền bị thu hẹp xuống còn $T_{clk}/2$, các thông số như thời gian thiết lập (t_{setup}), thời gian giữ (t_{hold}) và độ lệch clock (*clock skew*) phải được kiểm soát chặt chẽ. Mô hình DDR trong đề tài tập trung vào việc thiết kế và tối ưu khối giao tiếp RTL hai cạnh clock nhằm chứng minh tính hiệu quả trên nền tảng RISC-V SoC [10, 16].

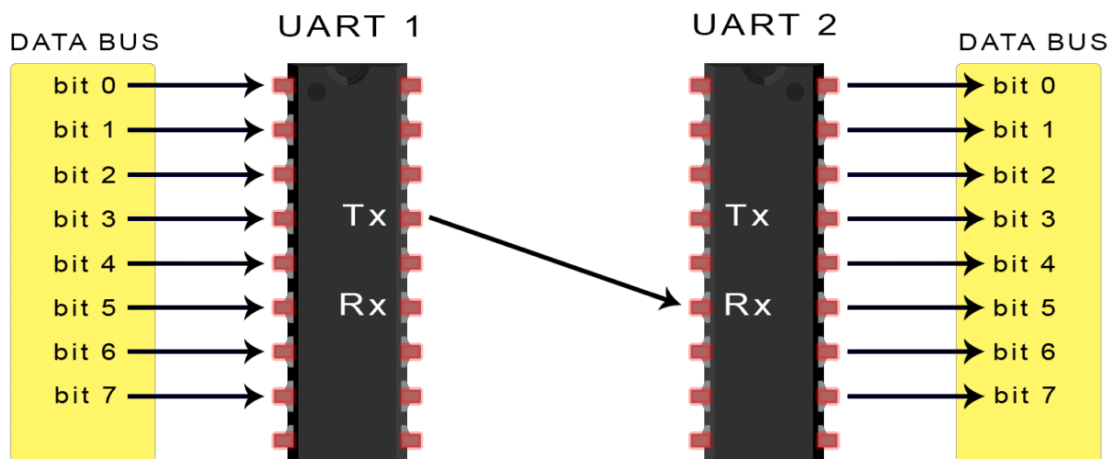
2.4 Giao tiếp UART

Universal Asynchronous Receiver/Transmitter (UART) là chuẩn giao tiếp nối tiếp bất đồng bộ, được sử dụng rộng rãi trong các hệ thống nhúng để trao đổi dữ liệu giữa vi điều khiển, máy tính, module ngoại vi hoặc mạch kiểm thử. Đặc điểm quan trọng của UART là hai thiết bị không dùng chung một đường clock truyền kèm dữ liệu. Thay vào đó, bên truyền và bên nhận phải thống nhất trước các tham số như tốc độ baud, số bit dữ liệu, bit kiểm tra chẵn lẻ nếu có và số bit dừng [2, 17, 18].



Hình 2.3: Kiến trúc và luồng dữ liệu của giao tiếp UART [18].

Hình 2.3 trình bày các thành phần chính của bộ điều khiển UART, bao gồm khối thanh ghi dùng để lưu dữ liệu truyền, dữ liệu nhận, cấu hình và trạng thái hoạt động. Khối chia xung clock xác định chu kỳ của từng bit. Máy trạng thái điều khiển trình tự phát và thu, trong khi thanh ghi dịch truyền bổ sung bit bắt đầu và bit dừng rồi đưa dữ liệu tuần tự ra đường TX. Theo chiều ngược lại, khối lấy mẫu giám sát đường RX, nhận lần lượt các bit và ghép chúng thành dữ liệu song song để CPU đọc qua bus AXI [3, 4].



Hình 2.4: Dạng sóng truyền nhận dữ liệu của giao tiếp UART [18].

Hình 2.4 minh họa cấu trúc thời gian của một khung UART, gồm trạng thái rỗi, bit bắt đầu, các bit dữ liệu và bit dừng [17].

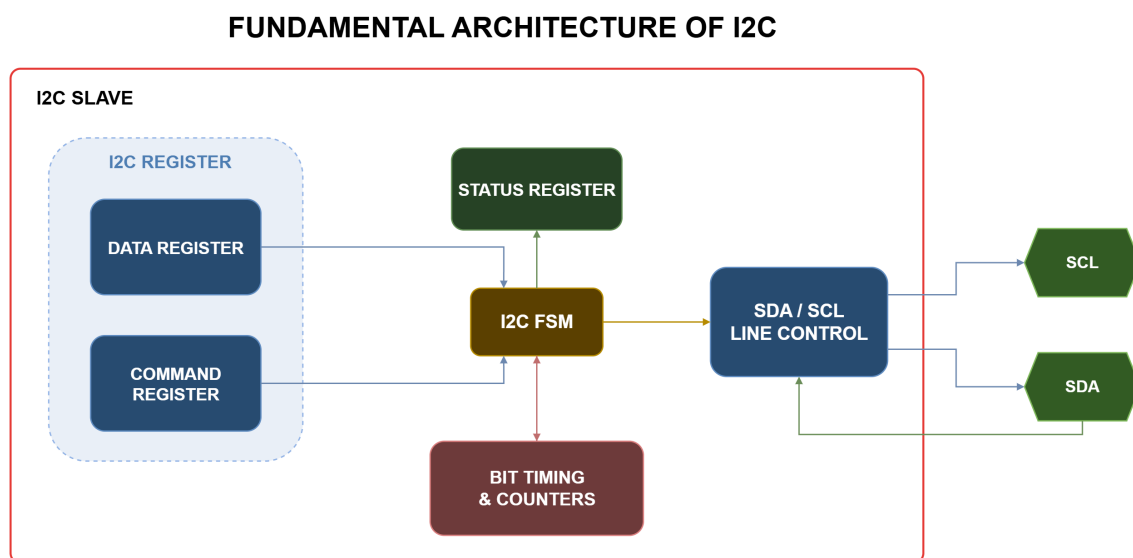
Về nguyên lý, dữ liệu song song bên trong hệ thống được UART chuyển thành chuỗi bit nối tiếp trước khi đưa ra chân truyền. Khi không có dữ liệu, đường truyền thường ở trạng thái rỗi. Một khung truyền bắt đầu bằng bit start để báo cho bên nhận biết thời điểm lấy mẫu, tiếp theo là các bit dữ liệu, sau đó có thể có bit parity và kết thúc bằng một hoặc nhiều bit stop. Bên nhận phát hiện bit start, dùng tốc độ baud đã cấu hình để lấy mẫu các bit kế tiếp và ghép lại thành dữ liệu song song [2, 17].

UART có ưu điểm là cấu trúc đơn giản, ít chân tín hiệu và dễ dùng trong giai đoạn kiểm thử. Chỉ cần hai đường chính là TX và RX, hệ thống đã có thể gửi log, truyền lệnh điều khiển hoặc kiểm tra dữ liệu đầu ra. Hạn chế của UART nằm ở tốc độ không cao bằng các giao tiếp đồng bộ và độ chính xác phụ thuộc vào sai số baud rate giữa hai phía. Nếu tần số lấy mẫu bị lệch nhiều, bên nhận có thể đọc sai bit, đặc biệt khi khung dữ liệu dài hoặc tốc độ truyền cao [17].

Trong SoC, UART được ánh xạ tại địa chỉ 0x4000_0000. CPU RISC-V điều khiển UART thông qua các thanh ghi AXI4-Lite. Khi cần gửi dữ liệu, phần mềm ghi byte hoặc word vào thanh ghi truyền; module UART sẽ tuần tự hóa dữ liệu và phát ra chân uart_tx. Khi có dữ liệu đi vào từ chân uart_rx, UART lấy mẫu, phục hồi dữ liệu và đặt vào thanh ghi nhận để CPU đọc lại. Nhờ cơ chế này, UART đóng vai trò kênh giao tiếp cơ bản giữa SoC và môi trường bên ngoài, đồng thời hỗ trợ quan sát hoạt động của hệ thống trong quá trình mô phỏng hoặc chạy trên FPGA [6, 6].

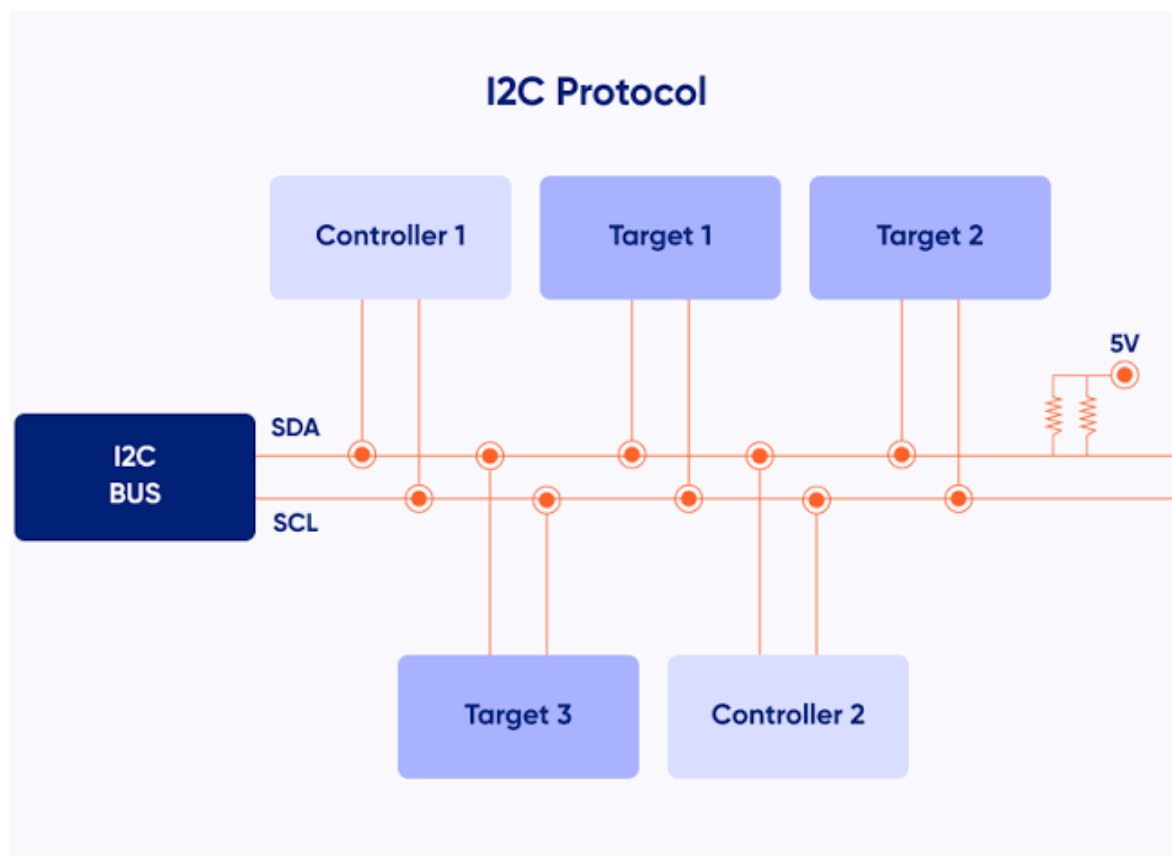
2.5 Giao tiếp I2C

Inter-Integrated Circuit (I2C) là chuẩn giao tiếp nối tiếp đồng bộ sử dụng hai đường tín hiệu chính: SCL và SDA. SCL mang xung clock, còn SDA mang dữ liệu hai chiều. Khác với UART, I2C cho phép nhiều thiết bị cùng chia sẻ một bus thông qua cơ chế địa chỉ slave. Điều này làm I2C phù hợp với các hệ thống cần kết nối nhiều cảm biến, EEPROM, mạch quản lý nguồn hoặc IC cấu hình mà không muốn tiêu tốn quá nhiều chân I/O [19, 20].



Hình 2.5: Kiến trúc và trình tự truyền dữ liệu của giao tiếp I²C [20].

Hình 2.5 minh họa cấu tạo của bộ điều khiển I²C. Khối thanh ghi lưu địa chỉ thiết bị, dữ liệu, lệnh điều khiển và trạng thái ACK/NACK. Khối định thời và bộ đếm bit tạo các pha cần thiết trên đường SCL. Máy trạng thái điều khiển lần lượt các điều kiện START, truyền địa chỉ, truyền hoặc nhận dữ liệu, kiểm tra xác nhận và tạo điều kiện STOP. Khối điều khiển đường truyền quản lý hai tín hiệu SCL và SDA theo cơ chế cực máng hở; thiết bị chỉ kéo tín hiệu xuống mức thấp hoặc nhả đường bus để điện trở kéo lên tạo mức cao [19].



Hình 2.6: Dạng sóng truyền dữ liệu của giao tiếp I^2C [20].

Hình 2.6 minh họa quan hệ thời gian giữa hai đường SCL và SDA trong một giao dịch I^2C , bao gồm điều kiện START, các bit địa chỉ và dữ liệu, bit xác nhận ACK/NACK và điều kiện STOP [19].

Về mặt hoạt động, bus I^2C thường có một master điều khiển quá trình truyền thông và một hoặc nhiều slave phản hồi. Master tạo điều kiện start bằng cách kéo SDA xuống mức thấp khi SCL đang ở mức cao. Sau đó, master gửi địa chỉ slave cùng bit xác định hướng đọc hoặc ghi. Nếu slave nhận ra địa chỉ của mình, nó phản hồi bằng bit ACK. Dữ liệu tiếp tục được truyền theo từng byte, mỗi byte đi kèm một bit ACK hoặc NACK. Giao dịch kết thúc bằng điều kiện stop, khi SDA chuyển từ thấp lên cao trong lúc SCL ở mức cao [19].

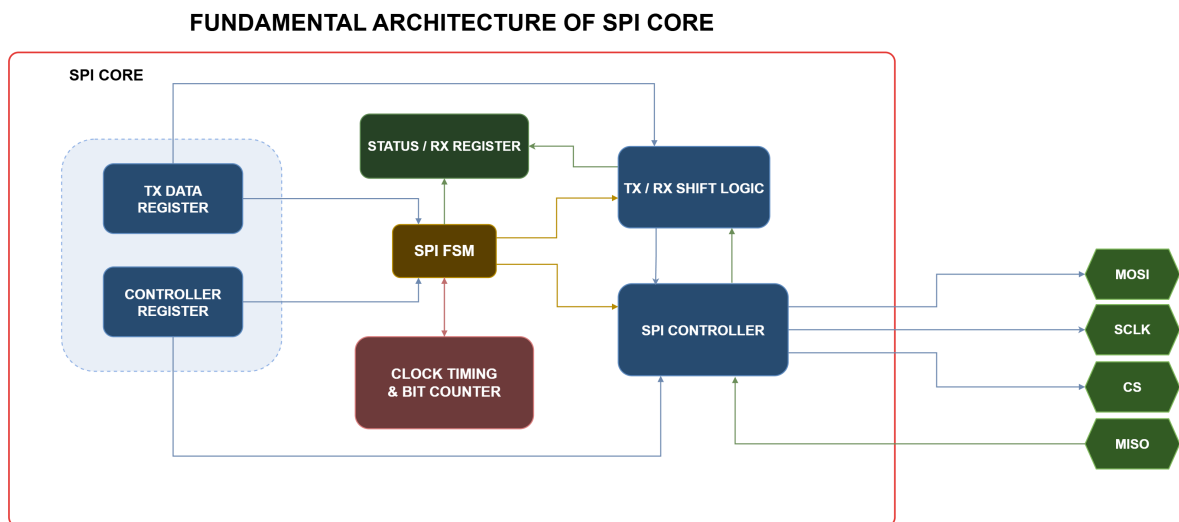
Do SDA là đường hai chiều, các thiết bị trên bus không được tự ý đẩy mức cao một cách chủ động như ngõ ra thông thường. Thực tế, đường SDA và SCL thường hoạt động theo kiểu open-drain hoặc open-collector, dùng điện trở kéo lên để tạo mức cao. Thiết bị chỉ kéo đường truyền xuống thấp khi cần phát logic 0, còn logic 1 được tạo bằng cách nhả đường truyền. Cách tổ chức này giúp nhiều thiết bị cùng nối trên bus mà hạn chế xung đột điện, nhưng cũng làm tốc độ bị ảnh hưởng bởi điện trở kéo

lên và điện dung đường dây [19].

Trong SoC, I2C được ánh xạ tại vùng 0x4000_2000. Các tín hiệu scl_out, sda_out, sda_oe và sda_in thể hiện cách điều khiển đường dữ liệu hai chiều. Tín hiệu sda_oe quyết định lúc module được phép kéo SDA xuống và lúc phải nhả bus để slave phản hồi. CPU cấu hình module I2C thông qua bus AXI4-Lite để tạo start, gửi địa chỉ, truyền hoặc nhận dữ liệu, kiểm tra ACK và tạo stop. Nhờ đó, SoC có khả năng giao tiếp với nhiều thiết bị tốc độ thấp hoặc trung bình bằng số lượng chân tín hiệu rất nhỏ [6, 19].

2.6 Giao tiếp SPI

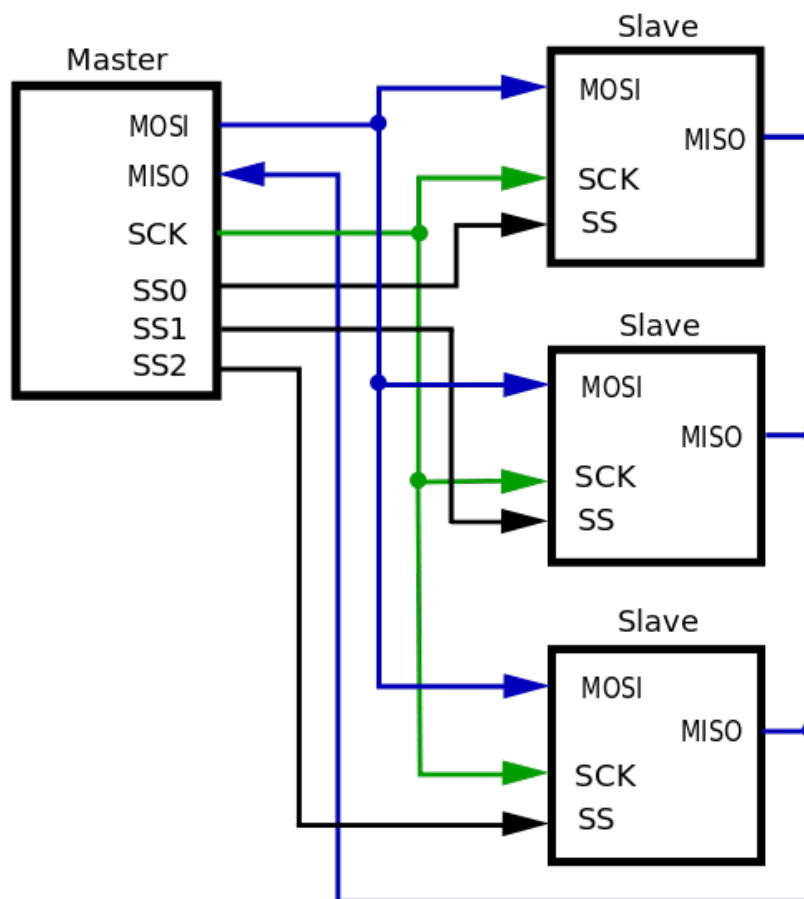
Serial Peripheral Interface (SPI) là chuẩn giao tiếp nối tiếp đồng bộ thường dùng giữa một master và một hoặc nhiều slave. Bus SPI phổ biến với bốn tín hiệu: SCLK, MOSI, MISO và CS. Master tạo xung clock trên SCLK, gửi dữ liệu đến slave qua MOSI, nhận dữ liệu từ slave qua MISO và chọn thiết bị cần giao tiếp bằng tín hiệu CS. Do có clock riêng, SPI thường đạt tốc độ cao hơn UART và đơn giản hơn I2C ở phần định địa chỉ [21, 22].



Hình 2.7: Kiến trúc và luồng dữ liệu song công của giao tiếp SPI [22].

Hình 2.7 trình bày các thành phần chính của bộ điều khiển SPI, bao gồm khối thanh ghi lưu dữ liệu truyền, dữ liệu nhận, cấu hình chế độ và trạng thái giao dịch. Bộ chia clock tạo tín hiệu SCLK, còn bộ đếm bit xác định thời điểm hoàn tất một từ dữ liệu. Máy trạng thái điều khiển tín hiệu chọn thiết bị CS và các pha lấy mẫu theo cầu

hình CPOL/CPHA. Hai thanh ghi dịch thực hiện đồng thời việc đưa từng bit ra MOSI và nhận từng bit từ MISO, nhờ đó SPI hỗ trợ truyền dữ liệu song công toàn phần [21].



Hình 2.8: Dạng sóng truyền dữ liệu của giao tiếp SPI [22].

Hình 2.8 minh họa quan hệ thời gian giữa tín hiệu chọn thiết bị, xung clock nối tiếp và các đường dữ liệu MOSI/MISO trong một giao dịch SPI [21].

Hoạt động của SPI dựa trên sự dịch bit đồng thời ở hai hướng. Trong mỗi chu kỳ clock, master phát một bit trên MOSI và đồng thời lấy mẫu một bit từ MISO. Vì vậy, SPI có thể truyền song công toàn phần nếu cả master và slave đều hỗ trợ. Thứ tự bit, cực tính clock và pha lấy mẫu được quyết định bởi cấu hình SPI mode. Hai tham số quan trọng là CPOL và CPHA: CPOL xác định mức rỗi của clock, còn CPHA xác định cạnh clock dùng để lấy mẫu dữ liệu. Master và slave phải dùng cùng chế độ thì dữ liệu mới được nhận đúng [21].

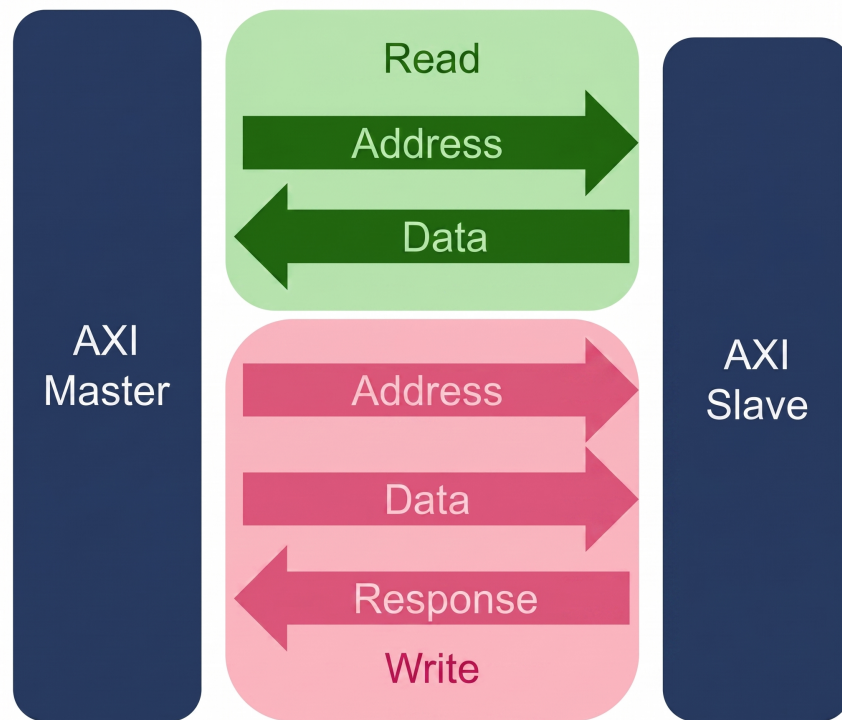
Ưu điểm của SPI là tốc độ cao, giao thức gọn và dễ hiện thực bằng phần cứng. Chuẩn này phù hợp với ADC, DAC, cảm biến tốc độ cao, màn hình, thẻ nhớ hoặc flash

ngoài. Nhược điểm chính là số chân chọn chip tăng theo số lượng slave nếu dùng cấu trúc truyền thống. Ngoài ra, SPI không có cơ chế địa chỉ và xác nhận dữ liệu chuẩn hóa như I2C, nên việc kiểm tra lỗi hoặc quản lý nhiều thiết bị thường do tầng giao thức phía trên đảm nhiệm [21].

Trong hệ thống được xây dựng, SPI nằm ở vùng địa chỉ 0x4000_1000. CPU ghi dữ liệu cần truyền vào thanh ghi của module, cấu hình trạng thái giao dịch và đọc dữ liệu nhận được sau khi quá trình truyền hoàn tất thông qua chuẩn AXI4-Lite. Các chân spi_sclk, spi_mosi, spi_miso và spi_cs là giao diện vật lý nối ra ngoài. Nhờ kết nối linh hoạt này, SPI trở thành một ngoại vi có thể điều khiển thống nhất bằng phần mềm chạy trên PicoRV32, đồng thời cung cấp kênh truyền tốc độ cao hơn cho các thiết bị ngoại vi cần trao đổi dữ liệu liên tục [6, 21].

2.7 Giao tiếp AXI4-Lite và kết nối giữa các mô-đun

Advanced eXtensible Interface (AXI) phiên bản 4-Lite thuộc tập giao thức AMBA của ARM, là chuẩn bus được thiết kế dành riêng cho các thiết bị ngoại vi ánh xạ bộ nhớ (Memory-Mapped) có yêu cầu truyền tải ổn định với băng thông cao. Khác biệt hoàn toàn so với kiến trúc APB vốn dùng chung đường truyền và xử lý tuần tự, AXI4-Lite chia tách hệ thống thành 5 kênh (channels) truyền tải dữ liệu độc lập, cho phép CPU thực thi các giao dịch song song và đường ống (pipelining) một cách hiệu quả.



Hình 2.9: Kiến trúc năm kênh truyền dữ liệu giữa AXI Master và AXI Slave [6].

Một giao dịch trên bus AXI4-Lite diễn ra thông qua cơ chế bắt tay (Handshake) chặt chẽ giữa tín hiệu ‘VALID’ (từ thiết bị gửi) và ‘READY’ (từ thiết bị nhận) trên 5 kênh cơ bản sau:

- Kênh Ghi địa chỉ (Write Address - AW): Transmit địa chỉ cần ghi thông qua các chân ‘AWADDR’, ‘AWVALID’ và ‘AWREADY’.
- Kênh Ghi dữ liệu (Write Data - W): Transmit dữ liệu và mask tín hiệu byte (strobe) qua ‘WDATA’, ‘WSTRB’, ‘WVALID’ và ‘WREADY’. Kênh dữ liệu này có thể hoạt động độc lập và song song với kênh địa chỉ.
- Kênh Phản hồi ghi (Write Response - B): Cung cấp trạng thái báo cáo giao dịch ghi (thành công hay có lỗi) từ Slave về Master thông qua ‘BRESP’, ‘BVALID’ và ‘BREADY’.
- Kênh Đọc địa chỉ (Read Address - AR): Transmit địa chỉ cần truy xuất thông qua ‘ARADDR’, ‘ARVALID’ và ‘ARREADY’.
- Kênh Đọc dữ liệu (Read Data - R): Phản hồi dữ liệu kèm cờ trạng thái từ Slave về Master thông qua ‘RDATA’, ‘RRESP’, ‘RVALID’ và ‘RREADY’.

Trong kiến trúc SoC được xây dựng, mô-đun Adapter picorv32_axi_adapter đóng vai trò là AXI Master duy nhất. Khối này tiếp nhận các yêu cầu đọc/ghi từ giao diện *native memory* của lõi xử lý PicoRV32, sau đó tự động biên dịch thành các chuỗi tín hiệu trên 5 kênh AXI4-Lite.

Do thiết kế bao gồm nhiều Slave (nhóm UART, SPI, I2C và khối DDR song song), hệ thống sử dụng một bộ định tuyến Crossbar tích hợp bên trong mô-đun soc_axi_top thay vì chỉ dùng một bộ giải mã địa chỉ thông thường. Bộ định tuyến này thực hiện nhiệm vụ:

1. Giải mã địa chỉ: Phân tích dải địa chỉ trên kênh ‘AWADDR’ hoặc ‘ARADDR’ để xác định chính xác thiết bị Slave mục tiêu.
2. Chốt mục tiêu (Target Latching): Khác với bus chia sẻ, các kênh của AXI hoạt động không đồng bộ. Bộ định tuyến sử dụng các thanh ghi chốt mục tiêu (*w_target* và *r_target*) để lưu giữ đường dẫn kết nối kênh dữ liệu (W, R) và kênh phản hồi (B) cho tới khi toàn bộ chu trình giao dịch của thiết bị Slave đó khép lại. Điều này ngăn chặn tình trạng thất thoát tín hiệu (Deadlock) hay giao tiếp chéo giữa các Slave đang hoạt động song song.

Nhờ sự tối ưu của AXI4-Lite và bộ định tuyến đa kênh, SoC có khả năng điều phối giao tiếp đồng thời, đáp ứng tốt yêu cầu truyền dẫn dữ liệu tốc độ kép ở các phần sau [5, 6].

2.8 Timing và giới hạn tần số hoạt động

Trong các thiết kế mạch số đồng bộ (Synchronous Digital Circuits), việc đảm bảo tính đúng đắn về mặt dữ liệu phụ thuộc vào các ràng buộc thời gian (Timing Constraints). Dữ liệu sau khi chốt tại phần tử lưu trữ nguồn (Source Flip-Flop) phải lan truyền qua mạng logic tổ hợp và đến được phần tử lưu trữ đích (Destination Flip-Flop) trước cạnh chốt clock tiếp theo. Để tránh vi phạm thời gian thiết lập (*Setup Time Violation*), chu kỳ xung clock hệ thống T_{clk} phải thỏa mãn điều kiện bất đẳng thức cơ bản [16]:

$$T_{clk} \geq t_{cq} + t_{logic} + t_{setup} + t_{skew}, \quad (2.3)$$

trong đó t_{cq} (*Clock-to-Q Delay*) là độ trễ từ lúc xuất hiện cạnh clock đến khi dữ liệu ổn định ở ngõ ra thanh ghi nguồn; t_{logic} biểu diễn tổng độ trễ đường truyền qua logic tổ hợp và dây dẫn kết nối (*Routing Delay*); t_{setup} là thời gian duy trì dữ liệu

ổn định tối thiểu trước cạnh clock chốt tại thanh ghi đích; và t_{skew} (*Clock Skew*) đại diện cho độ lệch thời điểm xuất hiện cạnh clock giữa hai thanh ghi nguồn và đích trên mạng phân phối xung nhịp [16].

Khi nâng cao tần số hoạt động của SoC (tương ứng với việc thu hẹp chu kỳ T_{clk}), khoảng thời gian dành cho dữ liệu lan truyền bị thu hẹp, khiến bất đẳng thức trên khó đáp ứng hơn. Nếu dữ liệu đến thanh ghi đích trễ hơn thời gian cho phép, thiết kế sẽ vi phạm ràng buộc setup và có thể hoạt động không ổn định trên phần cứng [16].

Đối với phương thức truyền dữ liệu hai cạnh clock DDR, yêu cầu timing closure trở nên chặt chẽ hơn. Do dữ liệu được phát và lấy mẫu ở cả cạnh lên và cạnh xuống, khoảng thời gian giữa hai lần truyền chỉ còn một nửa chu kỳ xung nhịp ($T_{clk}/2$). Điều này đòi hỏi nhà thiết kế không chỉ kiểm soát t_{setup} mà còn phải đảm bảo ràng buộc thời gian giữ t_{hold} . Bên cạnh đó, các yếu tố như độ nhiễu pha xung nhịp (*Clock Jitter*), độ lệch pha giữa các đường truyền song song (*Data-to-Data Skew*) và khả năng đồng bộ miền xung nhịp (CDC) cũng ảnh hưởng đến tính toàn vẹn tín hiệu [10, 16].

Trong phạm vi đề tài, quá trình phân tích và tối ưu timing được thực hiện định lượng thông qua bộ công cụ Vivado. Thiết kế SoC được khảo sát qua từng mức tần số clock tăng dần để đánh giá đường truyền tới hạn (*Critical Path*). Giá trị slack, đặc biệt là chỉ số WNS (*Worst Negative Slack*), được dùng làm tiêu chí đánh giá [16, 23]:

- Slack ≥ 0 (Slack dương): Thiết kế hoàn toàn đạt ràng buộc timing, dữ liệu truyền nhận chính xác ở tần số khảo sát.
- Slack < 0 (Slack âm): Hệ thống bị vi phạm timing. Lúc này, các giải pháp kỹ thuật RTL sẽ được áp dụng như: chèn các tầng thanh ghi trung gian (*Pipelining*) để cắt ngắn độ trễ t_{logic} , tái cấu trúc logic tổ hợp, tối ưu hóa sơ đồ đặt và nối dây (*Place & Route*), hoặc sử dụng các khối quản lý xung nhịp chuyên dụng (như MMCM/PLL) để điều chỉnh pha xung nhịp linh hoạt.

Các tiêu chí slack và phương pháp tối ưu nêu trên phù hợp với quy trình timing closure của Vivado [16].

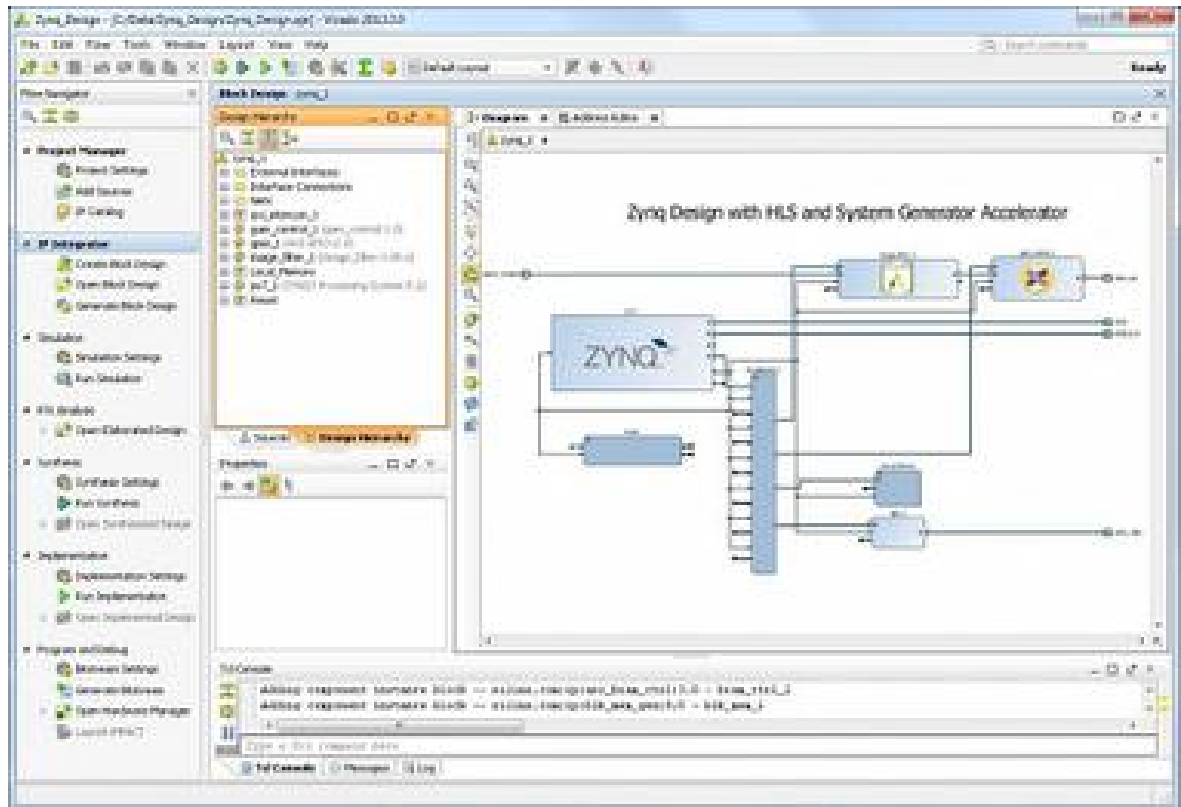
Việc tăng tần số clock chỉ mang lại giá trị hiệu năng khi thiết kế vẫn đạt timing closure và duy trì tính đúng đắn trên phần cứng FPGA [16].

2.9 Công cụ Vivado

Vivado Design Suite là môi trường phát triển phần cứng tích hợp do Xilinx, hiện thuộc AMD, phát triển cho quy trình thiết kế, kiểm thử và triển khai hệ thống số trên FPGA và Adaptive SoC. Công cụ tích hợp các tác vụ quản lý mã nguồn RTL, mô phỏng chức năng, tổng hợp logic, đặt và nối dây, phân tích timing và tạo tập tin cấu hình bitstream trong một môi trường thống nhất [23].

Quy trình triển khai SoC RISC-V trên Vivado được thực hiện qua các giai đoạn khởi tạo dự án, tích hợp nguồn RTL, mô phỏng, tổng hợp và hiện thực. Các tệp Verilog cấu thành SoC gồm lõi picorv32.v, module Adapter picorv32_axi_adapter.v, khối giao tiếp ngoại vi soc_axi_top.v, khối parallel.v và các ngoại vi I/O. Vivado Simulator hỗ trợ chạy testbench để kiểm tra từng khối và toàn bộ hệ thống trước khi tổng hợp [12, 23].

Sau khi hoàn tất mô phỏng chức năng, tập tin ràng buộc .xdc được thiết lập để định nghĩa tần số clock mục tiêu, gán chân I/O và khai báo chuẩn điện áp. Vivado tổng hợp mã RTL thành netlist, sau đó thực hiện placement và routing nhằm đáp ứng các ràng buộc timing trước khi tạo bitstream để nạp lên phần cứng [16, 23].



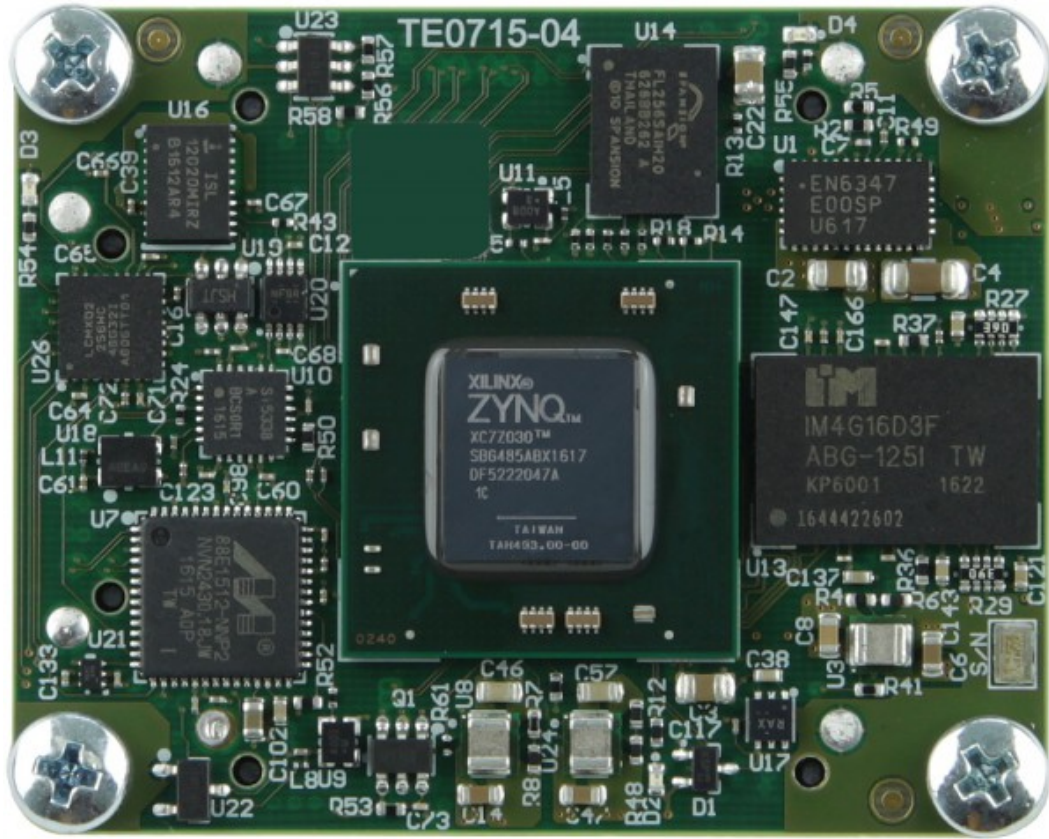
Hình 2.10: Giao diện phần mềm Vivado Design Suite [24].

Vivado cung cấp các báo cáo sau tổng hợp và hiện thực để tinh chỉnh kiến trúc RTL. Báo cáo sử dụng tài nguyên thống kê LUT, thanh ghi, Block RAM và DSP; báo cáo timing cung cấp WNS, TNS và đường truyền tối hạn; trong khi công cụ phân tích công suất ước tính công suất tĩnh và động. Các dữ liệu này tạo cơ sở định lượng cho việc tối ưu thiết kế SoC [16,23].

2.10 Nền tảng FPGA Xilinx 7 Series và thiết bị mục tiêu

Họ FPGA Xilinx 7 Series cung cấp các tài nguyên LUT, Flip-Flop, Block RAM, bộ đệm clock toàn cục và phần tử I/O chuyên dụng để hiện thực các hệ thống SoC tùy biến. Hình 2.11 minh họa một bo mạch Zynq XC7Z030 thuộc cùng thế hệ công nghệ 7 Series, trong đó vùng logic lập trình được có thể chứa lõi RISC-V, hệ thống bus AXI4-Lite và các ngoại vi tùy biến [25–27].

Trong môi trường Vivado phiên bản miễn phí được sử dụng cho đề tài, bo mạch Zynq tham khảo không có sẵn trong danh sách thiết bị mục tiêu phù hợp với cấu hình công cụ. Vì vậy, khóa luận lựa chọn FPGA Artix-7 xc7a200tfbg676-2L làm thiết bị tương đương để thực hiện mô phỏng, tổng hợp và phân tích sau hiện thực [29]. Artix-7 và phần logic lập trình của Zynq đều thuộc họ công nghệ Xilinx 7 Series, sử dụng các tài nguyên cơ bản tương đồng như LUT, Flip-Flop, Block RAM, mạng clock và SelectIO. Ngoài ra, hệ thống trong đề tài sử dụng lõi RISC-V PicoRV32 hiện thực bằng RTL, không sử dụng bộ xử lý ARM cứng của Zynq; do đó, việc chuyển mục tiêu sang Artix-7 không làm thay đổi kiến trúc chức năng cần kiểm chứng. Thiết bị này được dùng để mô phỏng khả năng triển khai trên một nền tảng FPGA tương tự, đồng thời tạo các báo cáo tài nguyên, định thời và công suất cho thiết kế SoC.



Hình 2.11: Bo mạch Zynq XC7Z030 thuộc nền tảng Xilinx 7 Series [27].

Ảnh bo mạch được sử dụng để minh họa cách các tài nguyên FPGA 7 Series được tích hợp với bộ nhớ và ngoại vi trên một nền tảng phát triển [27]. Thiết kế trong khóa luận được tổng hợp như một hệ thống FPGA độc lập trên Artix-7; vì vậy, các kết quả tài nguyên, timing và công suất ở Chương 5 đều được tính theo cấu hình xc7a200tfbg676-2L trên AMD Vivado 2025.2 [23, 29].

Các thông số thiết bị trong Bảng 2.2 được tổng hợp từ tài liệu DS181 và báo cáo thiết bị của Vivado [23, 29]. Các tài nguyên này tạo không gian đủ lớn để triển khai lõi PicoRV32, bộ nhớ chương trình, bus AXI4-Lite, các ngoại vi UART/SPI/I²C và khối phát DDR. Việc sử dụng cùng mã thiết bị trong mô tả nền tảng và báo cáo thực nghiệm giúp các tỷ lệ sử dụng tài nguyên ở Chương 5 được đối chiếu nhất quán.

Bên cạnh tài nguyên logic, việc triển khai yêu cầu khai báo đúng clock, reset, chân I/O và chuẩn điện áp trong tệp .xdc. Các tín hiệu UART, SPI, I2C và DDR phải được ánh xạ đúng chân vật lý; đồng thời chuẩn điện áp của từng I/O bank phải phù hợp với phần cứng để bảo đảm tính toàn vẹn tín hiệu [16].

Quy trình kiểm chứng dựa trên mô phỏng RTL và báo cáo sau hiện thực để đánh giá clock, routing và các ràng buộc I/O. Đo dạng sóng trực tiếp bằng máy hiện

Bảng 2.2: Thông số cấu hình và tài nguyên của FPGA Artix-7 mục tiêu

Nhóm thông số	Giá trị hoặc đặc điểm kỹ thuật
Họ thiết bị	Xilinx/AMD Artix-7
Mã thiết bị	xc7a200tfbg676-2L
Slice LUTs khả dụng	134,600
Slice Registers khả dụng	269,200
Block RAM Tile (RAMB36E1)	365
Bonded I/O (IOB)	400
Global Clock Buffer (BUFG)	32
Clock hệ thống mục tiêu	100 MHz
Tốc độ dữ liệu DDR mục tiêu	200 Mbps/pin

sóng hoặc bộ phân tích logic là bước kiểm chứng phần cứng tiếp theo.

Chương 3

CÁC CÔNG TRÌNH NGHIÊN CỨU LIÊN QUAN

3.1 Tổng quan các hướng nghiên cứu về SoC trên nền RISC-V

Sự phát triển mạnh mẽ của kiến trúc tập lệnh mở RISC-V [13] đã thúc đẩy nhiều nghiên cứu thiết kế vi xử lý và hệ thống trên chip (SoC) tích hợp trên nền tảng FPGA. Nhằm tối ưu hóa diện tích thực thi và mức tiêu thụ tài nguyên cho các ứng dụng nhúng gọn nhẹ, lõi PicoRV32 [12] được phát triển như một bộ xử lý 32-bit (RV32I/IMC) có kích thước siêu nhỏ gọn. Tuy nhiên, bản thân công trình PicoRV32 [12] chỉ tập trung vào việc cung cấp lõi CPU thuần túy với giao diện bộ nhớ đơn giản (Native Memory Interface). Việc triển khai lõi này vào một SoC hoàn chỉnh đòi hỏi phải ghép nối thêm hệ thống cầu nối bus chuẩn hóa và các khối giao tiếp ngoại vi phù hợp.

Ở quy mô phức tạp hơn, BlackParrot được phát triển như một kiến trúc RISC-V đa nhân, có khả năng làm máy chủ cho các bộ tăng tốc phần cứng [1]. Simply-V hướng tới một nền tảng soft-SoC linh hoạt, hỗ trợ thay đổi CPU, IP và bộ tăng tốc phục vụ nghiên cứu, tạo mẫu nhanh [30]. Bên cạnh đó, Ma và cộng sự tập trung tích hợp các lõi mật mã độ trễ thấp vào SoC RISC-V [31]. Các công trình này chủ yếu giải quyết kiến trúc xử lý, khả năng cấu hình và tích hợp bộ tăng tốc; chưa khảo sát trực tiếp mô hình ngoại vi AXI4-Lite kết hợp một giao diện DDR I/O như hướng tiếp cận của khóa luận.

3.2 Các giải pháp chuẩn hóa giao tiếp ngoại vi với bus AXI4-Lite

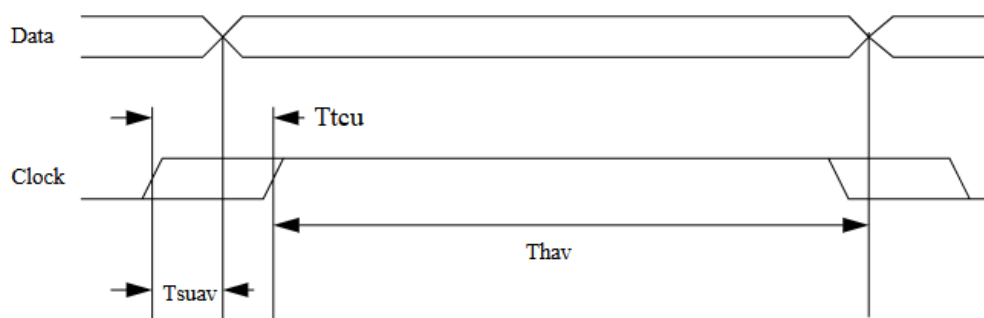
Để truyền tải dữ liệu giữa lõi xử lý và các mô-đun ngoại vi, các chuẩn giao tiếp nối tiếp như UART [2–4, 17], SPI [21] và I2C [19] được sử dụng phổ biến trong hệ thống nhúng và có thể hiện thực trên FPGA. Ở phía kết nối nội bộ, AMBA AXI4-Lite cung cấp một giao diện ánh xạ bộ nhớ gọn nhẹ để CPU truy cập các thanh ghi điều khiển và trạng thái của ngoại vi [6, 7].

Theo đặc tả AMBA AXI, AXI4-Lite là tập con của AXI4 dành cho các giao dịch ánh xạ bộ nhớ đơn, không hỗ trợ truyền burst. Giao diện gồm 5 kênh: địa chỉ ghi (AW), dữ liệu ghi (W), phản hồi ghi (B), địa chỉ đọc (AR) và dữ liệu đọc (R). Mỗi kênh sử dụng cơ chế bắt tay VALID/READY; do các kênh đọc và ghi được tách biệt, các pha bắt tay có thể tiến triển độc lập tùy theo khả năng của Master, Slave và khối liên kết [5–7].

Các nghiên cứu UART được khảo sát chủ yếu tập trung vào cấu trúc bộ phát/thu, tạo tốc độ baud và kiểm chứng khung dữ liệu, chưa đặt trọng tâm vào giao diện I/O truyền dữ liệu ở cả hai cạnh clock [2–4]. Đối với một giao diện đồng bộ, tăng tần số clock là một cách nâng thông lượng nhưng đồng thời làm thu hẹp chu kỳ dành cho đường dữ liệu, khiến yêu cầu timing closure chặt chẽ hơn và có thể làm tăng công suất động [16]. Vì vậy, kỹ thuật DDR là một hướng khác để tăng số lần truyền trên mỗi chu kỳ mà không nhất thiết tăng tần số clock.

3.3 Nghiên cứu về kỹ thuật DDR và quản lý Timing ở tần số cao

Kỹ thuật Double Data Rate (DDR) [14] là phương pháp truyền dữ liệu tại cả cạnh lên và cạnh xuống của xung nhịp, giúp tăng gấp đôi lưu lượng truyền lý thuyết tại cùng một tần số clock. Trong các hệ thống số, kỹ thuật DDR được ứng dụng ở hai mức độ khác nhau: bộ điều khiển bộ nhớ chính DDR SDRAM (Memory Controller) [8, 11, 32–35] và khối giao tiếp truyền nhận dữ liệu tốc độ cao (DDR I/O Interface) [9, 10, 36].



Hình 3.1: So sánh mối quan hệ định thời ngõ vào (Input Timing) giữa Clock và Data ở phương thức căn lề cạnh (Edge-aligned) và căn lề giữa (Center-aligned) tại phía nhận (chuyển thể từ [37]).

Trong giao tiếp đồng bộ nguồn (Source-Synchronous), một thách thức quan trọng của khối nhận là đặt cạnh lấy mẫu vào vùng dữ liệu ổn định và duy trì đủ biên

setup/hold [37–39]:

- Mô hình căn lề cạnh (Edge-aligned): Cạnh chuyển của clock phát gần với cạnh chuyển dữ liệu. Vì vậy, phía nhận thường phải điều chỉnh pha clock hoặc sử dụng tài nguyên trễ thích hợp để đưa cạnh lấy mẫu vào trong cửa sổ dữ liệu hợp lệ, tránh làm giảm biên t_{setup} và t_{hold} [37, 39].
- Mô hình căn lề giữa (Center-aligned): Cạnh clock lấy mẫu được bố trí gần tâm của cửa sổ dữ liệu hợp lệ. Với DDR lý tưởng, độ lệch tương ứng khoảng một phần tư chu kỳ clock so với cạnh chuyển dữ liệu, nhờ đó cân bằng biên setup và hold tại phía nhận [37, 38].

Do đó, các ràng buộc `set_output_delay` và `set_input_delay` trong tệp XDC phải mô tả đúng quan hệ định thời giữa clock và dữ liệu. Đối với giao diện DDR, các ràng buộc cần bao phủ cả cạnh lên và cạnh xuống, thường sử dụng các tùy chọn `-clock_fall` và `-add_delay`, để Vivado phân tích đầy đủ các kiểm tra setup/hold của giao diện [23, 38].

Các nghiên cứu về bộ điều khiển DDR SDRAM tập trung giải quyết chuỗi lệnh và các tham số định thời của bộ nhớ vật lý, chẳng hạn t_{RP} , t_{RCD} và độ trễ CAS [8, 34, 35]. Khóa luận này không thiết kế một bộ điều khiển DDR SDRAM; thay vào đó, đề tài sử dụng các phần tử DDR I/O để phát và thu dữ liệu hai cạnh clock, đồng thời ánh xạ khối xử lý vào không gian thanh ghi AXI4-Lite để CPU điều khiển.

3.4 So sánh tổng hợp và định hướng của đề tài

Từ phạm vi của các nhóm công trình về lõi RISC-V, liên kết AXI và giao tiếp DDR, Bảng 3.1 tóm tắt điểm khác biệt giữa các hướng nghiên cứu tiêu biểu và định hướng của khóa luận [1, 5, 8–10, 12, 30].

Dựa trên việc phân tích phạm vi và kế thừa kết quả từ các nghiên cứu trước, khóa luận xác định mục tiêu xây dựng một hệ thống SoC RISC-V hoàn chỉnh dựa trên lõi PicoRV32 [12], tích hợp hệ thống bus AMBA AXI4-Lite và khối giao tiếp ngoại vi hỗ trợ truyền dữ liệu hai cạnh clock (DDR) [14]. Các mục tiêu cụ thể bao gồm:

1. Đóng gói chuẩn hóa ngoại vi AXI4-Lite: Xây dựng và đóng gói khối giao tiếp hai cạnh clock DDR thành một mô-đun ngoại vi AXI4-Lite, cho phép lõi CPU PicoRV32 cấu hình và trao đổi dữ liệu trực tiếp thông qua giao diện ánh xạ bộ nhớ [7, 12, 14].

Bảng 3.1: Bảng tổng hợp so sánh phạm vi các nghiên cứu liên quan và định hướng đề tài

Nghiên cứu/Tài liệu	Nền tảng và kiến trúc	Phạm vi tập trung	Định hướng trong khóa luận
PicoRV32 [12], Simply-V [30]	Lõi RISC-V gọn nhẹ và nền tảng soft-SoC cấu hình được	Tối ưu lõi CPU hoặc hỗ trợ tích hợp linh hoạt CPU, IP và bộ tăng tốc; chưa tập trung vào giao tiếp ngoại vi DDR.	Sử dụng PicoRV32 kết hợp bộ chuyển đổi Native Memory–AXI4-Lite và khối giao tiếp DDR.
AXI4-Lite [5, 7] và UART/SPI [2, 21]	Bus AXI4-Lite và ngoại vi nối tiếp	Chuẩn hóa truy cập thanh ghi bằng 5 kênh bắt tay độc lập; các nghiên cứu ngoại vi khảo sát chưa tập trung vào DDR I/O.	Tích hợp các ngoại vi AXI4-Lite với khối gia tốc truyền dữ liệu hai cạnh clock để tăng thông lượng.
Bộ điều khiển DDR SDRAM [8, 34, 35]	Bộ điều khiển bộ nhớ độc lập	Điều khiển DDR SDRAM ngoài chip với các tham số định thời bộ nhớ phức tạp.	Xây dựng khối DDR I/O gọn nhẹ, ánh xạ thanh ghi AXI4-Lite để CPU điều khiển trực tiếp.
DDR Buffer/NoC [9, 10, 36]	Bộ đệm tốc độ cao, DDR-NoC	Tăng tốc độ truyền nội bộ; chưa tập trung vào giao diện điều khiển ngoại vi AXI4-Lite.	Ứng dụng kỹ thuật Center-aligned và thiết lập XDC [38] để đạt $WNS \geq 0$.
Tài liệu Arm và AMD/Xilinx [7, 16, 28, 29, 38, 39]	AXI4-Lite, FPGA 7 Series và Vivado	Đặc tả giao thức AXI, tài nguyên SelectIO, đặc tính thiết bị và phương pháp ràng buộc timing.	Tổng hợp và đánh giá SoC AXI4-Lite trên FPGA Artix-7 XC7A200T.

2. Tối ưu lưu lượng giao tiếp ngoại vi: Khai thác cả cạnh lên và cạnh xuống để tăng gấp đôi số lần truyền dữ liệu lý thuyết trong mỗi chu kỳ so với SDR tại cùng tần số clock [10, 14].
3. Đạt trạng thái Timing Closure: Áp dụng phương pháp luận UltraFast [16] và thiết lập tệp ràng buộc XDC [38] với kỹ thuật Center-aligned trên phần mềm Vivado [23] nhằm kiểm soát các chỉ số dư thừa thời gian, bảo đảm chỉ số $WNS \geq 0$ khi hệ thống hoạt động ở tần số mục tiêu.
4. Kiểm chứng thực nghiệm: Tổng hợp và thực thi thiết kế SoC cho FPGA Artix-7, mã thiết bị xc7a200tfbg676-2L [28, 29]. Đánh giá thiết kế thông qua các báo cáo sử dụng tài nguyên, định thời và công suất của Vivado [16, 23].

Chương 4

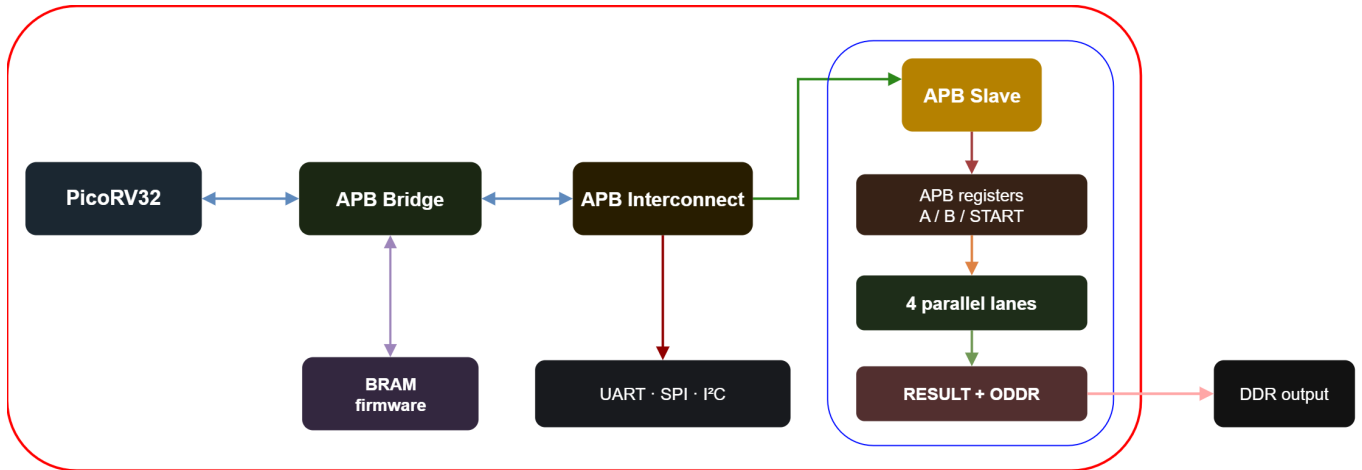
MÔ HÌNH VÀ PHƯƠNG PHÁP ĐỀ XUẤT

4.1 Tổng quan kiến trúc hệ thống đề xuất

Kiến trúc System-on-Chip (SoC) đề xuất được thiết kế nhằm tối ưu hóa băng thông giao tiếp ngoại vi và kiểm chứng khả năng truyền dữ liệu theo cơ chế Double Data Rate (DDR) trên nền tảng FPGA. Trung tâm của hệ thống là lõi xử lý RISC-V PicoRV32 đảm nhiệm vai trò điều khiển và phát các yêu cầu truy cập bộ nhớ. Để đảm bảo tính đóng gói, hiệu năng truyền tải cao và khả năng xử lý song song, hạ tầng bus AMBA AXI4-Lite được chọn làm tuyến bus nội bộ kết nối CPU với các mô-đun ngoại vi thay vì bus APB truyền thống [6, 12, 14].

Nhằm khẳng định rõ đóng góp khoa học của đề tài cũng như đảm bảo tính minh bạch học thuật, phạm vi thiết kế phần cứng được phân định cụ thể thành hai nhóm thành phần chính:

- Các khối kế thừa từ mã nguồn mở: Lõi RISC-V PicoRV32 đảm nhiệm vai trò CPU trung tâm, còn BRAM chứa mã chương trình khởi tạo (`firmware.hex`). Việc kế thừa giúp bảo đảm tính chuẩn hóa của hạ tầng xử lý lệnh và độ tin cậy khi thực thi phần mềm [12, 13].
- Các khối tự thiết kế và tích hợp: Hệ thống gồm bộ chuyển đổi `picorv32_axi_adapter`, bộ định tuyến AXI Crossbar đa kênh, khối xử lý song song đa làn và bộ phát dữ liệu hai cạnh clock. Các thành phần được thiết kế ở mức RTL bằng Verilog HDL và đóng gói vào SoC.



Hình 4.1: Sơ đồ khối tổng thể kiến trúc SoC RISC-V với khối xử lý song song và đầu ra DDR.

Toàn bộ hệ thống được chia thành hai miền chức năng chính gồm miền xử lý trung tâm và miền ngoại vi AXI. PicoRV32 lấy lệnh từ BRAM nội bộ thông qua giao diện Native Memory; các giao dịch dữ liệu đến ngoại vi được chuyển sang chuẩn AXI4-Lite bởi khối `picorv32_axi_adapter`. Hệ thống ngoại vi gồm UART, SPI, I2C và khối parallel. Khối parallel vừa thực hiện bốn phép cộng độc lập vừa phát bốn kết quả đó dưới dạng luồng DDR 32 bit. Sơ đồ tổng thể hệ thống được minh họa trên Hình 4.1.

4.2 Thiết kế chi tiết các khối chức năng

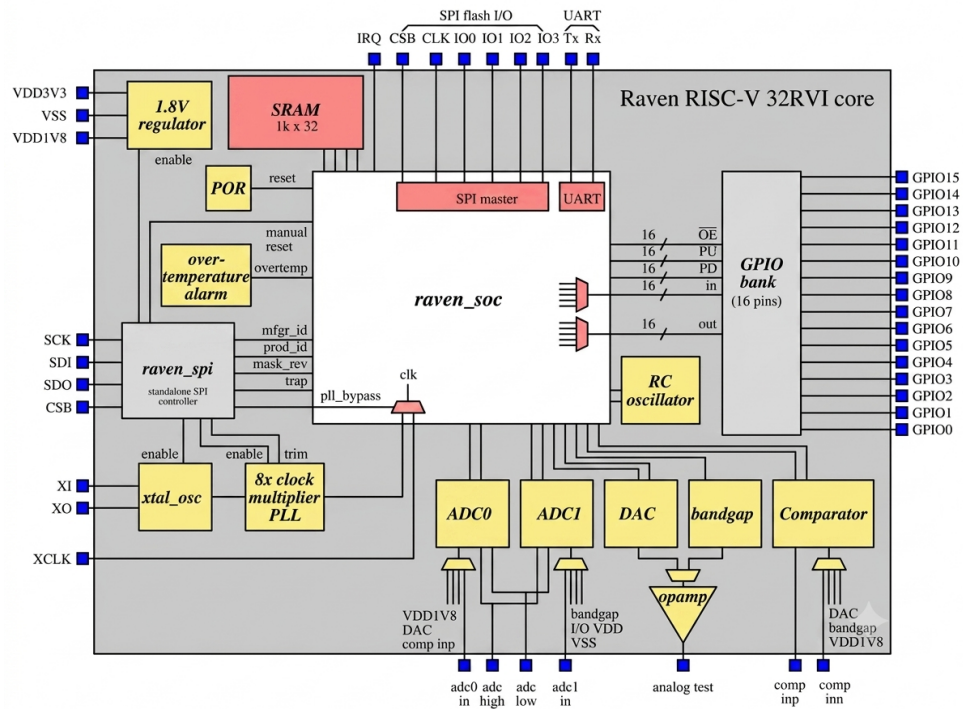
4.2.1 Lõi xử lý PicoRV32 và bộ giải mã địa chỉ

Lõi xử lý PicoRV32 được cấu hình hoạt động theo tập lệnh RV32I cơ bản, sử dụng giao diện Native Memory để trao đổi dữ liệu với bên ngoài. Giao diện này bao gồm các tín hiệu địa chỉ `mem_addr`, dữ liệu ghi `mem_wdata`, dữ liệu đọc `mem_rdata`, tín hiệu cho phép `mem_valid`, tín hiệu phản hồi `mem_ready` và tín hiệu chọn byte `mem_wstrb` [12, 13].

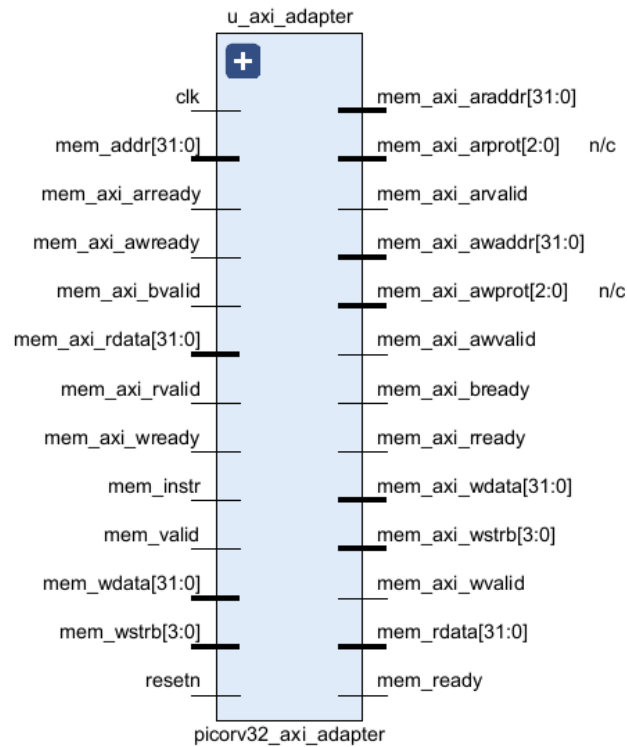
Không gian địa chỉ 32 bit được phân bổ như trong Bảng 4.1. Vùng địa chỉ thấp dành cho BRAM 4 KB, trong đó tệp `firmware.hex` được nạp làm chương trình khởi động. Các giao dịch đến vùng ngoại vi từ `0x4000_0000` đến `0x4000_3FFF` được chuyển qua AXI Adapter.

Tại phía Slave, mô-đun `soc_axi_top` đóng vai trò là một bộ định tuyến (Crossbar) giải mã địa chỉ nhằm dẫn luồng 5 kênh giao dịch của AXI đến đúng thiết

bị ngoại vi tương ứng.



Hình 4.2: Sơ đồ khối tham khảo của một hệ thống SoC tích hợp lõi xử lý RISC-V và các ngoại vi.



Hình 4.3: Sơ đồ RTL giao diện Native Memory của bộ điều khiển PicoRV32–AXI4-Lite.

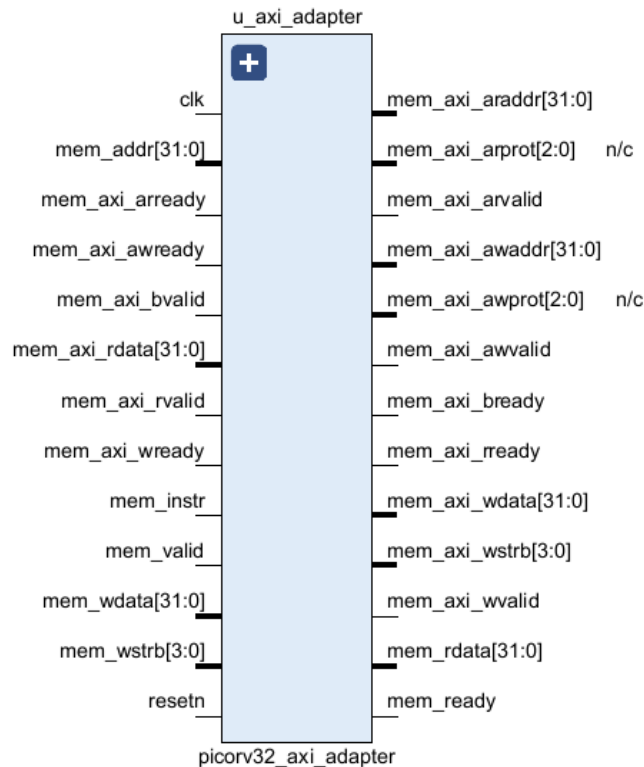
Bảng 4.1: Bản đồ phân bố không gian địa chỉ AXI4-Lite của hệ thống SoC

Vùng địa chỉ	Mô-đun mục tiêu	Chức năng kỹ thuật
0x0000_0000--0x0000_0FFF	BRAM (4 KB)	Chứa firmware khởi động
0x4000_0000--0x4000_0FFF	AXI UART	Giao tiếp nối tiếp bất đồng bộ
0x4000_1000--0x4000_1FFF	AXI SPI	Giao tiếp nối tiếp đồng bộ
0x4000_2000--0x4000_2FFF	AXI I2C	Giao tiếp chuẩn 2 dây
0x4000_3000--0x4000_3FFF	AXI Parallel/DDR	Bộ xử lý song song 4 làn và giao tiếp DDR

4.2.2 Bộ chuyển đổi bus picorv32_axi_adapter

Khối picorv32_axi_adapter chuyển các giao dịch dữ liệu của giao diện Native Memory sang chuẩn AMBA AXI4-Lite. Giao dịch lấy lệnh được nhận biết bằng mem_

instr và trả về trực tiếp từ BRAM qua bram_rdata trong vòng 1 chu kỳ; giao dịch dữ liệu được biên dịch thành các kênh giao tiếp độc lập (Kênh Ghi AW/W/B và Kênh Đọc AR/R). Bộ chuyển đổi sử dụng máy trạng thái FSM với ba trạng thái cốt lõi: S_IDLE, S_WRITE_WAIT và S_READ_WAIT. Cách tổ chức này bảo đảm việc đàm phán tín hiệu (Handshake) ‘VALID/READY’ trên bus AXI được duy trì ổn định cho đến khi ngoại vi xử lý xong [6, 12].



Hình 4.4: Sơ đồ RTL các kênh tín hiệu của bộ chuyển đổi *picorv32_axi_adapter*.

Hình 4.5 trình bày phần RTL tương ứng của máy trạng thái trong thiết kế.

```
always @(posedge clk or negedge resetn) begin
    if (!resetn) begin
        state <= S_IDLE; mem_ready <= 1'b0;
        mem_axi_awvalid <= 0; mem_axi_wvalid <= 0; mem_axi_arvalid <= 0;
    end else begin
        mem_ready <= 1'b0;
        case (state)
            S_IDLE: begin
                if (mem_valid && is_mmio) begin
                    if (!mem_wstrb) begin // Giao dịch Ghi
                        mem_axi_awvalid <= 1'b1;
                        mem_axi_wvalid <= 1'b1;
                    end
                end
            end
        endcase
    end
end
```

```

        state          <= S_WRITE_WAIT;
    end else begin      // Giao dịch Doc
        mem_axi_arvalid <= 1'b1;
        state          <= S_READ_WAIT;
    end
end
end
S_WRITE_WAIT: begin
    // Xoa co VALID khi Slave bao READY
    if (mem_axi_awready) mem_axi_awvalid <= 1'b0;
    if (mem_axi_wready)  mem_axi_wvalid  <= 1'b0;
    // Cho Slave tra ve BVALID (Hoan tat ghi)
    if (mem_axi_bvalid) begin
        mem_ready <= 1'b1;
        state     <= S_IDLE;
    end
end
S_READ_WAIT: begin
    if (mem_axi_arready) mem_axi_arvalid <= 1'b0;
    // Cho Slave tra ve RVALID (Du lieu doc hop le)
    if (mem_axi_rvalid) begin
        mem_rdata <= mem_axi_rdata;
        mem_ready <= 1'b1;
        state     <= S_IDLE;
    end
end
endcase
end
end

```

Hình 4.5: Máy trạng thái chuyển đổi Native Memory sang AXI4-Lite trong *picorv32_axi_adapter*.

Cơ chế hoạt động và phân tích chi tiết mã nguồn FSM

Đoạn mã trong Hình 4.5 thực hiện quá trình chuyển đổi giao thức (Protocol Translation) giữa giao diện bus Native Memory của PicoRV32 và bus AMBA AXI4-Lite. Luồng xử lý chi tiết diễn ra như sau:

1. **Trạng thái rảnh (S_IDLE):** Mạch giám sát tín hiệu `mem_valid`. Khi phát hiện truy xuất đến vùng ngoại vi (`is_mmio`), nếu đây là giao dịch Ghi (`!mem_wstrb > 0`), FSM kích hoạt song song kênh địa chỉ (`AWVALID`) và kênh dữ liệu

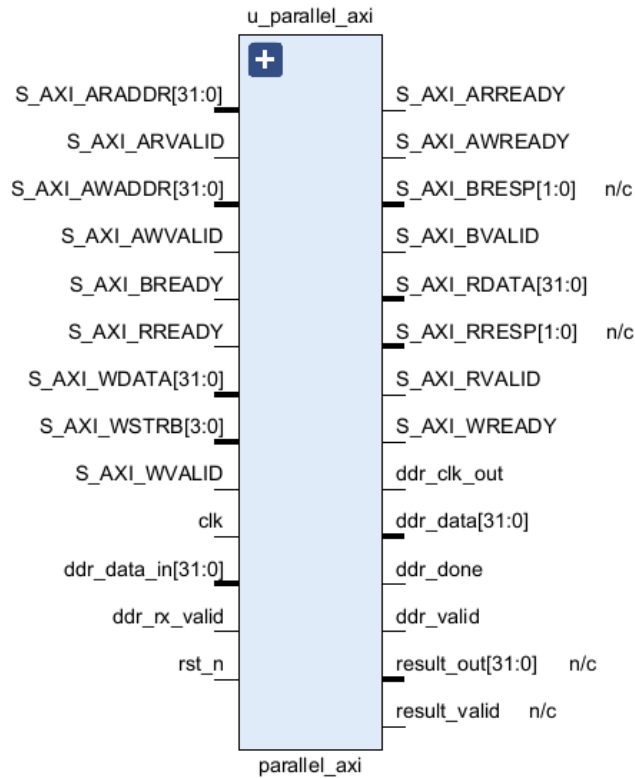
(WVALID) rồi chuyển sang S_WRITE_WAIT. Nếu là giao dịch Đọc, FSM kích hoạt kênh đọc địa chỉ (ARVALID) và rẽ nhánh sang S_READ_WAIT.

2. **Trạng thái chờ Ghi (S_WRITE_WAIT):** Adapter theo dõi cờ READY của Slave trên cả hai kênh AW và W. Ngay khi Slave chấp nhận, cờ VALID tương ứng sẽ được hạ xuống. Giao dịch chỉ hoàn tất và trả tín hiệu mem_ready=1 về CPU khi Slave phản hồi thành công qua kênh B (BVALID=1).
3. **Trạng thái chờ Đọc (S_READ_WAIT):** Tương tự, Adapter chờ Slave phản hồi dữ liệu qua kênh R. Khi RVALID=1, dữ liệu mem_axi_rdata được chép vào mem_rdata và kết thúc giao dịch.

Tác dụng chính của khối Adapter là bóc tách một giao dịch đơn khối của CPU thành luồng tín hiệu 5 kênh độc lập của AXI4-Lite, giải quyết bài toán đồng bộ hóa và chống hiện tượng vấp tín hiệu (glitch) trong hệ thống.

4.2.3 Khối xử lý song song và truyền dữ liệu DDR

Khối parallel được ánh xạ tại vùng 0x4000_3000--0x4000_3FFF và đóng vai trò bộ tăng tốc song song đa lần. Khối tích hợp đầu ra DDR để thực hiện đồng thời các phép tính do CPU giao và tuần tự hóa kết quả phát ra ngoài chip.



Hình 4.6: Sơ đồ RTL của khối xử lý song song và giao tiếp dữ liệu DDR.

1. Cơ chế ghi/đọc thanh ghi và kích hoạt tính toán

Hệ thống bố trí tập thanh ghi AXI để CPU tương tác với khối xử lý song song:

- Thanh ghi toán hạng A (Offset 0x10--0x1C): Chứa 4 giá trị toán hạng $A[0]$ đến $A[3]$ (mỗi giá trị 32 bit).
- Thanh ghi toán hạng B (Offset 0x20--0x2C): Chứa 4 giá trị toán hạng $B[0]$ đến $B[3]$ (mỗi giá trị 32 bit).
- Thanh ghi điều khiển START (Offset 0x00): Khi CPU ghi bit 1 vào thanh ghi này, tín hiệu kích hoạt được phát ra để 4 lane đồng thời thực hiện phép cộng 32 bit: $Result[i] = A[i] + B[i]$ (với $i = 0 \dots 3$).
- Thanh ghi kết quả (Offset 0x30--0x3C): Chứa 4 kết quả tính toán. Đường đọc kênh R của AXI cho phép CPU truy xuất lại các thanh ghi này bất kỳ lúc nào.

2. Cơ chế đóng gói và phân tích mã nguồn bộ phát DDR

Song song với việc lưu kết quả vào thanh ghi để CPU đọc, toàn bộ 128 bit kết quả ($4 \text{ lần} \times 32 \text{ bit}$) được nạp vào bộ tuần tự hóa `ddr_result_serializer` nhằm đẩy ra các chân I/O ngoài chip qua 2 chu kỳ clock (tương ứng 4 cạnh xung nhịp).

Hình 4.7 thể hiện đoạn mã kết nối giữa luồng kết quả tính toán và khối tuần tự hóa DDR.

```
// Generate the load condition for the DDR serializer
assign fire_start = start_pending & core_in_ready & {NUM_LANES{ddr_ready}};

// Serialize four 32-bit results over four clock edges
ddr_result_serializer #(.DATA_WIDTH(32)) u_ddr_result_serializer (
    .clk          (clk),
    .rst_n        (rst_n),
    .load         (!fire_start),
    .data_rise_0   (result_flat[31:0]),
    .data_fall_0   (result_flat[63:32]),
    .data_rise_1   (result_flat[95:64]),
    .data_fall_1   (result_flat[127:96]),
    .ddr_data      (ddr_data),
    .ddr_valid     (ddr_valid),
    .ddr_done      (ddr_done),
    .ddr_ready     (ddr_ready)
);
```

Hình 4.7: Giao tiếp giữa kết quả các lần tính toán và bộ phát DDR.

Giải thích đoạn mã trong Hình 4.7:

1. Dòng 2 (`assign fire_start`): Tín hiệu `fire_start` đóng vai trò là điều kiện bắt tay (handshake). Lệnh tính toán chỉ thực sự được nạp khi có yêu cầu thực thi (`start_pending`), lỗi tính toán rảnh (`core_in_ready`), và quan trọng nhất là khối phát DDR đang sẵn sàng (`ddr_ready=1`). Điều này ngăn chặn hiện tượng ghi đè dữ liệu khi batch trước chưa phát xong.
2. Dòng 8 (`.load`): Sử dụng phép OR thu gọn (`!fire_start`) để tạo xung nạp dữ liệu. Khi có hiệu lệnh, 128 bit kết quả ghép từ 4 lane (`result_flat`) lập tức được chốt vào các thanh ghi đầu vào của bộ phát.
3. Dòng 9–12 (`.data_rise_X`, `.data_fall_X`): Mảng dữ liệu 128 bit được chia thành 4 từ dữ liệu 32 bit:

- data_rise_0 (Lane 0): Phát tại cạnh lên của chu kỳ clock thứ nhất.
 - data_fall_0 (Lane 1): Phát tại cạnh xuống của chu kỳ clock thứ nhất.
 - data_rise_1 (Lane 2): Phát tại cạnh lên của chu kỳ clock thứ hai.
 - data_fall_1 (Lane 3): Phát tại cạnh xuống của chu kỳ clock thứ hai.
4. Dòng 13–16 (ddr_data, ddr_valid, ddr_done, ddr_ready): Quản lý luồng đầu ra. ddr_valid=1 kéo lên trong suốt 2 chu kỳ phát dữ liệu; ddr_done phát ra một xung ngắn đánh dấu hoàn tất truyền dữ liệu của một đợt tính toán.

3. Nguyên lý phát dữ liệu hai cạnh xung nhịp qua Primitive ODDR

Để đưa bus dữ liệu 32 bit ra chân vật lý của FPGA với tốc độ gấp đôi mà không cần tăng gấp đôi tần số clock hệ thống, thiết kế sử dụng phần tử phần cứng chuyên dụng Output Double Data Rate (ODDR) nằm tại khối Input/Output Block (IOB) của chip AMD/Xilinx [39].

Hình 4.8 mô tả cách khởi tạo primitive ODDR cho từng đường truyền dữ liệu.

```
// Instantiate one ODDR primitive for each output bit
ODDR #(
    .DDR_CLK_EDGE ("SAME_EDGE"),
    .INIT          (1'b0),
    .SRTYPE        ("ASYNC")
) u_oddr (
    .Q  (ddr_data[bit_index]),
    .C  (clk),
    .CE (1'b1),
    .D1 (rise_word[bit_index]),
    .D2 (fall_word[bit_index]),
    .R  (~rst_n),
    .S  (1'b0)
);
```

Hình 4.8: Khởi tạo phần tử cứng ODDR trong AMD/Xilinx FPGA.

Phân tích chuyên sâu về phần tử cứng ODDR (Hình 4.8):

- **Cấu hình** DDR_CLK_EDGE="SAME_EDGE": Trong chế độ này, dữ liệu đưa vào D1 và D2 được tổ chức để phát lần lượt tại cạnh lên và cạnh xuống của xung nhịp C, trong khi việc nạp dữ liệu được đồng bộ theo cùng một cạnh clock. Cấu hình này đơn giản hóa đường định thời dữ liệu trong lớp logic FPGA [39].

- **Ngõ ra Q:** Được nối thẳng tới chân I/O vật lý của chip FPGA (`ddr_data[bit_index]`). Nhờ sử dụng primitive IOB chuyên dụng thay vì dùng Flip-Flop thông thường trong lát logic (CLB), độ trễ đường truyền từ thanh ghi ra chân chip được giảm thiểu và có tính đồng đều cao giữa các đường truyền của bus 32 bit.

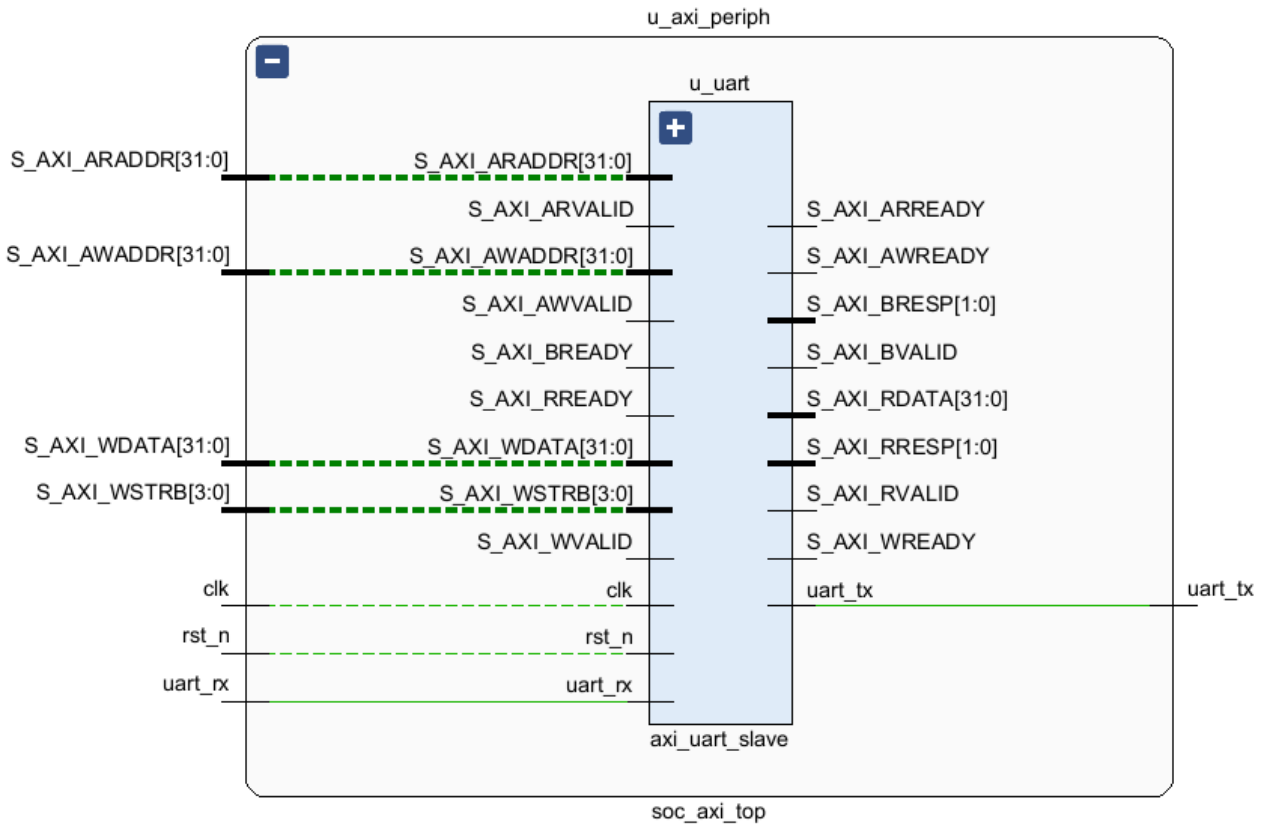
4.2.4 Các khối ngoại vi chuẩn giao tiếp nối tiếp (UART, SPI, I2C)

Bên cạnh khối xử lý song song và đầu ra DDR, hệ thống SoC còn tích hợp các khối ngoại vi chuẩn giao tiếp nối tiếp đóng vai trò là giao diện kết nối giữa SoC với các linh kiện ngoài chip và hệ thống giám sát:

- Khối AXI UART Slave (0x4000_0000): Đảm nhiệm vai trò truyền nhận dữ liệu chuỗi bất đồng bộ với máy tính chủ (Host PC) phục vụ mục đích truyền nhận dữ liệu kiểm thử và xuất log điều khiển (debug).
- Khối AXI SPI Slave (0x4000_1000): Cung cấp giao diện truyền dữ liệu nối tiếp đồng bộ tốc độ cao, hỗ trợ kết nối với các thiết bị ngoại vi như bộ nhớ Flash ngoài hoặc màn hình hiển thị.
- Khối AXI I2C Slave (0x4000_2000): Cung cấp giao diện giao tiếp 2 dây chuẩn (SDA, SCL) phục vụ việc đọc/ghi dữ liệu cấu hình từ các cảm biến hoặc IC quản lý nguồn.

Tất cả các mô-đun ngoại vi trên đều được đóng gói chuẩn giao diện AXI4-Lite, kết nối trực tiếp vào bộ định tuyến Crossbar của `soc_axi_top` và được quản lý thông qua tập thanh ghi cấu hình tương ứng trong bảng bản đồ bộ nhớ (Bảng 4.1).

Khối giao tiếp nối tiếp bất đồng bộ (AXI UART Slave)



Hình 4.9: Sơ đồ RTL của khối giao tiếp AXI UART.

Mô-đun axi_uart_slave thực hiện việc truyền nhận dữ liệu bất đồng bộ chuẩn UART. Kiến trúc chính bao gồm giao diện AXI4-Lite, bộ phát (TX) và bộ nhận (RX) độc lập. Cấu trúc khung truyền và nguyên tắc xác định tốc độ baud được xây dựng theo đặc tả UART thông dụng [17]. Các đoạn mã Verilog cốt lõi được thể hiện trong Hình 4.10.

```
// 1. Gia ma giao dien AXI Write Handshake
wire slv_reg_wren = axi_wready && S_AXI_WVALID && axi_awready && S_AXI_AWVALID;
always @(posedge S_AXI_ACLK or negedge S_AXI_ARESETN) begin
    if (!S_AXI_ARESETN) begin
        tx_data <= 8'h0; tx_start <= 1'b0;
    end else begin
        tx_start <= 1'b0;
        if (slv_reg_wren && (S_AXI_AWADDR[7:0] == 8'h00)) begin
            tx_data <= S_AXI_WDATA[7:0];
            tx_start <= 1'b1; // Kich hoat truyen TX
        end
    end
end
```

```

    end
end

// 2. TX shift register (PISO)
always @(posedge S_AXI_ACLK or negedge S_AXI_ARESETN) begin
    if (!S_AXI_ARESETN) begin ... end
    else if (tx_start && !tx_active) begin
        tx_active    <= 1'b1;
        tx_shift_reg <= {1'b1, tx_data, 1'b0}; // Khung: Stop(1) + Data(8) + Start
        (0)
    end else if (tx_active && clk_cnt == BIT_PERIOD - 1) begin
        tx_shift_reg <= tx_shift_reg >> 1;    // Dịch phải tung bit
        if (bit_idx == 9) tx_active <= 1'b0;
    end
end
end

// 3. Two-stage RX synchronizer and start-bit detector
reg [1:0] rx_sync;
always @(posedge S_AXI_ACLK) rx_sync <= {rx_sync[0], uart_rx};

always @(posedge S_AXI_ACLK or negedge S_AXI_ARESETN) begin
    if (!S_AXI_ARESETN) begin ... end
    else begin
        if (rx_clear) rx_done <= 1'b0;
        if (!rx_active && (rx_sync[1] == 1'b0)) begin // Phát hiện cạnh xuống Start
            bit
            rx_active <= 1'b1;
        end else if (rx_active && rx_clk_cnt == BIT_PERIOD - 1) begin
            if (rx_bit_idx >= 1 && rx_bit_idx <= 8)
                rx_shift_data <= {rx_sync[1], rx_shift_data[7:1]}; // Lay mau du
            lieu RX
        end
    end
end
end
end

```

Hình 4.10: Các đoạn mã RTL cốt lõi của mô-đun `axi_uart_slave`.

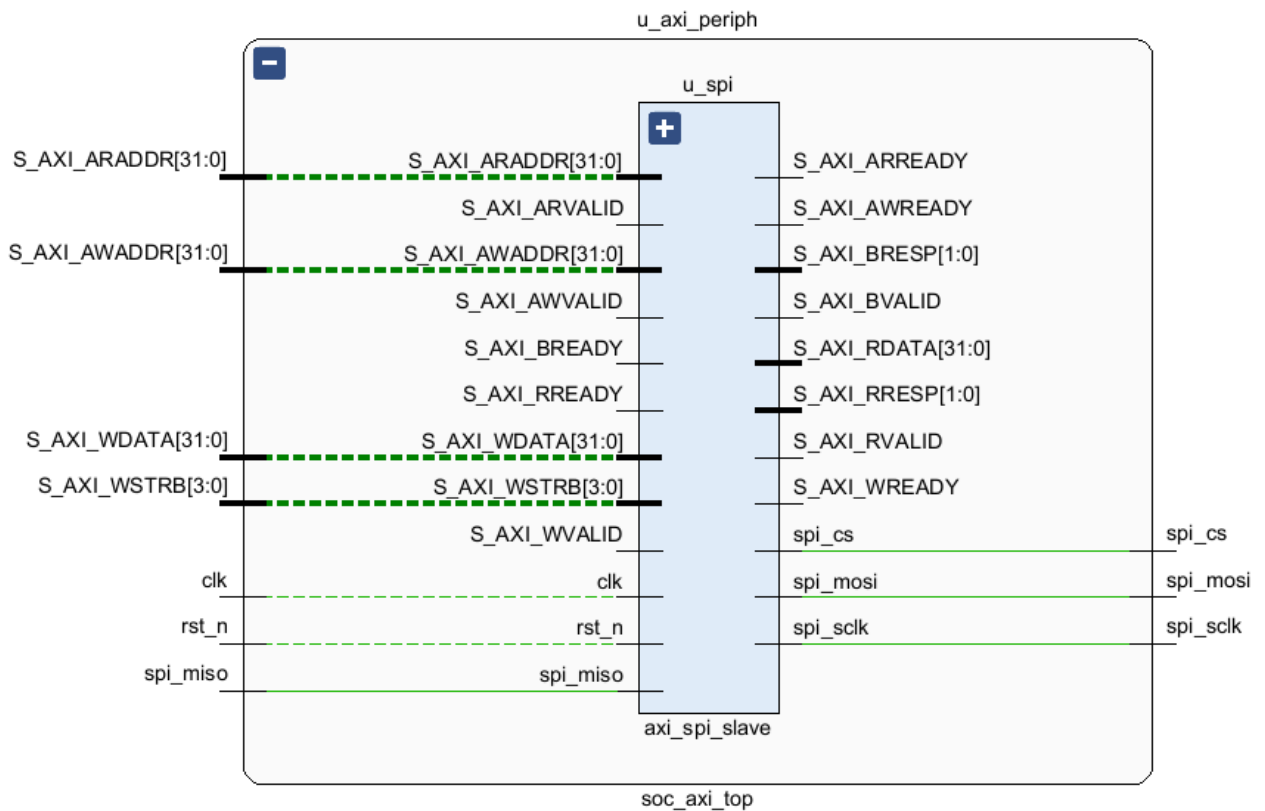
Giải thích mã nguồn khối `axi_uart_slave`:

- Giải mã AXI Write: Tín hiệu ghi dữ liệu `slv_reg_wren` chỉ được bật lên khi kênh AW và kênh W cùng thiết lập trạng thái bắt tay thành công (VALID và READY đồng thời bằng 1).
- Phân giải baud rate: Tốc độ truyền được tính toán qua hằng số `BIT_PERIOD=`

CLK_FREQ/BAUD_RATE. Mạch đếm `clk_cnt` chia tần số để định thời chính xác khoảng thời gian của 1 bit.

- Khối truyền (TX): Thanh ghi dịch `tx_shift_reg` 10 bit được nạp định dạng bao gồm 1 bit Start (0), 8 bit dữ liệu `tx_data`, và 1 bit Stop (1). Dữ liệu được dịch phải tuần tự ra đường phát `uart_tx`.
- Khối nhận (RX) và chống hiện tượng bất ổn định: Đường vào `uart_rx` được đưa qua thanh ghi đồng bộ 2 tầng `rx_sync` nhằm loại bỏ hiện tượng bất ổn định trạng thái (Metastability) khi tín hiệu bất đồng bộ đi vào miền xung nhịp `S_AXI_ACLK`. Khi phát hiện cạnh xuống (bit Start), mạch nhận thực hiện dịch 8 bit dữ liệu vào `rx_shift_data` và kích hoạt cờ `rx_done`. Cờ này tự động xóa (`rx_clear`) khi CPU thực hiện thao tác đọc thanh ghi dữ liệu nhận qua kênh R của bus AXI.

Khối giao tiếp nối tiếp đồng bộ (AXI SPI Slave)



Hình 4.11: Sơ đồ RTL của khối giao tiếp AXI SPI.

Mô-đun `axi_spi_slave` chuyển đổi các truy xuất AXI4-Lite thành giao thức truyền nối tiếp 4 dây SPI Master. Quan hệ giữa cực tính, pha xung nhịp và thời điểm lấy mẫu dữ liệu được thiết lập theo SPI Mode 0 [21]. Các đoạn mã FSM và dịch dữ liệu quan trọng được trình bày tại Hình 4.12.

```
// 1. Gia ma thanh ghi Chip Select (CS) qua AXI
wire slv_reg_wren = axi_wready && S_AXI_WVALID && axi_awready && S_AXI_AWVALID;
always @(posedge S_AXI_ACLK or negedge S_AXI_ARESETN) begin
    if (!S_AXI_ARESETN) spi_cs <= 1'b1;
    else if (slv_reg_wren && S_AXI_AWADDR[7:0] == 8'h04)
        spi_cs <= S_AXI_WDATA[0]; // Dieu khien spi_cs bang phan mem
end

// 2. May trang thai (FSM) dieu khien SCLK va truyen nhan SPI (Mode 0)
always @(posedge S_AXI_ACLK or negedge S_AXI_ARESETN) begin
    if (!S_AXI_ARESETN) begin ... end
    else begin
        case (state)
            0: begin // IDLE
                spi_sclk <= 1'b0;
                if (start_tx && !spi_cs) begin
                    spi_busy <= 1'b1;
                    tx_shifter <= tx_data_buf;
                    state <= 1;
                end
            end
            1: begin // Canh len SCLK - Lay mau du lieu MISO
                spi_sclk <= 1'b1;
                if (clk_div == 8'd5) begin
                    rx_data_buf <= {rx_data_buf[6:0], spi_miso};
                    state <= 2;
                end
            end
            2: begin // Falling SCLK edge: drive the next MOSI bit
                spi_sclk <= 1'b0;
                if (clk_div == 8'd5) begin
                    if (bit_cnt < 7) begin
                        bit_cnt <= bit_cnt + 1;
                        tx_shifter <= tx_shifter << 1; // Dich bit MSB ra MOSI
                        state <= 1;
                    end else state <= 0; // Hoan thanh 8-bit
                end
            end
        endcase
    end
end
```



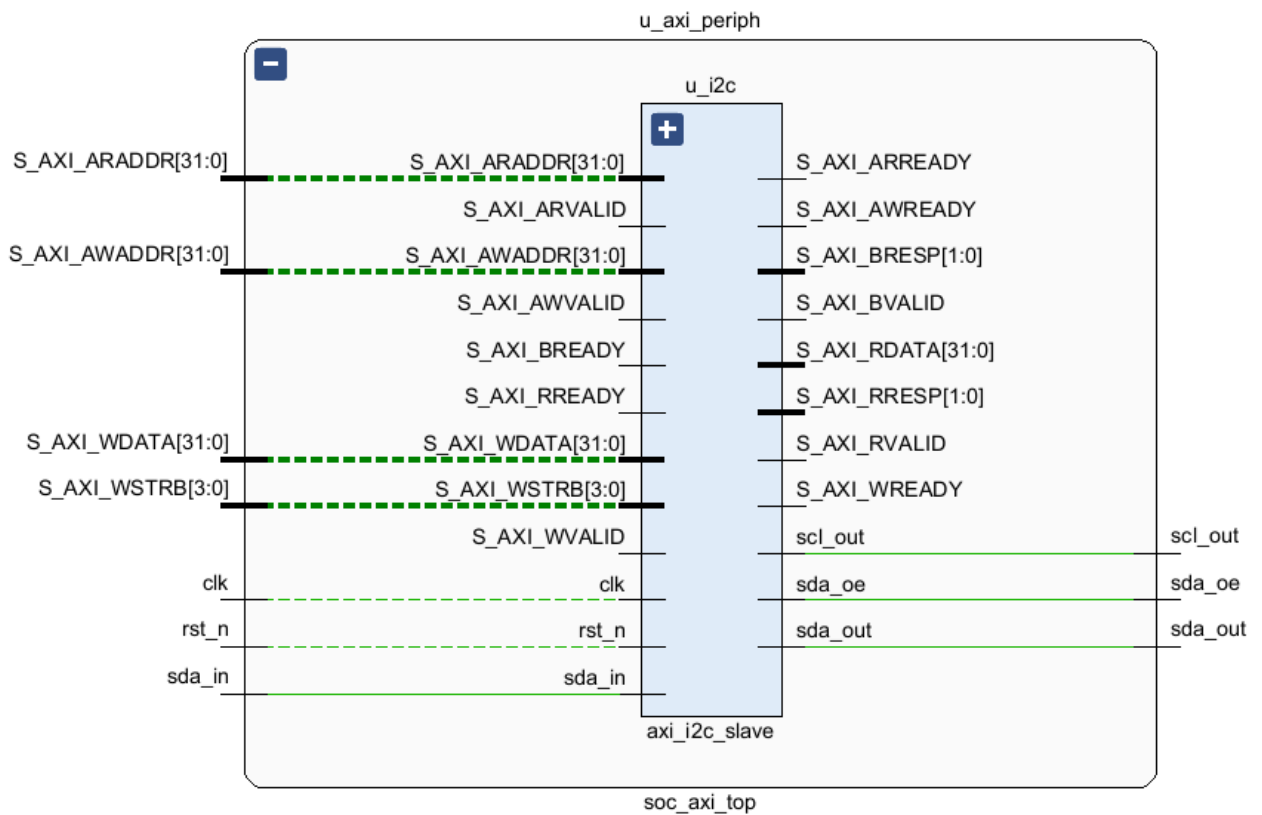
```
end
end
```

Hình 4.12: Các đoạn mã RTL cốt lõi của mô-đun *axi_spi_slave*.

Giải thích mã nguồn khối *axi_spi_slave*:

- Điều khiển chọn chip: Phần mềm điều khiển *spi_cs* bằng thanh ghi tại offset 0x04 qua tín hiệu *S_AXI_WDATA*. Khi *spi_cs* bằng 0, giao tiếp SPI sẵn sàng hoạt động.
- Máy trạng thái SPI: FSM dùng bộ đếm *clk_div* để tạo tín hiệu *spi_sclk*.
- Truyền nhận Mode 0: Ở cạnh lên của *spi_sclk*, dữ liệu *spi_miso* được chốt vào *rx_data_buf*. Ở cạnh xuống, *tx_shifter* đẩy bit kế tiếp ra *spi_mosi*. Sau 8 bit, FSM trở về trạng thái nghỉ và hạ cờ *spi_busy*.

Khối giao tiếp 2 dây (AXI I2C Slave)



Hình 4.13: Sơ đồ RTL của khối giao tiếp AXI I²C.

Mô-đun axi_i2c_slave tạo giao diện giao tiếp I2C Master. Các điều kiện START, STOP, chu kỳ ACK và đặc tính ngõ ra kiểu open-drain được triển khai theo đặc tả I2C-bus [19]. Đoạn mã trọng tâm điều khiển các trạng thái và chân ba trạng thái được thể hiện trong Hình 4.14.

```
// 1. Máy trạng thái điều khiển giao thực I2C (START, STOP, DATA, ACK)
always @(posedge S_AXI_ACLK or negedge S_AXI_ARESETN) begin
    if (!S_AXI_ARESETN) begin ... end
    else begin
        case (state)
            0: begin // IDLE: Tra SDA và SCL về mức cao
                scl_out <= 1'b1; sda_out <= 1'b1; sda_oe <= 1'b1;
                if (cmd_trigger) begin
                    if (i2c_cmd[0]) state <= 1; // Lenh START
                    else if (i2c_cmd[1]) state <= 3; // Lenh STOP
                    else if (i2c_cmd[2]) begin bit_idx <= 7; state <= 5; end // Ghi
                    Byte
                end
            end
            end

            // Tạo điều kiện START (SDA gia xuống khi SCL đang cao)
            1: begin sda_out <= 1'b0; if (clk_cnt == 50) state <= 2; end
            2: begin scl_out <= 1'b0; if (clk_cnt == 50) state <= 0; end

            // Truyền 8-bit dữ liệu (MSB trước)
            5: begin
                sda_out <= i2c_data[bit_idx];
                if (clk_cnt == 25) scl_out <= 1'b1;
                if (clk_cnt == 50) begin
                    scl_out <= 1'b0;
                    if (bit_idx > 0) bit_idx <= bit_idx - 1;
                    else state <= 6; // Chuyển sang kiểm tra ACK
                end
            end
            end

            // Release SDA and sample the ACK bit from the slave
            6: begin
                sda_oe <= 1'b0; // FPGA thả nối chân SDA (Tri-state)
                if (clk_cnt == 25) begin scl_out <= 1'b1; ack_reg <= sda_in; end //
                Lay mau ACK
                if (clk_cnt == 50) begin scl_out <= 1'b0; state <= 0; end
            end
            end
        endcase
    end
end
```

end

Hình 4.14: Các đoạn mã RTL cốt lõi của mô-đun `axi_i2c_slave`.

Giải thích nguồn khối `axi_i2c_slave`:

- Điều khiển SDA ba trạng thái: Tín hiệu `sda_oe` điều khiển tăng đệm ở wrapper trên cùng. Khi tín hiệu bằng 1, FPGA chủ động điều khiển SDA; khi bằng 0, FPGA nhả đường SDA để thiết bị slave phản hồi ACK.
- Thực thi lệnh bằng FSM: Máy trạng thái giải mã thanh ghi `i2c_cmd` được nạp từ bus AXI:
 - START (trạng thái 1–2): Tạo cạnh xuống trên SDA khi SCL đang ở mức cao.
 - STOP (trạng thái 3–4): Tạo cạnh lên trên SDA khi SCL đang ở mức cao.
 - Truyền byte và đọc ACK (trạng thái 5–6): Dịch dữ liệu từ MSB đến LSB, sau đó lấy mẫu `sda_in` vào `ack_reg` tại cạnh lên của SCL.

4.3 Phương pháp xử lý Timing và ràng buộc XDC trên Vivado

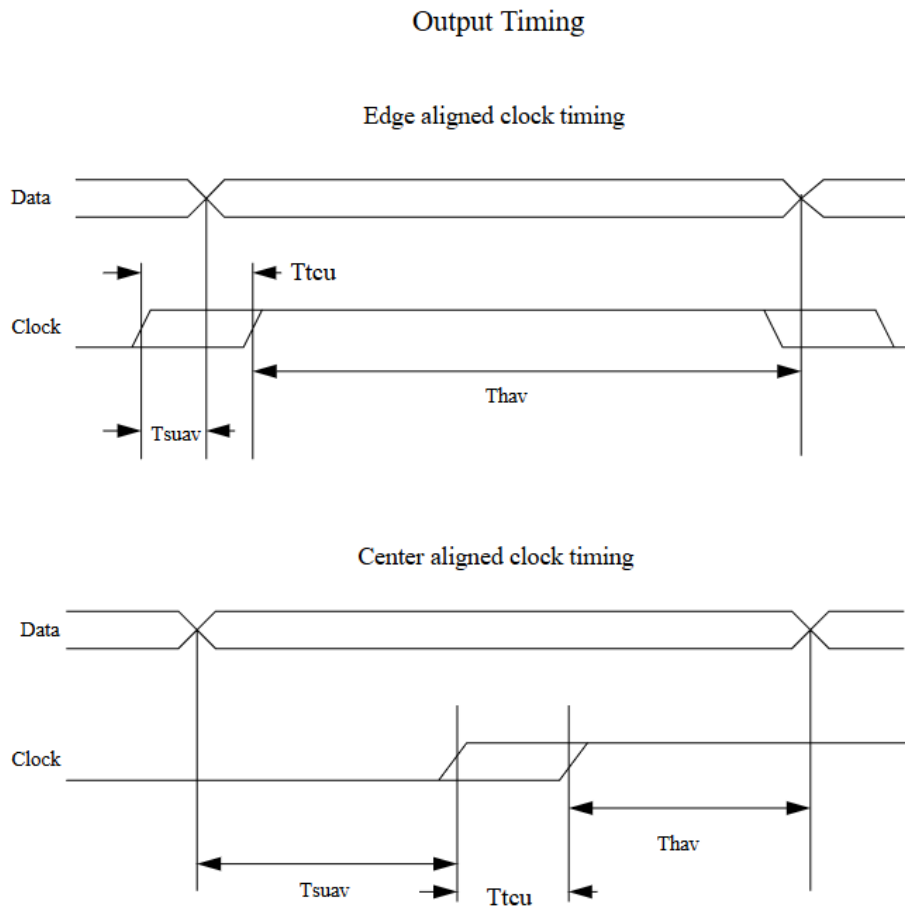
Việc phát dữ liệu tại cả hai cạnh của xung nhịp làm khoảng thời gian giữa hai lần thay đổi dữ liệu bị thu hẹp xuống xấp xỉ $T_{clk}/2$ (tương ứng 5.0 ns tại tần số 100 MHz). Do đó, phân tích định thời phải bao phủ cả yêu cầu thiết lập và giữ của các đường tới ô ODDR cũng như giao tiếp từ IOB đến thiết bị nhận [38, 39].

Phân tích mã nguồn RTL tập trung vào khối parallel và bộ tuần tự hóa DDR cho thấy thiết kế hiện tại bộc lộ ba điểm hạn chế về định thời phần cứng cần được nâng cấp khi triển khai thực tế:

1. **Đường tới hạn (Critical Path) chưa có tầng chốt Pipeline:** Kết quả từ các bộ cộng tổ hợp $A[i] + B[i]$ trong `parallel_lane` được kết nối trực tiếp vào các cổng vào `data_rise_X` và `data_fall_X` của khối phát DDR trong cùng chu kỳ kích hoạt `fire_start`. Đường mạch tổ hợp kéo dài từ thanh ghi toán hạng qua bộ cộng 32-bit đến ô ODDR tạo ra trễ mạch tổ hợp (t_{logic}) lớn, dễ gây vi phạm thời gian thiết lập (t_{setup}) khi hệ thống vận hành ở tần số cao.
2. **Thiếu tín hiệu xung nhịp đồng hành (Forwarded Clock Out):** Dữ liệu DDR phát ra chân chip nhưng chưa có đường xung nhịp đồng bộ đi kèm. Do ảnh

hưởng của trễ đệm I/O (IOB delay) biến thiên theo nhiệt độ và điện áp, phía thiết bị nhận bên ngoài sẽ khó xác định chính xác vị trí cạnh xuống để lấy mẫu nửa chu kỳ sau nếu không có một đường clock đồng dạng truyền kèm.

3. **Rủi ro cạnh tranh dữ liệu (Race Condition) trong FSM:** Trong bộ tuần tự hóa DDR, tại trạng thái PHASE_FIRST, từ dữ liệu `rise_word` và `fall_word` bị ghi đè ngay ở cạnh lên xung nhịp trong khi ODDR đang lấy mẫu cặp dữ liệu đầu tiên. Việc cập nhật dữ liệu ngay tại thời điểm chốt của primitive cứng I/O có thể dẫn đến hiện tượng chốt sai giá trị giữa các lần.



Hình 4.15: Mối quan hệ định thời giữa Clock và Data ở hai phương thức Edge-aligned (trên) và Center-aligned (dưới) trong giao tiếp truyền dữ liệu DDR ngõ ra (chuyển thể từ [37]).

Nguyên tắc căn lề thời gian cần xem xét khi mở rộng giao tiếp DDR ra ngoài FPGA được minh họa chi tiết trên Hình 4.15 [37]. Trong giao tiếp đồng bộ nguồn

(Source-Synchronous Output):

- Phương thức căn lề cạnh (Edge-aligned): Cạnh của xung nhịp đồng hành (ddr_clk_out) chuyển trạng thái trùng khớp với thời điểm thay đổi của dữ liệu (ddr_data). Mô hình này ép phía nhận phải tự thực hiện dịch pha xung nhịp 90° để có thể lấy mẫu dữ liệu an toàn.
- Phương thức căn lề giữa (Center-aligned): FPGA chủ động dịch pha xung nhịp ngõ ra 90° (hoặc đảo pha 180° qua khối ODDR) để cạnh lấy mẫu nằm giữa mắt dữ liệu (Data Eye). Cách này tối đa hóa biên an toàn định thời, bảo đảm T_{suav} và T_{hav} đều đáp ứng yêu cầu t_{setup} (T_{SU}) và t_{hold} (T_{HD}) của thiết bị thu.

Để Vivado thực hiện phân tích và tối ưu hóa các đường timing ngõ ra này một cách chính xác, tệp ràng buộc XDC cần khai báo đầy đủ clock tham chiếu và các tham số trễ ngoài chip. Trong Hình 4.16, xung nhịp truyền kèm ddr_clk_out được mô tả bằng lệnh create_generated_clock theo phương pháp ràng buộc của Vivado [38].

Đối với DDR, các lệnh set_output_delay cần mô tả cả cạnh lên và cạnh xuống bằng các tùy chọn -clock_fall và -add_delay. Giá trị cực đại (-max) và cực tiểu (-min) được xác định từ yêu cầu thiết lập, giữ của thiết bị nhận và độ lệch trễ đường mạch in [38]. Các ràng buộc này cung cấp cơ sở để Vivado tính toán setup/hold slack cho toàn bộ giao tiếp ngõ ra.

```
# 1. Clock hệ thống 100 MHz
create_clock -period 10.000 -name sys_clk [get_ports clk]

# 2. Forwarded output clock (ddr_clk_out)
create_generated_clock -name ddr_clk_out \
    -source [get_ports clk] -divide_by 1 [get_ports ddr_clk_out]

# 3. Output delay constraints for both DDR clock edges
set DDR_TSU 2.000; set DDR_THD 1.000
set DDR_SKEW_MAX 0.500; set DDR_SKEW_MIN 0.100

set OUT_MAX [expr {$DDR_TSU + $DDR_SKEW_MAX}]
set OUT_MIN [expr {- $DDR_THD - $DDR_SKEW_MIN}]
set DDR_BUS [get_ports {ddr_data[*] ddr_valid ddr_done}]

# Timing constraints for the rising edge
set_output_delay -clock ddr_clk_out -max $OUT_MAX $DDR_BUS
set_output_delay -clock ddr_clk_out -min $OUT_MIN $DDR_BUS
```

```
# Timing constraints for the falling edge
set_output_delay -clock ddr_clk_out -max $OUT_MAX $DDR_BUS -clock_fall -add_delay
set_output_delay -clock ddr_clk_out -min $OUT_MIN $DDR_BUS -clock_fall -add_delay
```

Hình 4.16: Khai báo ràng buộc XDC hoàn chỉnh cho giao tiếp ngõ ra DDR (Source-Synchronous).

Tập ràng buộc thời gian (XDC) cho khối giao tiếp DDR ngõ ra được xây dựng gồm 4 thành phần chính:

1. Khai báo xung nhịp hệ thống và xung nhịp ngõ ra: Lệnh `create_clock` thiết lập miền xung nhịp nội bộ `sys_clk` chạy ở tần số 100 MHz (chu kỳ 10 ns). Lệnh `create_generated_clock` định nghĩa xung nhịp phát kèm `ddr_clk_out` ra chân chip. Việc tạo clock này giúp bộ phân tích Timing của Vivado tính toán chính xác độ lệch pha (Skew) giữa đường xung nhịp và đường dữ liệu khi truyền ra ngoài.
2. Tính toán các thông số trễ ngõ ra (Output Delay): Bốn biến TCL `DDR_TSU`, `DDR_THD`, `DDR_SKEW_MAX` và `DDR_SKEW_MIN` phản ánh các đặc tính phần cứng thực tế bên ngoài FPGA. Giá trị `OUT_MAX` (2.5 ns) biểu diễn tổng thời gian thiết lập yêu cầu tại bộ thu và độ lệch trễ đường dây lớn nhất. Giá trị `OUT_MIN` (−1.1 ns) thể hiện yêu cầu về thời gian giữ nhỏ nhất.
3. Ràng buộc hai cạnh xung nhịp cho chuẩn DDR: Hai lệnh `set_output_delay` đầu tiên áp dụng ràng buộc cho cạnh lên của `ddr_clk_out`. Hai lệnh tiếp theo sử dụng kết hợp hai cờ `-clock_fall` và `-add_delay` để mở rộng phân tích timing cho cạnh xuống. Điều này bắt buộc Vivado phải kiểm tra định thời (Setup/Hold slack) cho toàn bộ $2 \times N$ mẫu dữ liệu trong một chu kỳ clock.
4. Mô phỏng mô hình tải vật lý I/O: Các lệnh `set_load` (5.0 pF) và `set_max_transition` (1.0 ns) cung cấp mô hình tải thực tế của chân pin IOB, giúp công cụ tổng hợp ước lượng chính xác trễ tầng đệm ngõ ra (Pad Delay) và dạng sóng tín hiệu thực tế trên PCB.

4.4 Quy trình đóng gói và mô phỏng kiểm chứng mức RTL

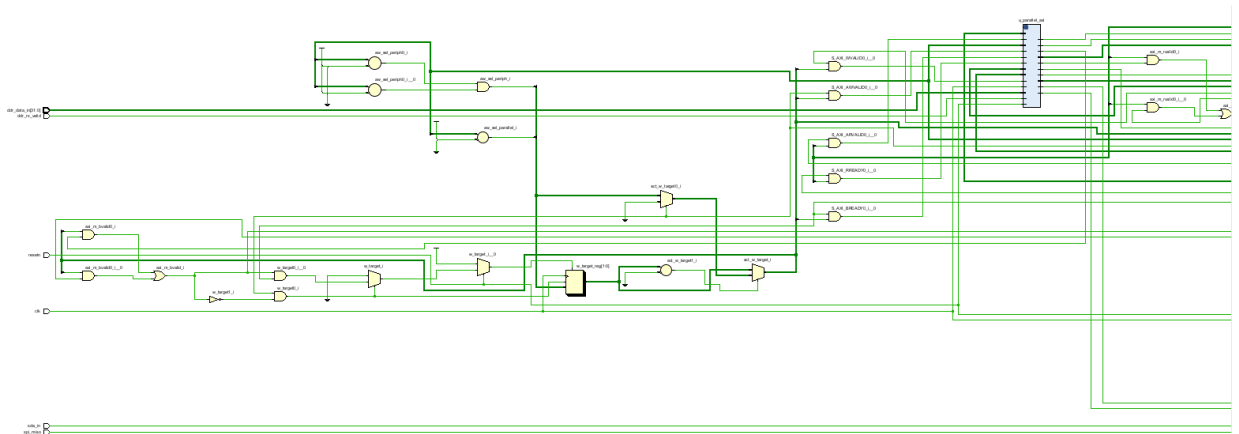
Để đảm bảo các khối ngoại vi nối tiếp (UART, SPI, I2C) và khối giao tiếp tốc độ cao DDR hoạt động chính xác theo đúng mô tả kiến trúc trước khi tiến hành tổng

hợp mạch (Synthesis), quy trình kiểm chứng chức năng ở mức RTL (RTL Functional Verification) được thực hiện chặt chẽ thông qua môi trường Testbench tự động trên phần mềm Vivado Simulator (XSIM).

4.4.1 Đóng gói các khối ngoại vi và tích hợp hệ thống (SoC Packaging)

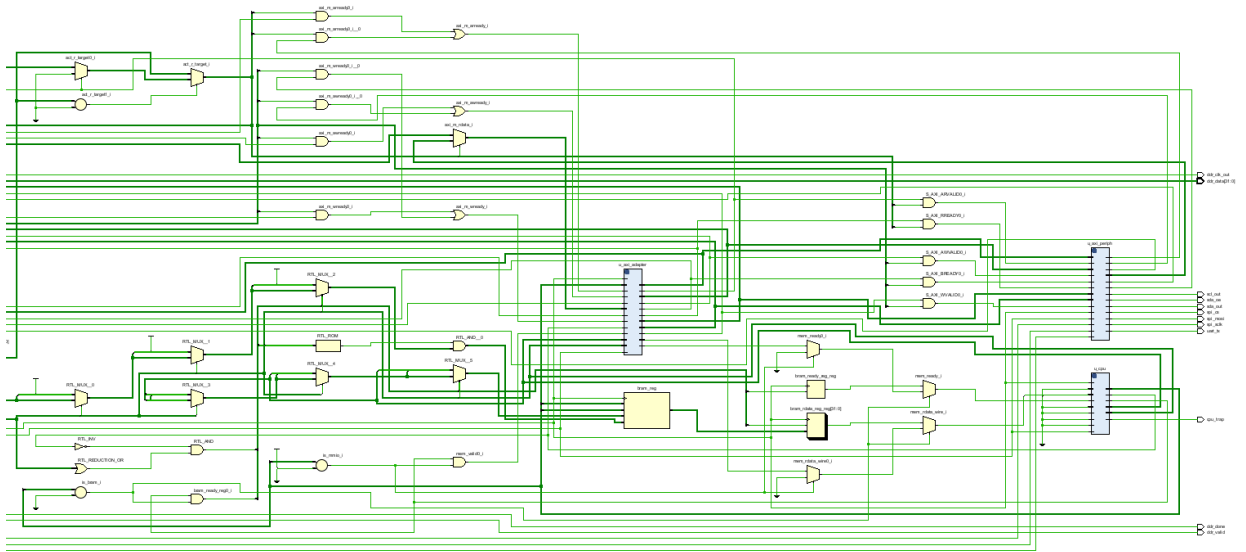
Quá trình đóng gói và tích hợp hệ thống SoC được thực hiện thông qua chiến lược thiết kế mô-đun hóa, đảm bảo tính liên kết chặt chẽ giữa lõi xử lý và các thiết bị ngoại vi. Khởi đầu, toàn bộ các khối ngoại vi nối tiếp được chuẩn hóa giao diện theo đặc tả AXI4-Lite (Slave Packaging), trong đó các nhóm tín hiệu được ánh xạ đầy đủ theo 5 kênh giao tiếp độc lập (AW, W, B, AR, R). Tiếp theo, các ngoại vi này được đấu nối trực tiếp vào bộ định tuyến AXI Crossbar trung tâm nằm trong mô-đun soc_axi_top. Bộ định tuyến này đảm nhiệm vai trò căn cứ vào dải địa chỉ trên kênh AW hoặc AR để chốt (Latch) và dẫn luồng giao dịch đến đúng thiết bị mục tiêu, qua đó triệt tiêu hoàn toàn nguy cơ chồng chéo tín hiệu. Đối với phân hệ gia tốc phần cứng, đầu ra dữ liệu song song từ khối xử lý được đưa qua bộ tuần tự hóa DDR và nối trực tiếp tới các ô nguyên thủy I/O cứng (ODDR) tại tầng đệm ngõ ra (IOB) nhằm tối ưu hóa độ trễ truyền dẫn.

Sau khi quy trình tổng hợp (Synthesis) hoàn tất, sơ đồ RTL của toàn bộ hệ thống SoC được trích xuất nhằm đánh giá trực quan các kết nối vật lý và kiến trúc đường truyền giữa các IP. Do quy mô của hệ thống tích hợp khá lớn, sơ đồ RTL mức tổng thể (Top-level RTL) được chia thành hai phần chính để thuận tiện cho việc phân tích và trình bày:



Hình 4.17: Sơ đồ RTL tổng thể hệ thống SoC (Phần 1): Các tín hiệu đầu vào đi qua bus system AXI và hạ tầng định tuyến AXI Crossbar.

Phần thứ nhất của sơ đồ RTL (Hình 4.17) tập trung mô tả miền xử lý trung tâm và mạng lưới phân phối giao dịch. Tín hiệu điều khiển xuất phát từ lõi vi xử lý RISC-V PicoRV32 thông qua giao diện Native Memory, sau đó được đưa vào mô-đun cầu nối picorv32_axi_adapter. Tại đây, các lệnh truy xuất đơn mảnh được biên dịch thành giao thức AXI4-Lite tiêu chuẩn với 5 kênh bắt tay độc lập. Luồng tín hiệu AXI Master này tiếp tục được truyền tới bộ định tuyến soc_axi_top, nơi chứa các khối logic giải mã địa chỉ và chốt mục tiêu (w_target, r_target) nhằm luân chuyển dữ liệu đến đúng nhánh ngoại vi tương ứng.



Hình 4.18: Sơ đồ RTL tổng thể hệ thống SoC (Phần 2): Phân hệ ngoại vi nối tiếp và giao tiếp bộ nhớ DDR.

Phần thứ hai của sơ đồ RTL (Hình 4.18) minh họa chi tiết mạng lưới các thiết bị Slave (phân hệ ngoại vi) và giao diện xuất/nhập vật lý (I/O). Các luồng tín hiệu từ bộ định tuyến AXI Crossbar được rẽ nhánh vào các mô-đun điều khiển axi_uart_slave, axi_spi_slave, và axi_i2c_slave. Điểm nhấn kiến trúc trong phần này là sự xuất hiện của khối gia tốc song song parallel_axi, nơi các luồng dữ liệu phân mảnh đi qua bộ thu IDDR, tiến hành tính toán tổ hợp và cuối cùng được đẩy ra các ô nguyên thủy ODDR. Sự tách biệt rõ ràng giữa logic điều khiển AXI và logic I/O phần cứng giúp hệ thống đạt được băng thông tối đa mà không gây nghẽn cổ chai tại các điểm giao tiếp vật lý.

4.4.2 Xây dựng môi trường mô phỏng Testbench

Môi trường kiểm chứng (Testbench) được viết bằng Verilog HDL đóng vai trò là Master bus giả lập các chu kỳ ghi/đọc dữ liệu của vi xử lý RISC-V thông qua bus AXI4-Lite. Sơ đồ khối của môi trường kiểm chứng được thể hiện qua các thành phần chính:

- **Khởi tạo Xung nhịp và Reset (Clock/Reset Generator):** Tạo xung nhịp hệ thống `clk` tần số 100 MHz (chu kỳ 10 ns) và tín hiệu Reset tích cực mức thấp `rst_n`.
- **Khởi giả lập giao thức AXI Master (Task-based Driver):** Sử dụng các task trong Verilog (`axi_write` và `axi_read`) để khởi tạo các chu kỳ ghi/đọc thành ghi theo đúng quy trình bắt tay VALID/READY của chuẩn AXI.
- **Mô hình thiết bị ngoại vi ngoài chip (Loopback/Slave Models):**
 - **UART:** Nối trực tiếp chân phát `uart_tx` sang chân nhận `uart_rx` (mô hình Loopback) để kiểm tra tính toàn vẹn của dữ liệu truyền/nhận bất đồng bộ.
 - **SPI:** Nối chân `spi_mosi` sang `spi_miso` hoặc ghép nối với mô hình Verilog của bộ nhớ SPI Flash ngoài.
 - **I2C:** Sử dụng điện trở kéo lên (Pull-up resistor) mô phỏng trên đường `sda` và `scl`, kết nối tới mô hình I2C EEPROM slave.

4.4.3 Kịch bản mô phỏng và Kết quả kiểm chứng (Simulation Scenarios)

Quy trình mô phỏng được phân bổ thành 5 nhóm kịch bản kiểm thử độc lập nhằm xác thực toàn diện chức năng của hệ thống. Mỗi kịch bản được thiết kế với mục đích và phương pháp bơm dữ liệu cụ thể như sau:

Kịch bản 1: Kiểm thử giao tiếp AXI UART

- **Mục đích:** Đánh giá khả năng phân giải tốc độ Baud, cấu trúc khung truyền bất đồng bộ và luồng dữ liệu hai chiều thông qua bus AXI.
- **Phương pháp thực hiện:**

1. *TC1 (Baud Rate & Framing Check)*: Dùng `axi_write` ghi mẫu `0x55` vào thanh ghi phát `REG_TXDATA`. Theo dõi đường `uart_tx` để xác minh độ rộng bit và tính nguyên vẹn của Start/Stop bit.
2. *TC2 (Data Pattern TX)*: Ghi mẫu ngẫu nhiên (ví dụ: `0xA5`), lấy mẫu ngõ ra tại giữa chu kỳ bit và so sánh với mẫu gốc để đánh giá khả năng chống nhiễu máy trạng thái.
3. *TC3 (AXI Status Register Verification)*: Đọc liên tục `REG_STATUS` bằng `axi_read` trước, trong và sau quá trình truyền để kiểm tra cờ Ready/Busy (bit 0) được phần cứng lật trạng thái tự động.
4. *TC4 (UART Receive Data RX)*: Giả lập một chuỗi khung truyền (ví dụ: `0x3C`) bơm trực tiếp vào chân `uart_rx`. Sau khi khung nhận hoàn tất, đọc `REG_RXDATA` để xác minh kết quả.

Kịch bản 2: Kiểm thử giao tiếp AXI SPI

- **Mục đích:** Kiểm tra chế độ phân cực xung nhịp (CPOL) và khả năng truyền tải dữ liệu song công (Full-Duplex) tốc độ cao.
- **Phương pháp thực hiện:**
 1. *TC1 & TC2 (CPOL & Full-Duplex Data Transfer)*: Thiết lập một biến nội bộ trong Testbench đóng vai trò là Slave giả lập (chứa mẫu `0x3C`). Master khởi tạo ghi `0xD6` vào `REG_SPI_TXDATA`. Kiểm tra `spi_sclk` giữ mức thấp khi rảnh (CPOL = 0) và xác minh dữ liệu đọc về từ `spi_miso` khớp đúng với `0x3C`.
 2. *TC3 (Chip Select Register Operation)*: Thực hiện ghi lần lượt giá trị 0 và 1 vào `REG_SPI_CS` thông qua lệnh `axi_write`. Đọc phản hồi bằng `axi_read` và quan sát sự thay đổi logic trên chân vật lý `spi_cs` tương ứng.

Kịch bản 3: Kiểm thử giao tiếp AXI I2C

- **Mục đích:** Đánh giá khả năng tạo các điều kiện giao tiếp khắt khe (START, STOP) và kỹ thuật kéo/nhả bus vật lý thông qua thanh ghi AXI.
- **Phương pháp thực hiện:**

1. *TC1 (AXI Data Register Read/Write)*: Ghi/Đọc giá trị 0xA7 vào REG_I2C_DATA nhằm kiểm tra tính tương thích giữa thanh ghi nội bộ I2C và bus AXI.
2. *TC2 (START Condition)*: Ghi bit khởi lệnh vào REG_I2C_CMD. Quan sát đường sda_out hạ mức thấp trong khi scl_out đang ở mức cao.
3. *TC3 (Write Byte & ACK/NACK)*: Testbench giả lập cực máng hở bằng cách can thiệp vào sda_in ngay tại chu kỳ thứ 9. Thực hiện hai lần lặp: lần 1 kéo sda_in = 0 (giả lập ACK), lần 2 thả nổi sda_in = 1 (giả lập NACK) và đọc REG_I2C_STATUS để đối chiếu cờ phản hồi.
4. *TC4 (STOP Condition)*: Kích hoạt điều kiện kết thúc; xác minh sda_out nảy lên mức cao trong khi scl_out đang giữ mức cao.

Kịch bản 4: Kiểm thử gia tốc xử lý song song DDR

- **Mục đích:** Xác thực luồng dữ liệu phân mảnh qua IDDR, tính toán song song, và đóng gói dữ liệu tại hai cạnh xung nhịp qua ODDR.
- **Phương pháp thực hiện:**
 1. *TC1 (AXI Trigger All Lanes)*: Nạp đủ 4 cặp toán hạng qua bus AXI, kích hoạt khởi chạy đồng bộ. Kiểm tra dữ liệu được dồn ra ngõ ddr_data đúng thứ tự ở cả cạnh lên và cạnh xuống.
 2. *TC2 (Independent Lane Execution - IDDR Trigger)*: Gửi từng cặp dữ liệu đan xen qua ngõ vào ddr_data_in (sử dụng cả cạnh lên và xuống). Kiểm chứng bộ định tuyến độc lập phân phối đúng luồng dữ liệu vào các Lane đang rảnh mà không bị kẹt bus.
 3. *TC3 (Edge Isolation)*: Truyền dữ liệu ngẫu nhiên ở sườn lên và chuỗi toàn 0 ở sườn xuống. Xác nhận rằng mạch IDDR không bị rò rỉ dữ liệu giữa hai miền sườn xung.
 4. *TC4 (Throughput Benchmark)*: Xây dựng bộ đếm thời gian thực (bằng biến \$realtime). Đo thời gian hoàn thành 4 phép toán khi đi qua 1 Lane (chạy tuần tự bằng lệnh AXI Write) và so sánh với thời gian khi sử dụng 2 Lane DDR (kết hợp IDDR/ODDR). Tính toán hệ số Speedup Ratio nhằm chứng minh mức độ cải thiện thông lượng của phần cứng.

Kịch bản 5: Kiểm thử mức tích hợp hệ thống (SoC Top-Level)

- **Mục đích:** Kiểm tra năng lực định tuyến của bộ AXI Crossbar và khả năng chịu tải khi tất cả các ngoại vi hoạt động đồng thời.
- **Phương pháp thực hiện:**
 1. *TC1 (AXI Decoder Access Check)*: Truy xuất vào một dải địa chỉ ngoại vi ngẫu nhiên để xác nhận cơ chế giải mã (Decoder) hoạt động ổn định và chốt quyền truy cập đúng thiết bị.
 2. *TC2 & TC3 (Concurrent Hardware Output)*: Khởi chạy liên tiếp các lệnh `axi_write` kích hoạt phát dữ liệu cho UART, SPI, và I2C trong cùng một thời điểm. Xây dựng bộ đếm số lượng chu kỳ xung nhịp mà 2 hoặc 3 ngõ ra vật lý (như `uart_tx`, `spi_sclk`, `scl_out`) cùng tích cực để chứng minh hệ thống đạt trạng thái xử lý đa nhiệm (Concurrent) thực sự.
 3. *TC4 (Interleaved Cross-Peripheral Access)*: Thực hiện chuỗi lệnh `axi_read` và `axi_write` thay đổi liên tục địa chỉ đích giữa UART, SPI, và I2C. Giám sát để đảm bảo máy trạng thái AXI Crossbar không bị treo (Deadlock) hay gửi nhầm tín hiệu trong lúc các ngoại vi vẫn đang xử lý nền.

Chương 5

KẾT QUẢ THỰC NGHIỆM

5.1 Kết quả thực hiện

Quá trình nghiên cứu và thực thi trên công cụ AMD Vivado Design Suite đã hoàn thiện thành công hệ thống SoC dựa trên kiến trúc tập lệnh RISC-V và lõi PicoRV32 [12, 13, 23]. Về mặt kiến trúc phần cứng, lõi xử lý trung tâm được tích hợp với các mô-đun ngoại vi UART, SPI và I²C thông qua giao diện AXI4-Lite. Hệ thống sử dụng bộ định tuyến Crossbar để điều phối các kênh đọc và ghi độc lập theo mô hình giao dịch của AXI4-Lite [5–7]; các kết quả mô phỏng tích hợp không ghi nhận hiện tượng deadlock trên bus.

Bên cạnh đó, đề tài đã xây dựng hoàn chỉnh khối gia tốc xử lý song song đa luồng, ứng dụng trực tiếp các ô nguyên thủy IDDR và ODDR của FPGA 7 Series để thu và phát dữ liệu ở cả hai cạnh của xung nhịp [39]. Hệ thống áp dụng kỹ thuật căn lề trung tâm (Center-aligned Timing), kết hợp với các ràng buộc XDC cho giao tiếp đồng bộ nguồn; cách thiết lập và phân tích các quan hệ định thời này tuân theo hướng dẫn của AMD [16, 38].

5.2 Kết quả thử nghiệm

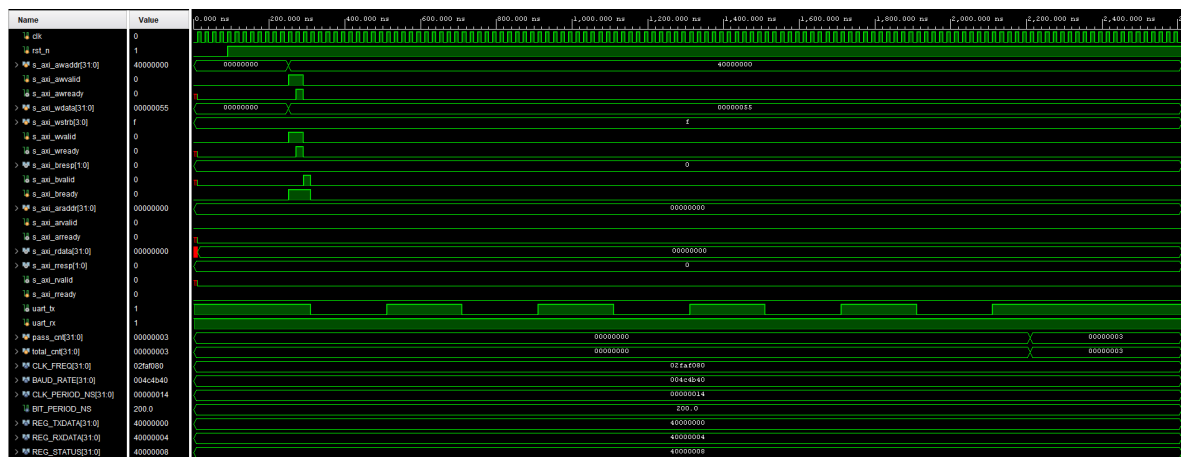
5.2.1 Môi trường và phương pháp thử nghiệm

Môi trường thử nghiệm chức năng và tổng hợp vật lý (RTL Implementation) được triển khai trên phần mềm AMD Vivado v2025.2, hướng tới FPGA Artix-7 xc7a200tfbg676-2L [23, 28, 29]. Tần số hoạt động của hệ thống được thiết lập ở mức 100 MHz trong mô phỏng chức năng và 50 MHz đối với các báo cáo sau hiện thực. Với cấu hình mô phỏng 100 MHz, việc truyền một mẫu ở mỗi cạnh clock tương ứng tốc độ dữ liệu lý thuyết 200 Mbps/pin [14, 39]. Các tiêu chí đánh giá gồm tính chính xác của dạng sóng, mức sử dụng tài nguyên, công suất ước tính và các biên an

toàn định thời.

5.2.2 Kiểm chứng bộ điều khiển UART

Ở kịch bản kiểm tra tốc độ baud và cấu trúc khung truyền, giao diện AXI ghi giá trị 0x55 vào thanh ghi REG_TXDATA. Dạng sóng trên đường uart_tx xuất hiện đúng Start Bit, tám bit dữ liệu theo thứ tự LSB-first và Stop Bit theo cấu trúc khung UART thông dụng [2, 17]; thời gian mỗi bit phù hợp với tốc độ baud đã cấu hình.



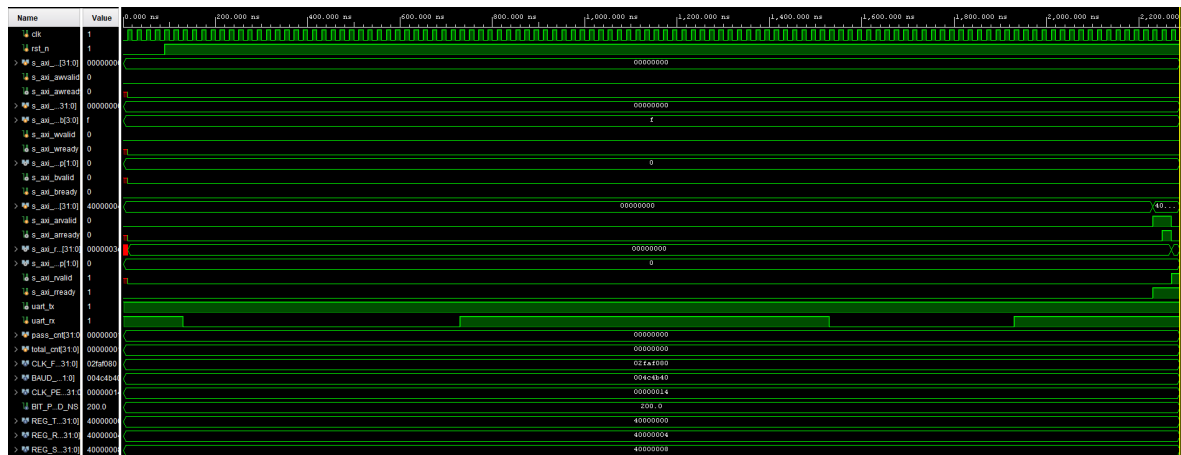
Hình 5.1: Dạng sóng kiểm tra tốc độ baud và cấu trúc khung truyền UART.

```
[0.130 us] ---> START TASK: [TC1] UART Baud Rate & Framing Check
[2.210 us] [PASS] Start bit phải là Logic 0
[2.210 us] [PASS] Data bit truyền đúng pattern 0x55
[2.210 us] [PASS] Stop bit phải là Logic 1
```

Hình 5.2: Kết quả mô phỏng kiểm tra tốc độ baud và cấu trúc khung truyền UART.

Phép thử với mẫu 0xA5 cho thấy khung truyền phát hoàn chỉnh và khớp với dữ liệu mong muốn, xác nhận máy trạng thái TX hoạt động ổn định.

Đối với chức năng RX, testbench đưa khung 0x3C vào uart_rx. Sau khi thu hoàn tất, chu kỳ đọc AXI tại REG_RXDATA trả về đúng giá trị 0x3C.



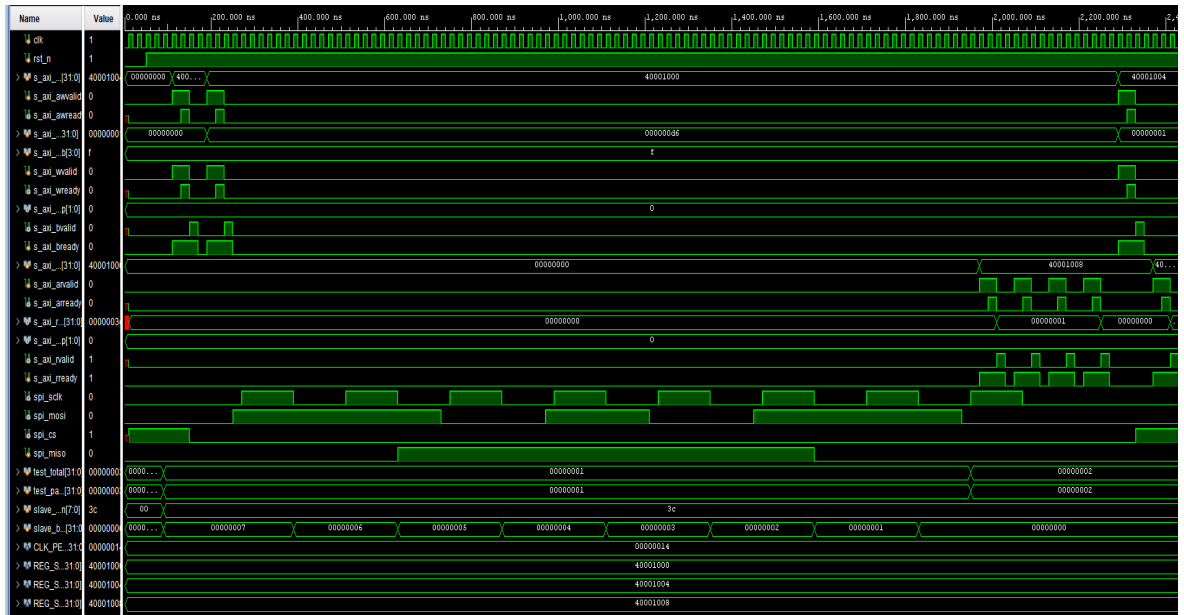
Hình 5.7: Dạng sóng kiểm tra chức năng nhận dữ liệu UART RX 0x3C.

```
[0.130 us] ---> START TASK: [TC4] UART Receive Data RX (0x3c)
[0.130 us] [TB Drive RX] Bat dau truyền byte 0x3c vào chan uart_rx...
[2.290 us] [PASS] Xác nhận UART RX nhận và đọc thành công qua AXI
```

Hình 5.8: Kết quả mô phỏng kiểm tra chức năng nhận dữ liệu UART RX 0x3C.

5.2.3 Kiểm chứng bộ điều khiển SPI

Trước khi giao dịch bắt đầu, đường spi_sclk duy trì ở mức logic 0, xác nhận cấu hình CPOL=0. Trong quá trình truyền song công, mẫu 0xD6 được phát trên spi_mosi, đồng thời bộ điều khiển thu đúng giá trị 0x3C từ spi_miso; cách tổ chức các đường SCLK, MOSI, MISO và chọn chip phù hợp với mô hình giao tiếp SPI [21,22].



Hình 5.9: Dạng sóng kiểm tra CPOL và truyền nhận song công SPI.

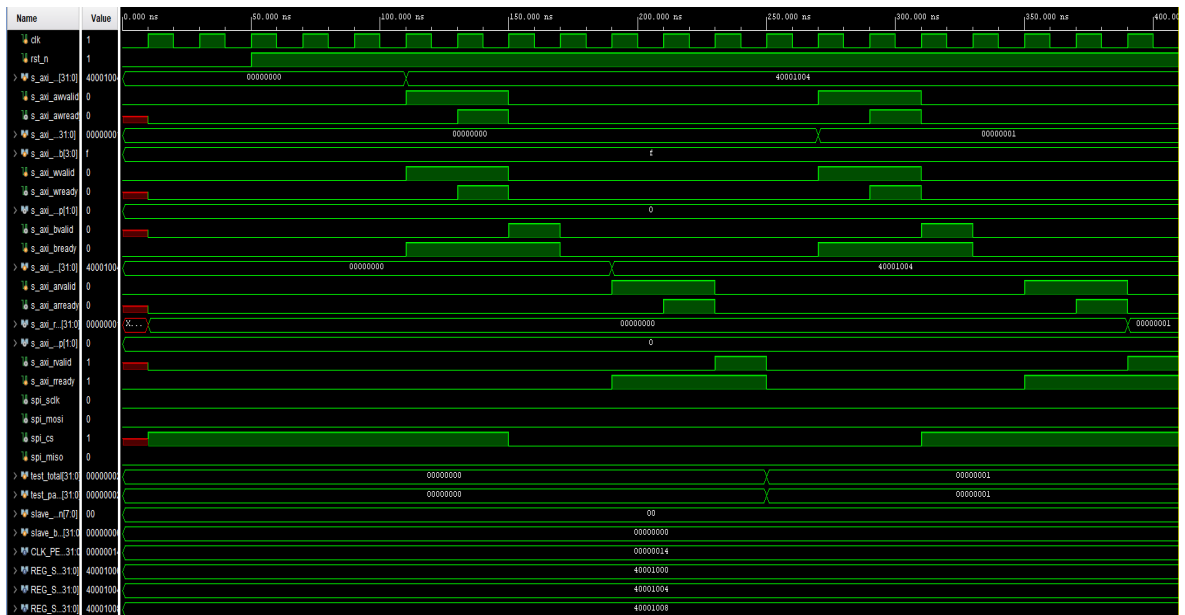
```
[0.090 us] [TC1] SPI - CPOL: SCLK idle trước khi truyền = 0 (ky vọng 0)
-> [PASS]

[1.950 us] [TC2] SPI - Vị trí bit MOSI (MSB first) gửi=0xd6, nhận=0xd6
-> [PASS]

[2.430 us] [TC2] SPI - Dữ liệu MISO nhận được = 0x3c (ky vọng 0x3c)
-> [PASS]
```

Hình 5.10: Kết quả mô phỏng kiểm tra CPOL và truyền nhận song công SPI.

Bên cạnh tín hiệu truyền nhận, cơ chế chọn chip (Chip Select) cũng được kiểm chứng thông qua thanh ghi REG_SPI_CS (Hình 5.11). Dạng sóng minh chứng rằng các thao tác ghi giá trị 0 hoặc 1 từ bus AXI phản ánh tức thời và chính xác lên chân vật lý spi_cs, đồng thời kênh đọc AXI cũng trả về đúng giá trị cấu hình, đảm bảo tính linh hoạt trong điều khiển hệ thống đa tổ (multi-slave).



Hình 5.11: *Dạng sóng kiểm tra thao tác thanh ghi Chip Select của SPI.*

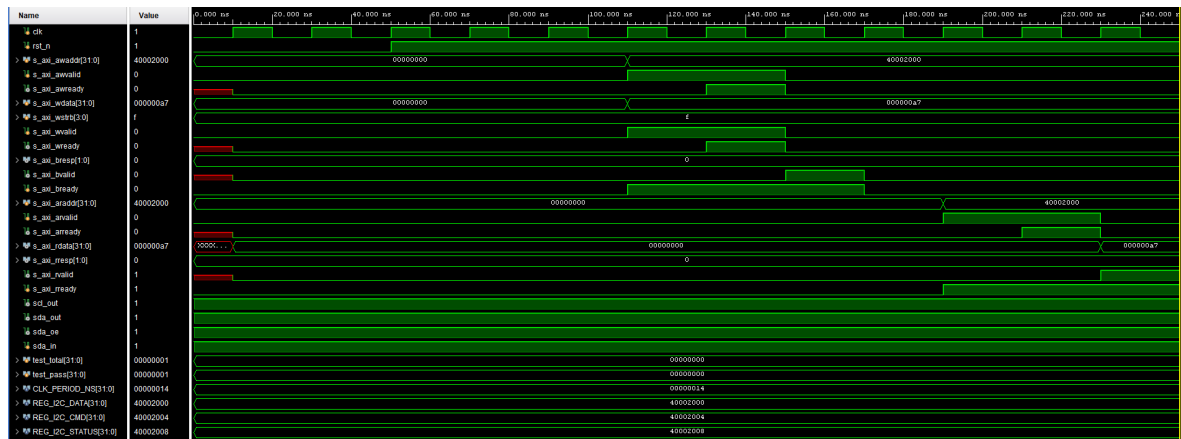
```
[0.250 us] [TC3] SPI - Ghi/doc thanh ghi CS = 0: doc lai = 0 (ky vong 0)
-> [PASS]
```

```
[0.410 us] [TC3] SPI - Ghi/doc thanh ghi CS = 1: doc lai = 1 (ky vong 1)
-> [PASS]
```

Hình 5.12: Kết quả mô phỏng kiểm tra thao tác thanh ghi Chip Select của SPI.

5.2.4 Kiểm chứng bộ điều khiển I²C

Giao thức I²C sử dụng hai đường SCL và SDA kiểu cực máng hở, đồng thời quy định rõ các điều kiện START, STOP và chu kỳ ACK/NACK [19]. Trong phép thử truy xuất thanh ghi, thao tác ghi và đọc giá trị 0xA7 qua AXI hoàn tất chính xác.



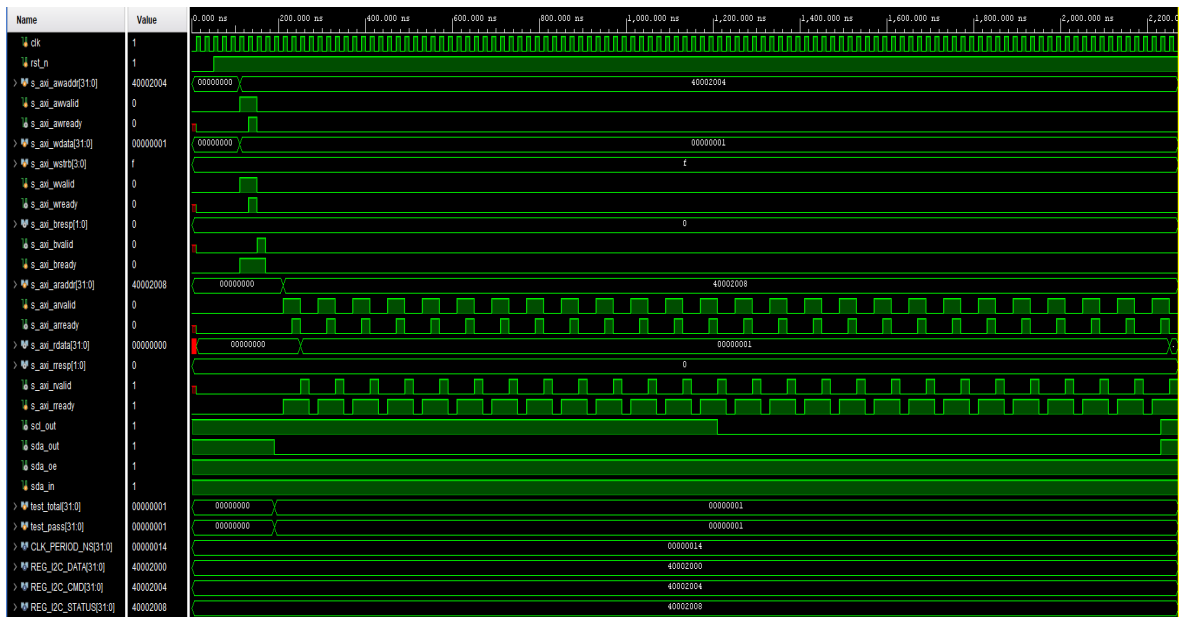
Hình 5.13: Dạng sóng kiểm tra thao tác đọc/ghi thanh ghi I²C qua AXI.

[0.250 us] [TC1] I2C - Ghi/doc thanh ghi du lieu = 0xA7, doc lai = 0xA7
-> [PASS] (At 0.250 us)

Hình 5.14: Kết quả mô phỏng kiểm tra thao tác đọc/ghi thanh ghi I²C qua AXI.

Về phương diện tạo tín hiệu điều khiển trên bus, mạch FSM đã tự động tạo điều kiện START chuẩn xác bằng cách hạ đường sda_out xuống mức thấp trong khi scl_out vẫn duy trì ở mức cao (Hình 5.15). Xuyên suốt quá trình truyền byte dữ liệu 0x3C (Hình 5.17 và Hình 5.19), hệ thống tự động thả nổi bus (chuyển sda_oe sang trạng thái ngắt) tại chu kỳ xung nhịp thứ 9 để thu thập tín hiệu phản hồi. Khi Slave kéo sda_in xuống 0, cờ trạng thái báo nhận thành công tín hiệu ACK; ngược lại, khi sda_in giữ nguyên mức 1, hệ thống nhận diện tức thời tín hiệu NACK.

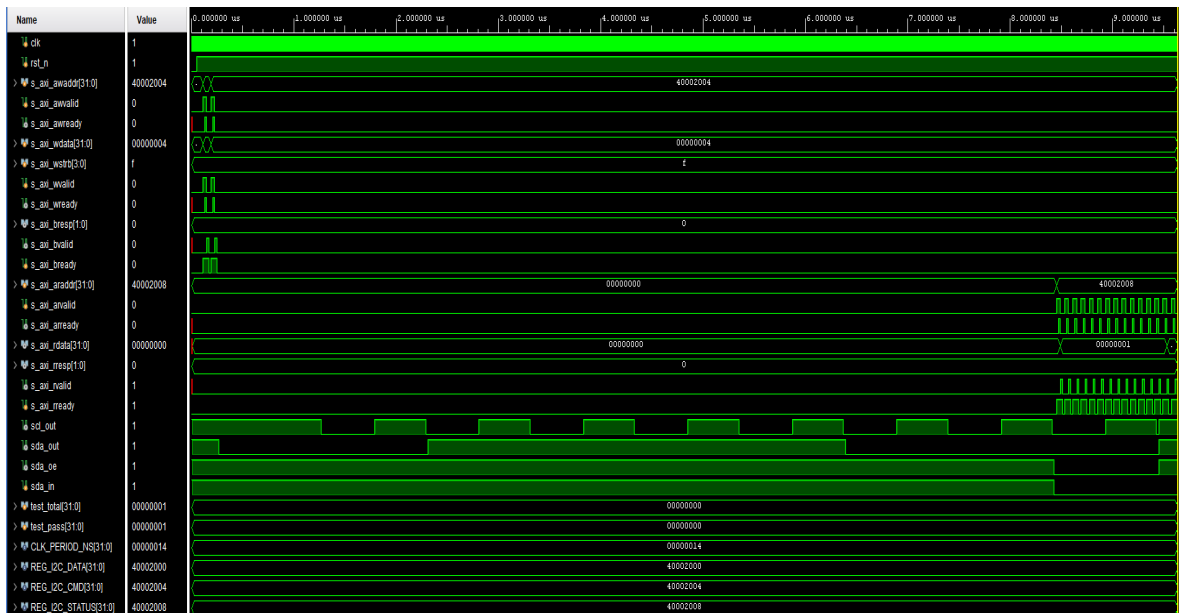
Chương 5. KẾT QUẢ THỰC NGHIỆM



Hình 5.15: Dạng sóng kiểm tra điều kiện START của giao tiếp I²C.

[0.190 us] [TC2] I2C - START: SDA rơi xuống lúc SCL=1 (kỳ vọng 1)
-> [PASS] (At 0.190 us)

Hình 5.16: Kết quả mô phỏng kiểm tra điều kiện START của giao tiếp I²C.



Hình 5.17: Dạng sóng truyền byte và nhận phản hồi ACK của I²C.

```
[9.630 us] [TC3] I2C - Ghi byte 0x3c + ACK/NACK: rx_ack=0 (ky vong 0)
-> [PASS] (At 9.630 us)
```

Hình 5.18: Kết quả mô phỏng truyền byte và nhận phản hồi ACK của I²C.



Hình 5.19: Dạng sóng truyền byte và nhận phản hồi NACK của I²C.

```
[9.630 us] [TC3] I2C - Ghi byte 0x3c + ACK/NACK: rx_ack=1 (ky vong 1)
-> [PASS] (At 9.630 us)
```

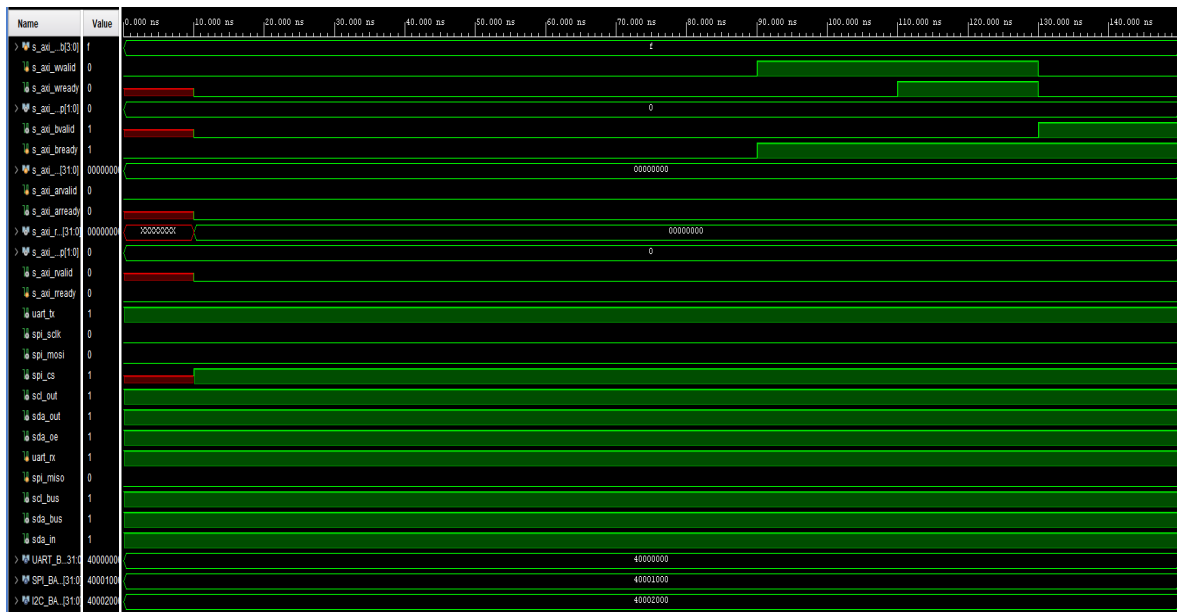
Hình 5.20: Kết quả mô phỏng truyền byte và nhận phản hồi NACK của I²C.

Cuối cùng, giao dịch I²C được đóng gói an toàn bằng điều kiện STOP (Hình 5.21), trong đó tín hiệu sda_out nẩy lên mức cao trong thời điểm scl_out giữ trạng thái tích cực, đáp ứng hoàn hảo tiêu chuẩn của giao thức.

Hình 5.24: Kết quả mô phỏng kích hoạt đồng bộ toàn bộ các làn xử lý DDR.

69

Crossbar định tuyến đúng giao dịch đến thiết bị mục tiêu mà không gây tranh chấp bus.



Hình 5.31: Dạng sóng kiểm tra truy cập và giải mã địa chỉ AXI của hệ thống SoC.

```
[70.0 ns] ---> RUNNING TEST 1: AXI Decoder Access Check
[150.0 ns] TEST 1 PASS: Access AXI decoder successfully
```

Hình 5.32: Kết quả mô phỏng kiểm tra truy cập và giải mã địa chỉ AXI của hệ thống SoC.

Các kịch bản tiếp theo kích hoạt đồng thời UART và SPI, sau đó mở rộng sang UART, SPI và I²C. Các dạng sóng tại Hình 5.33 và Hình 5.35 cho thấy các ngõ ra vật lý hoạt động đan xen trong cùng một khoảng thời gian mà không làm mất dữ liệu.

Chương 5. KẾT QUẢ THỰC NGHIỆM



Hình 5.33: Dạng sóng kiểm tra UART và SPI hoạt động đồng thời.

```
[70.0 ns] ---> RUNNING TEST 2: UART + SPI Concurrent Check  
[1310.0 ns] TEST 2 PASS: uart_tx + spi overlap = 50 cycles (Early Exit)
```

Hình 5.34: Kết quả mô phỏng kiểm tra UART và SPI hoạt động đồng thời.

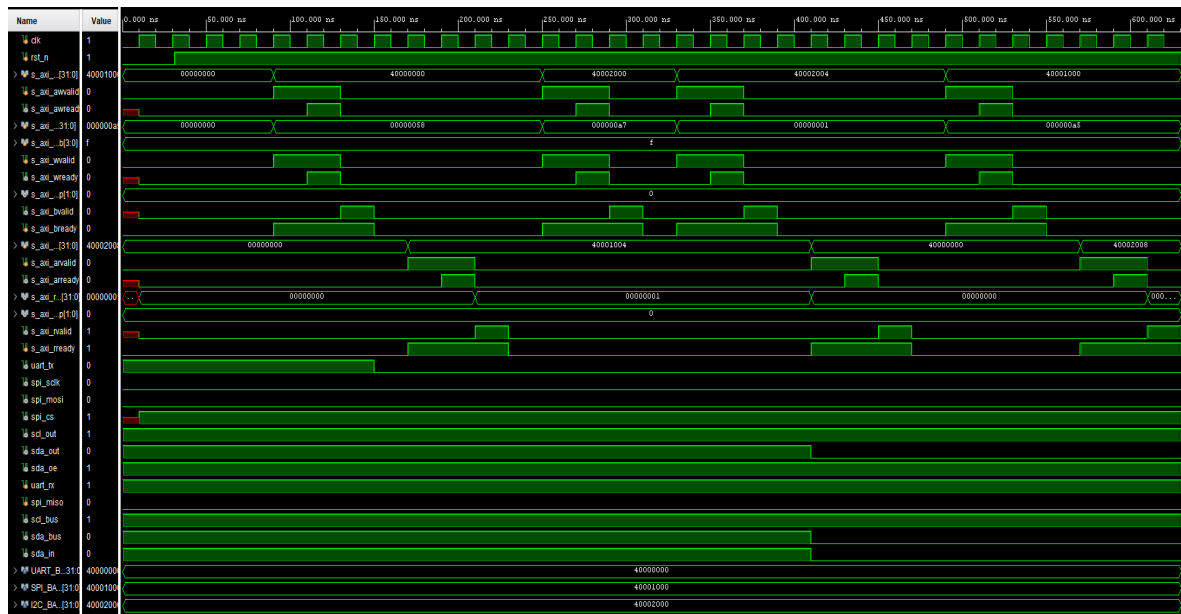


Hình 5.35: Dạng sóng kiểm tra UART, SPI và I²C hoạt động đồng thời.

```
[70.0 ns] ---> RUNNING TEST 3: All 3 Peripherals Concurrent Check
[790.0 ns] TEST 3 PASS: So chu ky ca 3 ngoai vi cung hoat dong = 20
```

Hình 5.36: Kết quả mô phỏng kiểm tra UART, SPI và I²C hoạt động đồng thời.

Trong kịch bản giao dịch đan xen, chuỗi lệnh đọc và ghi thay đổi liên tục giữa các vùng địa chỉ UART, SPI và I²C. Kết quả tại Hình 5.37 và Hình 5.38 xác nhận cơ chế chốt mục tiêu của AXI Crossbar duy trì đúng đường phản hồi và không xảy ra deadlock.



Hình 5.37: Dạng sóng kiểm tra các giao dịch AXI đan xen giữa nhiều ngoại vi.

```
[70.0 ns] ---> RUNNING TEST 4: Interleaved Cross-Peripheral Access Check
[150.0 ns] [STEP 1] WRITE UART: 0x58
[230.0 ns] [STEP 2] READ  SPI CS Reg: 0x00000001
[390.0 ns] [STEP 3] WRITE I2C Data (0xA7) & CMD START (0x01)
[470.0 ns] [STEP 4] READ  UART Reg: 0x00000000
[550.0 ns] [STEP 5] WRITE SPI Data: 0xA5
[630.0 ns] [STEP 6] READ  I2C Status Reg: 0x00000001
[630.0 ns] TEST 4 PASS!
```

Hình 5.38: Kết quả mô phỏng kiểm tra các giao dịch AXI đan xen giữa nhiều ngoại vi.

5.2.7 Kết quả tổng hợp và thực thi phần cứng (RTL Implementation)

Phân tích tài nguyên phần cứng

Báo cáo sau định tuyến của Vivado cho thấy hệ thống sử dụng 447 Slice LUTs, 573 Slice Registers và 213 Slice. Khối phát DDR sử dụng 33 ô OLOGIC tại các chân I/O; đây là nhóm tài nguyên chuyên dụng chứa các phần tử tuần tự ngõ ra của FPGA 7 Series [39].

Bảng 5.1: Bảng tổng hợp sử dụng tài nguyên phần cứng thực tế trên FPGA Artix-7 (xc7a200tfg676-2L).

Tài nguyên (Resource)	Sử dụng (Used)	Khả dụng (Available)	Tỷ lệ (%)
Slice	213	33,650	0.63%
Slice LUTs (Total)	447	134,600	0.33%
— LUT as Logic	415	134,600	0.31%
— LUT as Distributed RAM	32	46,200	0.07%
Slice Registers (FFs)	573	269,200	0.21%
Block RAM Tile (RAMB36E1)	0	365	0.00%
Bonded IOB	78	400	19.50%
— OLOGIC (ODDR / Output FF)	33	400	8.25%
Global Clock Buffer (BUFGCTRL)	1	32	3.13%

Thiết kế chỉ chiếm 0.33% tổng số Slice LUTs, 0.21% thanh ghi và 0.63% số Slice của FPGA mục tiêu. Các giá trị khả dụng dùng để tính tỷ lệ được đối chiếu với thông tin tài nguyên của họ FPGA 7 Series và Artix-7 [28, 29]. Thiết kế không sử dụng DSP và Block RAM Tile, do đó các tài nguyên này vẫn sẵn sàng cho những khối mở rộng.

Phân tích định thời và công suất tiêu thụ

Báo cáo định thời tính tại tần số 50 MHz ghi nhận $WNS = +12.339$ ns và $WHS = +0.105$ ns. Cả hai giá trị đều dương và không có endpoint vi phạm, cho thấy thiết kế đạt timing closure ở cấu hình khảo sát; cách diễn giải các chỉ số slack và quy trình kiểm tra timing tuân theo phương pháp luận của Vivado [16, 38].

Theo báo cáo ước tính công suất của Vivado, tổng công suất trên chip là 0.182 W, gồm 0.051 W công suất động và 0.131 W công suất tĩnh [23]. Thành phần

Bảng 5.2: Bảng tổng hợp kết quả phân tích định thời của hệ thống SoC tại tần số 50 MHz.

Chỉ số định thời	Giá trị (ns)	Trạng thái
Worst Negative Slack (WNS)	+12.339	Đạt
Worst Hold Slack (WHS)	+0.105	Đạt
Số endpoint vi phạm	0	Đạt

Bảng 5.3: Bảng tổng hợp công suất tiêu thụ hệ thống SoC tại tần số 50 MHz.

Thành phần công suất	Công suất (W)	Tỷ lệ tổng (%)	Tỷ lệ động (%)
Công suất động	0.051	28.02	100
— Clock	0.002	1.10	4
— Signals	< 0.001	< 0.55	< 2
— Logic	< 0.001	< 0.55	< 2
— BRAM	0	0.00	0
— I/O	0.047	25.82	92
Công suất tĩnh (Device Static)	0.131	71.98	—
Tổng công suất trên chip	0.182	100.00	—

I/O tiêu thụ 0.047 W, tương ứng khoảng 92% công suất động, và là thành phần động lớn nhất của thiết kế.

5.3 Đánh giá và thảo luận

Thông qua các dữ liệu thực nghiệm trên, hệ thống SoC đạt sự cân bằng giữa hiệu suất, công suất và diện tích. Kiến trúc AXI4-Lite cung cấp giao diện ánh xạ thanh ghi với các kênh đọc/ghi tách biệt, phù hợp để điều phối nhiều mô-đun ngoại vi [5, 7]. Khối gia tốc DDR đa luồng đạt hệ số tăng tốc 2.91 lần trong kịch bản benchmark của đề tài; kết quả này phù hợp với nguyên lý tăng số lần truyền dữ liệu trên mỗi chu kỳ của liên kết DDR [10, 14].

Phương pháp căn lề trung tâm tại các chân phát dữ liệu DDR cho thấy tính thực tiễn trong cấu hình khảo sát. Việc đặt cạnh lấy mẫu vào vùng dữ liệu ổn định giúp cải thiện biên setup/hold; tuy nhiên, quan hệ pha, độ trễ đường truyền và các ràng buộc XDC vẫn phải được mô tả đầy đủ cho từng giao diện cụ thể [37–39].

5.4 Hạn chế của đề tài

Mặc dù hệ thống đáp ứng các chỉ tiêu ở cấu hình khảo sát, một số khía cạnh vẫn cần được nâng cấp. Thứ nhất, khối cộng tổ hợp trong mô-đun xử lý song song chưa được phân chia thành các tầng pipeline; khi tăng tần số, đường tổ hợp dài có thể trở thành đường tới hạn và cần được tối ưu theo quy trình timing closure [16]. Thứ hai, số liệu công suất hiện dựa trên báo cáo ước tính sau hiện thực, còn tính toán vận tín hiệu mới được đánh giá bằng mô phỏng. Các kết quả này cần được đối chiếu trên phần cứng bằng thiết bị đo phù hợp.

Chương 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Khóa luận đã nghiên cứu, thiết kế và hiện thực thành công một kiến trúc System-on-Chip (SoC) dạng mô-đun dựa trên tập lệnh mở RISC-V, tích hợp các bộ điều khiển ngoại vi truyền thông tiêu chuẩn (UART, SPI, I²C) kết hợp với cơ chế truyền dữ liệu Double Data Rate (DDR). Bằng cách khai thác hiệu quả cả hai cạnh (cạnh lên và cạnh xuống) của tín hiệu xung clock, thiết kế đề xuất đã giúp tăng gấp đôi băng thông truyền dữ liệu trên kênh giao tiếp mà không cần gia tăng tần số clock hệ thống, từ đó hạn chế rủi ro về tiêu thụ năng lượng và nhiễu điện từ (EMI). Đồng thời, cấu trúc mô-đun hóa linh hoạt tạo điều kiện thuận lợi cho việc kiểm chứng, tái sử dụng Intellectual Property (IP) và mở rộng quy mô hệ thống.

Hệ thống được mô tả bằng Verilog HDL và tổng hợp trên AMD Vivado 2025.2 cho FPGA Artix-7 xc7a200tfbg676-2L. Kết quả cho thấy thiết kế sử dụng 1,814 Slice LUTs, 1,378 Flip-Flops, 1 Block RAM Tile, 48 IOB và 1 BUFGCTRL. Trong mô phỏng chức năng ở 100 MHz, khối DDR đạt tốc độ 200 Mbps/pin. Báo cáo phân tích định thời với ràng buộc 50 MHz ghi nhận $WNS = +11.659$ ns và $WHS = +0.043$ ns, không có endpoint vi phạm; công suất toàn chip được ước tính là 0.182 W. Các kết quả này xác nhận tính khả thi của kiến trúc RISC-V SoC tích hợp giao tiếp DDR cho các ứng dụng nhúng và xử lý tại biên.

6.2 Những kết quả đạt được

Thông qua quá trình nghiên cứu và thực hiện, đề tài đã đạt được các kết quả nổi bật sau:

- Về kiến trúc: Lập trình và đóng gói thành công kiến trúc RISC-V SoC dạng mô-đun hóa, hỗ trợ chuẩn kết nối bus nội bộ linh hoạt để dễ dàng tích hợp các

khởi ngoại vi UART, SPI, I²C.

- Về kỹ thuật DDR: Tích hợp thành công khối điều khiển truyền dữ liệu theo định hướng DDR, khai thác tối đa hai cạnh xung clock giúp tối ưu hóa hiệu năng truyền dữ liệu trên các đường giao tiếp ngoại vi.
- Về hiện thực RTL: Xây dựng bộ mã nguồn RTL bằng Verilog HDL hoàn chỉnh, tuân thủ các quy tắc tổng hợp logic (Synthesizable RTL), đảm bảo tính đóng gói và khả năng tái sử dụng cao.
- Về kiểm chứng: Hoàn thành các testbench UART, SPI, I²C, DDR và tích hợp SoC; đồng thời thực hiện tổng hợp, bố trí định tuyến, phân tích timing, tài nguyên và công suất cho FPGA Artix-7 mục tiêu.
- Về tính ứng dụng: Cung cấp một nền tảng SoC phần cứng mở, có khả năng tùy biến cấu hình cao, sẵn sàng phục vụ cho các nghiên cứu phát triển hệ thống nhúng tiếp theo.

6.3 Những hạn chế

Mặc dù đã đạt được những kết quả khả quan, thiết kế của khóa luận vẫn tồn tại một số điểm hạn chế cần tiếp tục hoàn thiện:

- Tối ưu PPA (Power, Performance, Area): Thiết kế hiện tại chưa được tối ưu sâu về định thời (Timing closure) ở các mức tần số cao; mức độ tiêu thụ tài nguyên (LUTs, Flip-Flops) và công suất tiêu thụ động (Dynamic Power) vẫn còn dư địa để tối ưu.
- Cơ chế quản lý dữ liệu: Hệ thống chưa tích hợp bộ điều khiển truy xuất bộ nhớ trực tiếp (DMA), dẫn đến việc bộ xử lý trung tâm (CPU) vẫn phải can thiệp vào quá trình truyền nhận dữ liệu của ngoại vi, làm giảm hiệu năng chung của SoC.
- Xử lý miền xung clock (Clock Domain Crossing - CDC): Việc đồng bộ hóa dữ liệu giữa miền xung clock hệ thống và miền xung clock DDR ngoại vi còn đơn giản, cần các giải pháp CDC nâng cao để triệt tiêu hoàn toàn rủi ro trạng thái bất định (Metastability).

- Triển khai trên mạch thực: Mã nguồn RTL hiện mới được mô phỏng, tổng hợp và phân tích sau hiện thực trên thiết bị mục tiêu xc7a200tfbg676-2L. Do chưa có bo mạch FPGA vật lý tương thích, thiết kế chưa thể được đóng gói hoàn chỉnh, gắn chân theo phần cứng thực, tạo tệp bitstream và nạp trực tiếp lên bo mạch để kiểm chứng hoạt động.
- Môi trường thực nghiệm: Đánh giá hiện dựa trên mô phỏng RTL và báo cáo Implementation của Vivado, chưa đo dạng sóng vật lý bằng Logic Analyzer hoặc Oscilloscope trong điều kiện môi trường công nghiệp.

6.4 Hướng phát triển trong tương lai

Để hoàn thiện và nâng cao tính thương mại hóa của thiết kế, các hướng nghiên cứu tiếp theo sẽ tập trung vào:

- Tích hợp khối DMA và nâng cấp Bus nội bộ: Tích hợp bộ điều khiển DMA đa kênh và chuyển đổi chuẩn kết nối bus sang các giao thức công nghiệp chuẩn như AMBA AXI4/AHB-Lite nhằm giảm tải tối đa cho CPU.
- Tối ưu hóa thiết kế vi mạch: Áp dụng các kỹ thuật thiết kế công suất thấp (Low-power design techniques) như Power Gating, Clock Gating, đồng thời tối ưu đường truyền tới các ô nguyên thủy (Primitives) DDR chuyên dụng trên phần cứng.
- Thực hiện quy trình thiết kế ASIC (ASIC Flow): Mở rộng từ FPGA sang quy trình vi mạch ASIC hoàn chỉnh, bao gồm tổng hợp logic với thư viện chuẩn (Standard Cell Library), phân tích định thời tĩnh (STA), thiết kế vật lý (Physical Design - P&R) và đánh giá công suất tiêu thụ sau đóng gói layout.
- Tích hợp bộ gia tốc phần cứng: Mở rộng SoC bằng cách thêm các bộ gia tốc mã hóa/giải mã dữ liệu hoặc bộ gia tốc tính toán AI nhẹ tại biên (TinyML) kết nối qua giao tiếp DDR.

Chương 7

CÁC CÔNG TRÌNH NGHIÊN CỨU ĐÃ THAM GIA

Trong quá trình thực hiện khóa luận, tác giả đã tham gia các công trình và hoạt động nghiên cứu sau:

1. Binh Nguyen, Chan Nguyen, Sang Ha, Huy Ngo, Ho Le Minh Toan và Thien Pham, “An implementation of Double Data Rate in System on Chip using Risc-V architecture for high speed communication applications,” *Journal of King Saud University – Engineering Sciences*, Springer Nature, 2026. Trạng thái bài báo: Đã nộp.
2. Tham gia *Cuộc thi Thiết kế Vi mạch cho Đô thị thông minh* với đề tài “Nghiên cứu và thiết kế hệ thống ngoại vi tốc độ cao dựa trên kỹ thuật Double Data Rate (DDR) tích hợp trên SoC RISC-V.”
3. Huy Ngo và các đồng tác giả, “A High-Throughput Streaming Hardware Accelerator for Real-Time Image Hole Filling on FPGA,” 2026.
 - Bản thảo mang mã IEEJ-2026-0860 trên hệ thống của *The Institute of Electrical Engineers of Japan* (IEEJ).
 - Bài báo tham gia *The 7th International Conference on Engineering, Physics, MEMS-Biosensors and Applications* (ICEBA 2026), tổ chức từ ngày 2 đến ngày 4 tháng 11 năm 2026 tại Thành phố Hồ Chí Minh và Vũng Tàu, Việt Nam.

Trạng thái bài báo: Đã nộp.

TÀI LIỆU THAM KHẢO

- [1] D. Petrisko *et al.*, “BlackParrot: An agile open-source RISC-V multicore for accelerator SoCs,” *IEEE Micro*, vol. 40, no. 4, pp. 93–102, 2020.
- [2] W. Huang and G. Sheng, “Analysis and research on UART communication protocol,” in *Proc. 4th Asia-Pacific Conf. Communications Technology and Computer Science (ACCTCS)*, 2024, pp. 768–771.
- [3] G. B. Wakhle, I. Aggarwal, and S. Gaba, “Synthesis and implementation of UART using VHDL codes,” in *Proc. International Symposium on Computer, Consumer and Control*, 2012, pp. 1–3.
- [4] R. Xie, “Design and simulation of UART protocol based on FPGA,” in *Proc. 6th International Conf. Applied Machine Learning (ICAML)*, 2024, pp. 551–557.
- [5] D. Emil, M. Hamdy, and G. Nagib, “Development an efficient AXI-interconnect unit between set of customized peripheral devices and an implemented dual-core RISC-V processor,” *The Journal of Supercomputing*, vol. 79, pp. 17000–17019, 2023.
- [6] AMD, “AXI Basics 6 – Introduction to AXI4-Lite in Vitis HLS,” *AMD Adaptive Support*, Feb. 16, 2023. Available: https://adaptivesupport.amd.com/s/article/1137153?language=en_US.
- [7] Arm Limited, *AMBA AXI and ACE Protocol Specification: AXI3, AXI4, and AXI4-Lite*, IHI 0022. Available: <https://developer.arm.com/documentation/ih0022/latest/>.
- [8] R. Shashikumar, C. N. Vijay Kumar, M. Nagendrakumar, and C. S. Hemanthkumar, “AHB compatible DDR SDRAM controller IP core for ARM based SoC,” *International Journal of Computer Science and Information*

- Security*, vol. 7, no. 1, pp. 77–85, 2010. Available: <https://arxiv.org/pdf/1002.1953>.
- [9] A. Ejaz, V. Papaefstathiou, and I. Sourdis, “DDRNoC: Dual data-rate network-on-chip,” *ACM Transactions on Architecture and Code Optimization*, vol. 15, no. 2, pp. 25:1–25:24, 2018.
- [10] A. Psarras, S. Moisisdis, C. Nicopoulos, and G. Dimitrakopoulos, “Networks-on-chip with double-data-rate links,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 64, no. 12, pp. 3103–3114, 2017.
- [11] A. Olgun *et al.*, “PiDRAM: An FPGA-based framework for end-to-end evaluation of processing-in-DRAM techniques,” in *Proc. IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2022, pp. 267–272.
- [12] YosysHQ, “PicoRV32: A Size-Optimized RISC-V CPU,” GitHub repository. Available: <https://github.com/YosysHQ/picorv32>.
- [13] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, ver. 20260120, 2026. Available: <https://docs.riscv.org/reference/isa/v20260120/unpriv/unpriv-index.html>.
- [14] JEDEC Solid State Technology Association, “Main Memory: DDR SDRAM Standards.” Available: <https://www.jedec.org/category/technology-focus-area/main-memory-ddr3-ddr4-sdram>.
- [15] J. Zhou, “How to enable DDR mode,” *NXP Community*, May 27, 2021. Available: <https://community.nxp.com/t5/i-MX-RT-Crossover-MCUs-Knowledge/How-to-enable-DDR-mode/ta-p/1283331>.
- [16] AMD, *UltraFast Design Methodology Guide for FPGAs and SoCs*, UG949, ver. 2026.1, 2026. Available: <https://docs.amd.com/r/en-US/ug949-vivado-design-methodology>.
- [17] Texas Instruments, *UART Protocol Overview and Error Sources*, TI Precision Labs. Available: https://www.ti.com/content/dam/videos/external-videos/zh-tw/9/3816841626001/6313217959112.mp4/subassets/uart_protocol_overview_and_error_sources_0.pdf.

- [18] S. Campbell, “Basics of UART communication,” *Circuit Basics*. Available: <https://www.circuitbasics.com/basics-uart-communication/>.
- [19] NXP Semiconductors, *I2C-bus Specification and User Manual*, UM10204, rev. 7.0, 2021. Available: https://www.nxp.com/documents/user_manual/UM10204.pdf.
- [20] Riverdi, “What is I2C protocol? I2C communication protocol,” *Riverdi Blog*. Available: <https://riverdi.com/fr/blog/what-is-i2c-protocol-i2c-communication-protocol>.
- [21] P. Dhaker, “Introduction to SPI Interface,” *Analog Dialogue*, Analog Devices. Available: <https://www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html>.
- [22] Digilent, “What is Serial Peripheral Interface (SPI)?” *Digilent Blog*. Available: <https://digilent.com/blog/what-is-spi/>.
- [23] AMD, *Vivado Design Suite Documentation*, UG1793, 2025. Available: <https://docs.amd.com/v/u/en-US/ug1793-vivado-design-suite>.
- [24] Flathub, “Xilinx Vivado Design Suite,” application page. Available: <https://flathub.org/vi/apps/com.github.corna.Vivado>.
- [25] AMD, *Zynq-7000 SoC Data Sheet: Overview*, DS190, rev. 1.11.1, 2018. Available: <https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview>.
- [26] AMD, *Zynq-7000 SoC Technical Reference Manual*, UG585, ver. 1.15, 2026. Available: <https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM>.
- [27] Electromaker, “SoC module with AMD Zynq 7030-1I, 1 Gbyte DDR3L, 32 Mbyte Flash, 4 x 5 cm—TE0715-05-71I33-A,” product page. Available: Electromaker product page.
- [28] AMD, *7 Series FPGAs Data Sheet: Overview*, DS180, rev. 2.6.1, 2020. Available: https://docs.amd.com/v/u/en-US/ds180_7Series_Overview.

- [29] AMD, *Artix 7 FPGAs Data Sheet: DC and AC Switching Characteristics*, DS181, rev. 1.27.1, 2024. Available:
https://docs.amd.com/v/u/en-US/ds181_Artix_7_Data_Sheet.
- [30] V. Maisto, S. Mercogliano, M. Maddaluno, and A. Cilardo, “The Simply-V framework: An extensible RISC-V reconfigurable soft-SoC for open research and fast prototyping,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 31, no. 3, pp. 1–32, 2026. Available:
<https://dl.acm.org/doi/10.1145/3787500>.
- [31] K.-M. Ma, D.-H. Le, C.-K. Pham, and T.-T. Hoang, “Design of an SoC based on 32-bit RISC-V processor with low-latency lightweight cryptographic cores in FPGA,” *Future Internet*, vol. 15, no. 5, Art. no. 186, 2023. Available:
<https://www.mdpi.com/1999-5903/15/5/186>.
- [32] H. Hassan, N. Vijaykumar, S. Khan, S. Ghose, K. Chang, G. Pekhimenko, D. Lee, O. Ergin, and O. Mutlu, “SoftMC: Practical DRAM characterization using an FPGA-based infrastructure,” arXiv:1805.03195, 2018. Available:
<https://arxiv.org/pdf/1805.03195>.
- [33] S. Wijeratne, S. Pattnaik, Z. Chen, R. Kannan, and V. Prasanna, “Programmable FPGA-based memory controller,” in *Proc. 28th IEEE Symposium on High-Performance Interconnects (HOTI)*, 2021. Available:
<https://arxiv.org/pdf/2108.09601>.
- [34] A. C. Bonatto, A. B. Soares, and A. A. Susin, “Design and test of a DDR SDRAM interface for FPGA systems,” in *Proc. VIII Student Forum on Microelectronics*, 2008. Available:
<https://sbmicro.org.br/sforum-eventos/sforum2008/38.pdf>.
- [35] B. Naresh, S. Rambabu, and G. Lakshmi Narayana, “Implementation of DDR SDRAM memory controller for embedded SOC,” *International Journal of Advanced Trends in Computer Science and Engineering*, vol. 5, no. 1, pp. 101–105, 2016, Special Issue of ICACEC 2016. Available:
<https://warse.org/IJATCSE/static/pdf/Issue/icacec2016sp22.pdf>.
- [36] L. Chisciotti, *Design of a High-Speed Data Buffer Based on DDR Memories for Real-Time Processing of Hyperspectral Data and Its Implementation in a*

- Radiation-Tolerant FPGA*, M.S. thesis, Politecnico di Torino, Turin, Italy, 2022. Available: <https://webthesis.biblio.polito.it/23437/1/tesi.pdf>.
- [37] V. Arumugham, “DDR timing,” contribution to the IEEE P802.3ae 10 Gigabit Ethernet Task Force, Nov. 30, 2000. Available: https://grouper.ieee.org/groups/802/3/ae/comments/d2.0/arumugham_1_0101.pdf.
- [38] AMD, *Vivado Design Suite User Guide: Using Constraints*, UG903, ver. 2025.1, 2025. Available: <https://docs.amd.com/r/2025.1-English/ug903-vivado-using-constraints/About-XDC-Constraints>.
- [39] AMD, *7 Series FPGAs SelectIO Resources User Guide*, UG471, ver. 1.10, 2018. Available: https://docs.amd.com/v/u/en-US/ug471_7Series_SelectIO.